



Распределенное обучение: Data-Parallel, All-Reduce

Лекция 8
2026, Дмитрий Нестеров

План лекции

01

Бэкграунд и мотивация

Зачем нам распределенное обучение?

02

Виды параллелизма

Обзорно: DP, PP, TP, SP

03

Data Parallelism

Разбираем Parameter Server

04

Коллективные операции

Эффективный All-Reduce

05

Оптимизация памяти в DP

ZeRO-1/2/3 и FSDP



План лекции

01

Бэкграунд и мотивация

Зачем нам распределенное обучение?

02

Виды параллелизма

Обзорно: DP, PP, TP, SP

03

Data Parallelism

Разбираем Parameter Server

04

Коллективные операции

Эффективный All-Reduce

05

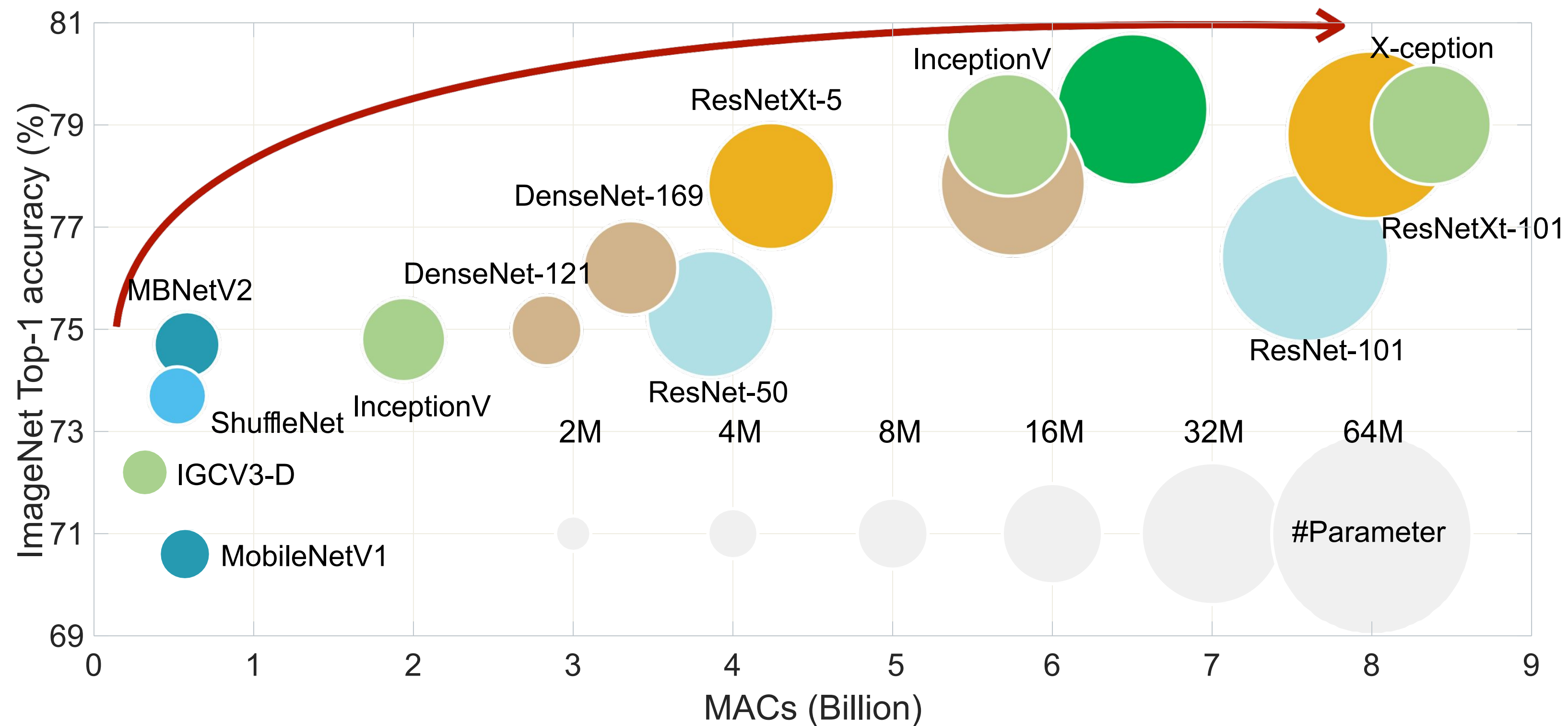
Оптимизация памяти в DP

ZeRO-1/2/3 и FSDP



Модели становятся все больше

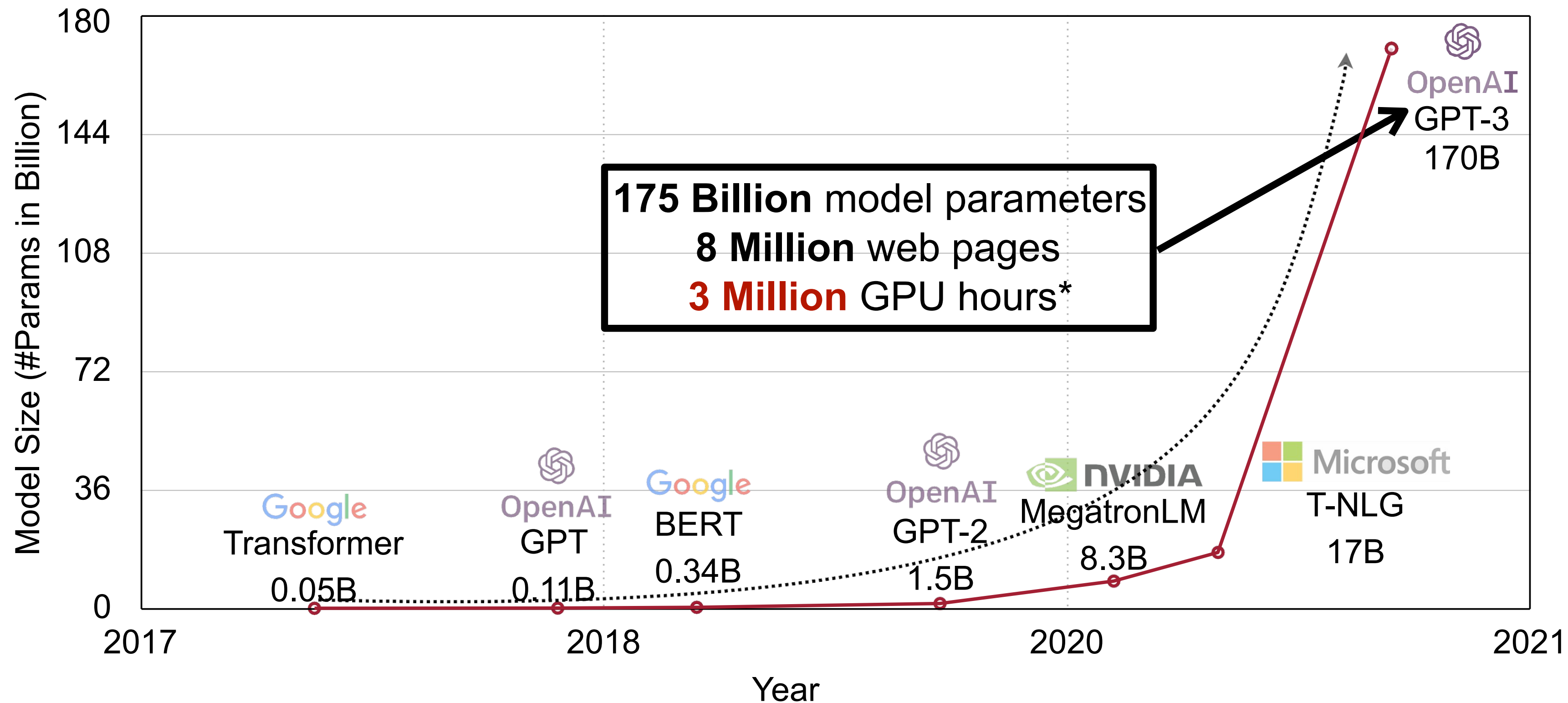
Более качественные модели сопряжены с более высокими вычислительными затратами (CV)



MAC = multiply-accumulate

Модели становятся все больше

Более качественные модели сопряжены с более высокими вычислительными затратами (NLP)



Модели становятся все больше

Обучение больших моделей занимает больше времени

Models	#Params (M)	Training Time (GPU Hours)
ResNet-50	26	31
ResNet-101	45	44
BERT-Base	108	84
Turing-NLG 17B	17,000	TBA
GPT-3 175B	175,000	3,100,000

На карте Nvidia A100

Без распределенного обучения потребовалось бы 355 лет, чтобы обучить GPT-3!

Модели становятся все больше

Обучение больших моделей занимает больше времени



Тимлид: Что ты сделал за месяц?

Модели становятся все больше

Обучение больших моделей занимает больше времени



Тимлид: Что ты сделал за месяц?

Ты: Я обучал модель, одну эпоху.



Модели становятся все больше

Обучение больших моделей занимает больше времени



Тимлид: Что ты сделал за месяц?

Ты: Я обучал модель, одну эпоху.



Тимлид: Эмм, хорошо, а какие у тебя планы на следующий месяц?

Модели становятся все больше

Обучение больших моделей занимает больше времени



Тимлид: Что ты сделал за месяц?

Ты: Я обучал модель, одну эпоху.



Тимлид: Эмм, хорошо, а какие у тебя планы на следующий месяц?

Ты: Ну... обучу еще одну эпоху?



Модели становятся все больше

Обучение больших моделей занимает больше времени



Тимлид: Что ты сделал за месяц?

Ты: Я обучал модель, одну эпоху.



Тимлид: Эмм, хорошо, а какие у тебя планы на следующий месяц?

Ты: Ну... обучу еще одну эпоху?



Распределенное обучение необходимо

- Время разработчиков / исследователей **ценнее** железа
- Если обучение занимает **10 дней на одной GPU**
 - Параллелим его с помощью распределенного обучения
 - 1024 GPU могут завершить обучение за 14 минут (в идеале)!
- Цикл разработки и исследований значительно ускорится

Распределенное обучение необходимо

- Время разработчиков / исследователей **ценнее** железа
- Если обучение занимает **10 дней на одной GPU**
 - Параллелим его с помощью распределенного обучения
 - 1024 GPU могут завершить обучение за 14 минут (в идеале)!
- Цикл разработки и исследований значительно ускорится

Давайте рассмотрим кейс с распределенным обучением!

Пример large-scale распределенного обучения

SUMMIT Supercomputer (2019)



- CPU: 2 x 16 Core IBM POWER9 (connected via dual NVLINK bricks, 25GB/s each side)
- GPU: 6 x NVIDIA Tesla V100
- RAM: 512 GB DDR4 memory
- Storage: 250 PB
- Connection: Dual-rail EDR InfiniBand network of 200 Gbit/s

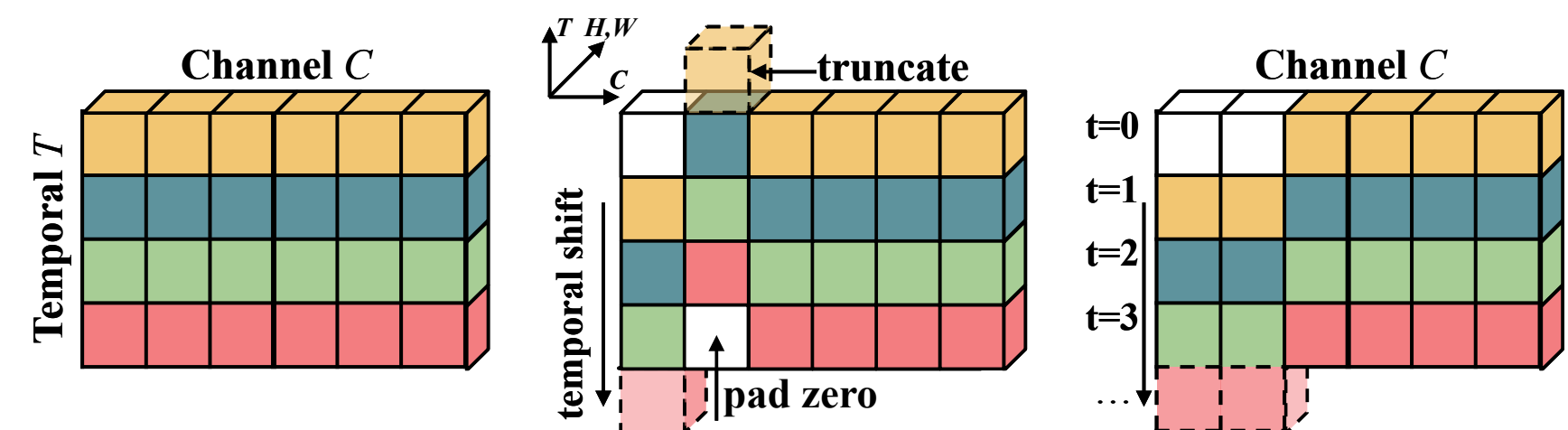
Пример large-scale распределенного обучения

Обучение видео-модели (TSM) на 660 часах видео

Models	Training Time	Accuracy
1 SUMMIT Nodes (6 GPUs)	49h 50min	74.1%
128 SUMMIT Nodes (768 GPUs)	28min	74.1%
256 SUMMIT Nodes (1536 GPUs)	14min (211x)	74.0%

Ускорение обучения 200x, с 2 дней до 14 минут.

- **Модель:** 8-frame ResNet-50 TSM for video recognition
- **Датасет:** Kinetics (240k training videos) x 100 epoch



План лекции

01

Бэкграунд и мотивация

Зачем нам распределенное обучение?

02

Виды параллелизма

Обзорно: DP, PP, TP, SP

03

Data Parallelism

Разбираем Parameter Server

04

Коллективные операции

Эффективный All-Reduce

05

Оптимизация памяти в DP

ZeRO-1/2/3 и FSDP

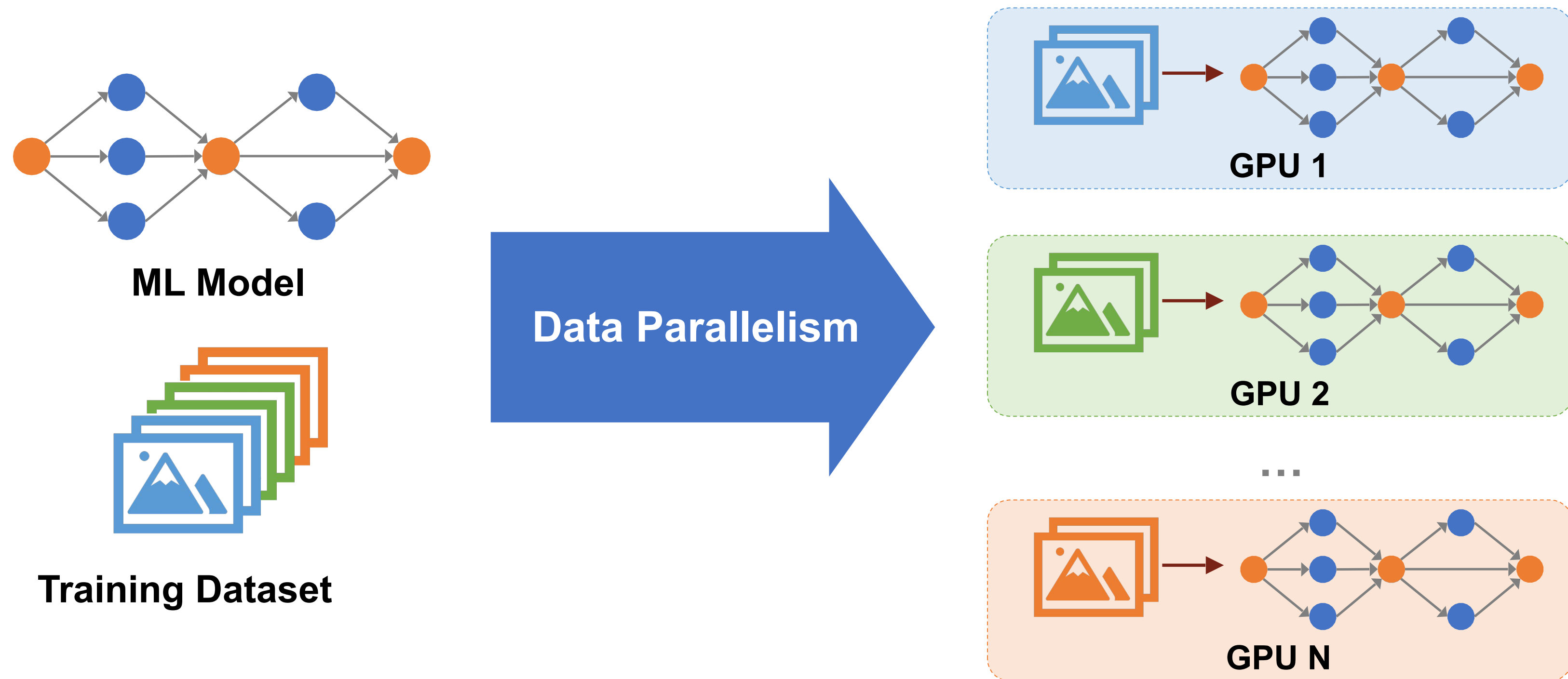


Виды параллелизма в распределенном обучении

- **Data Parallelism**
- Pipeline Parallelism
- Tensor Parallelism
- Sequence Parallelism

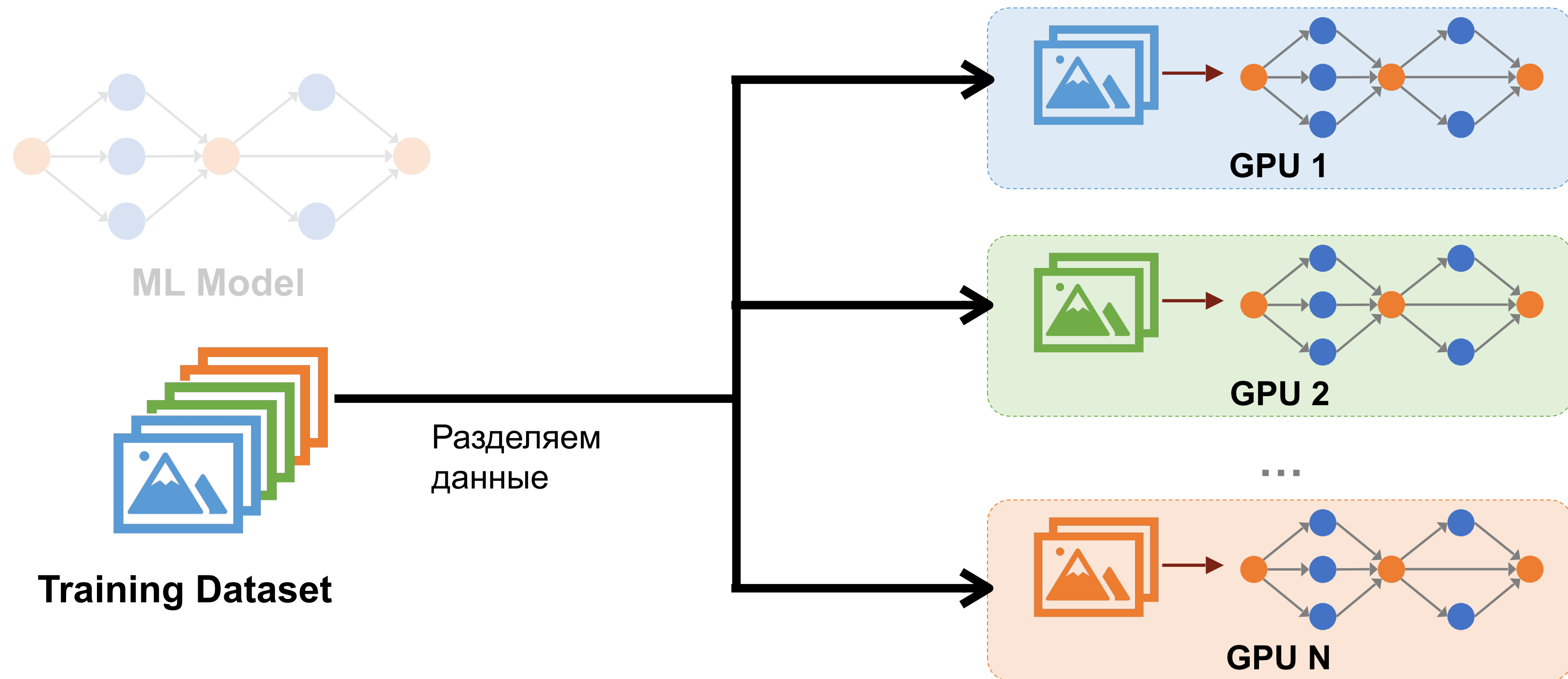
Виды параллелизма в распределенном обучении

Data Parallelism



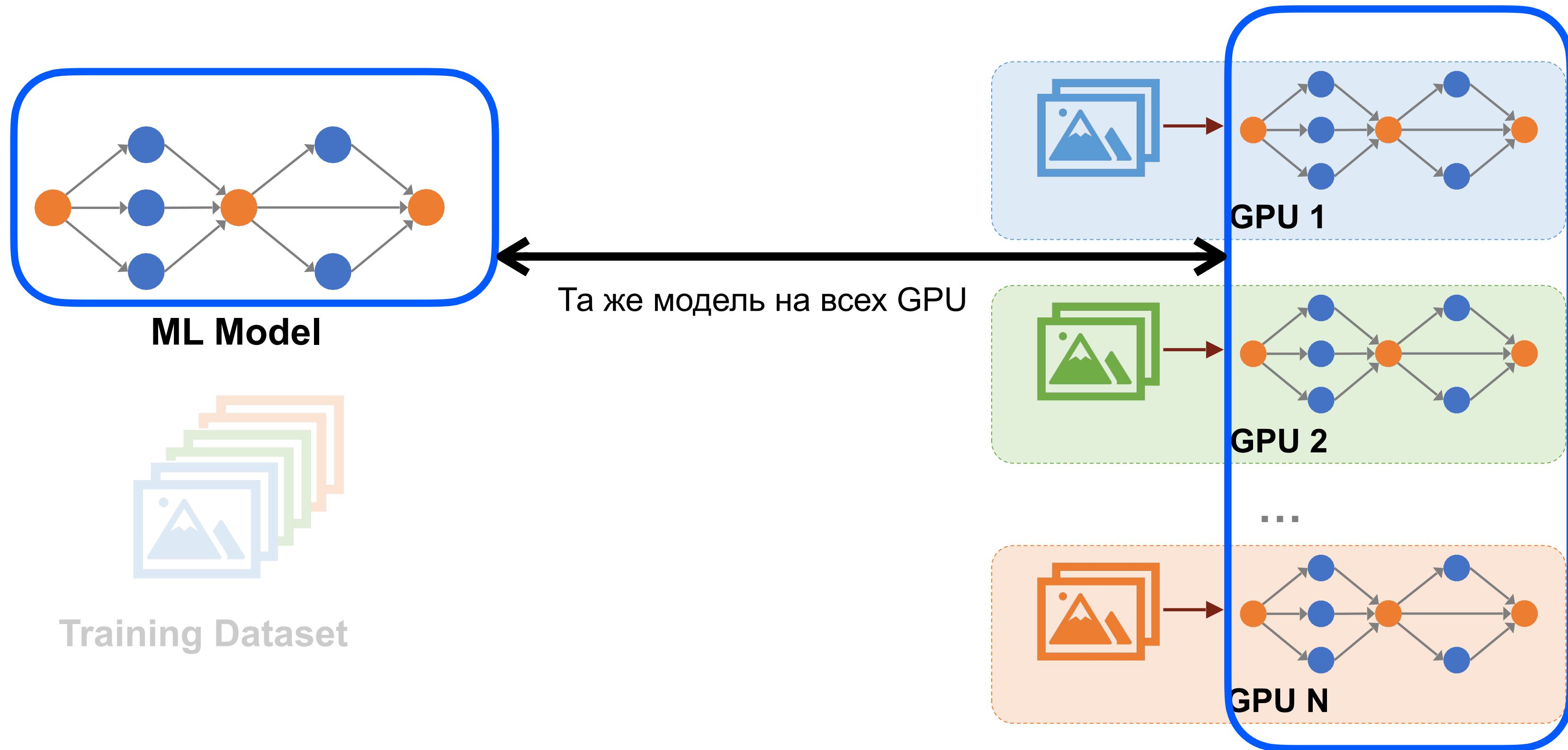
Виды параллелизма в распределенном обучении

Data Parallelism



Виды параллелизма в распределенном обучении

Data Parallelism



Виды параллелизма в распределенном обучении

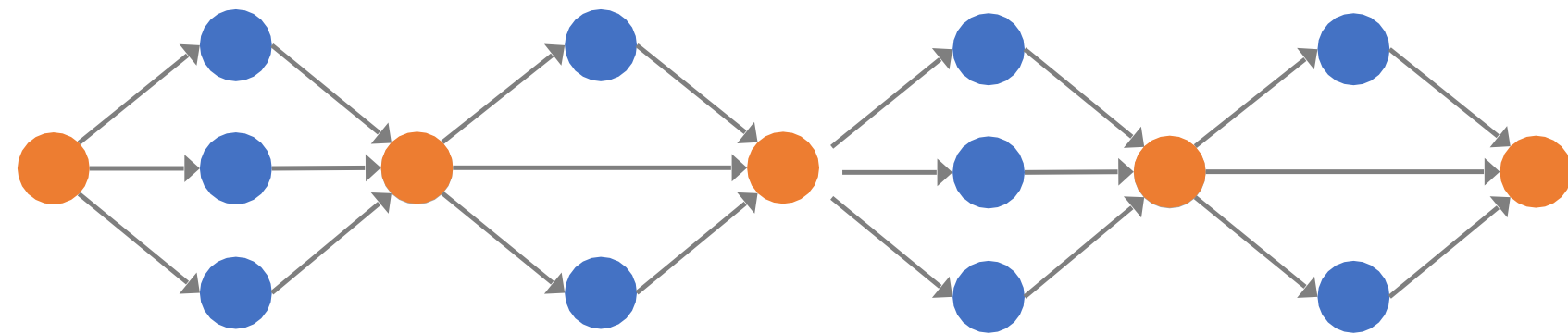
- Data Parallelism
- **Pipeline Parallelism**
- Tensor Parallelism
- Sequence Parallelism

Виды параллелизма в распределенном обучении

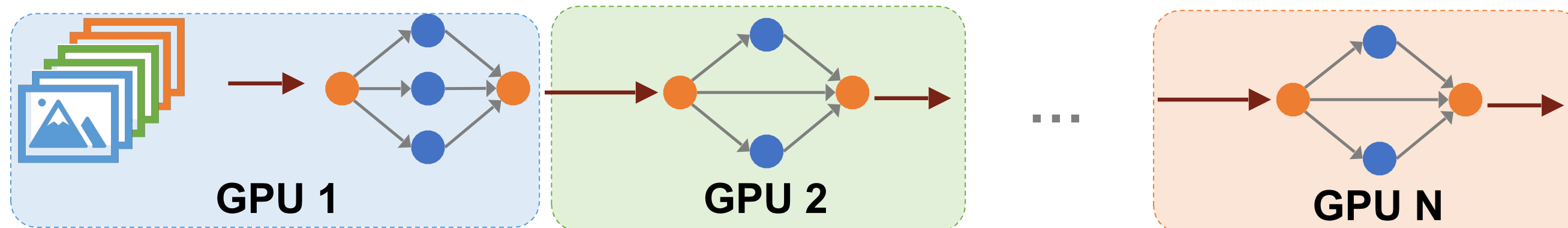
Pipeline Parallelism



Training Dataset

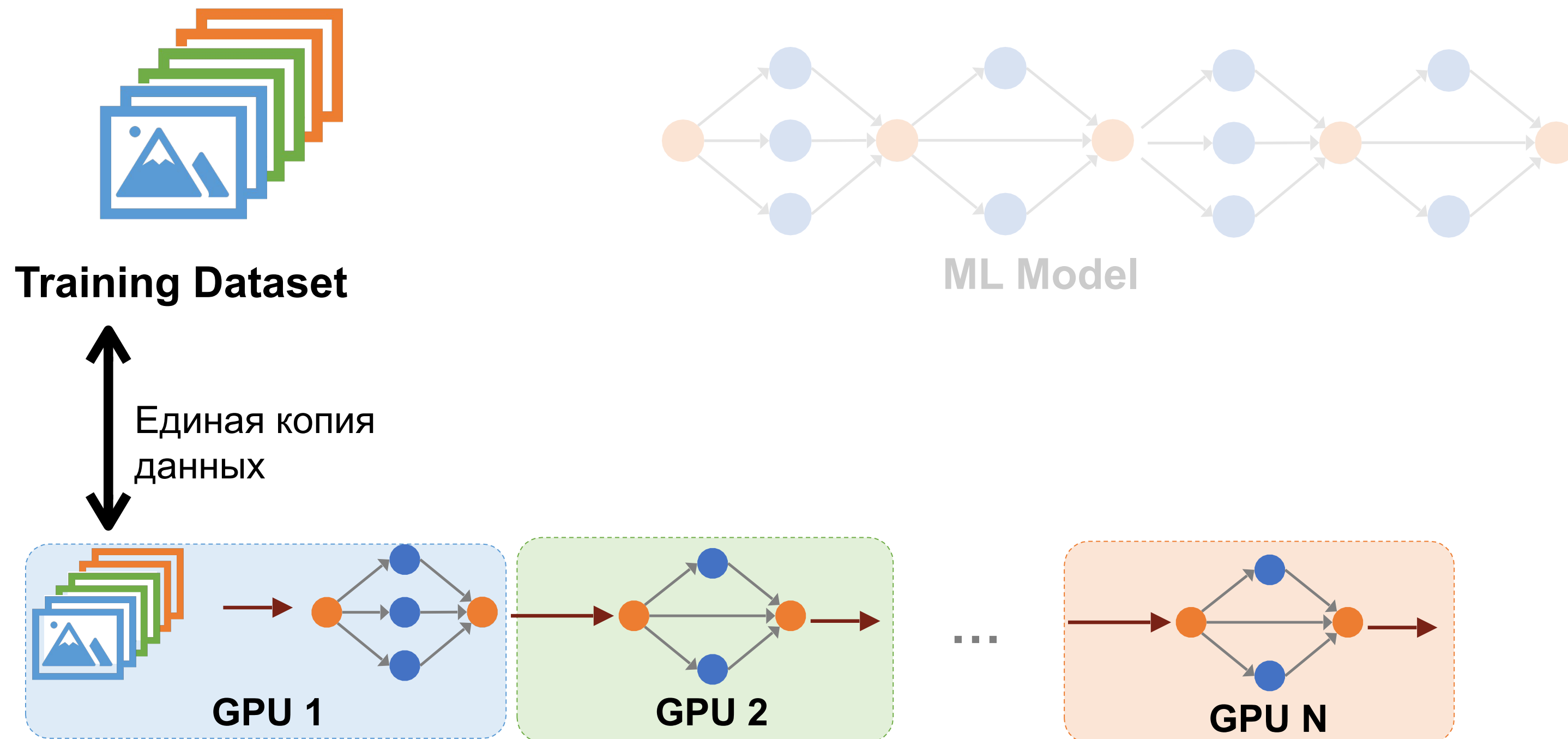


ML Model



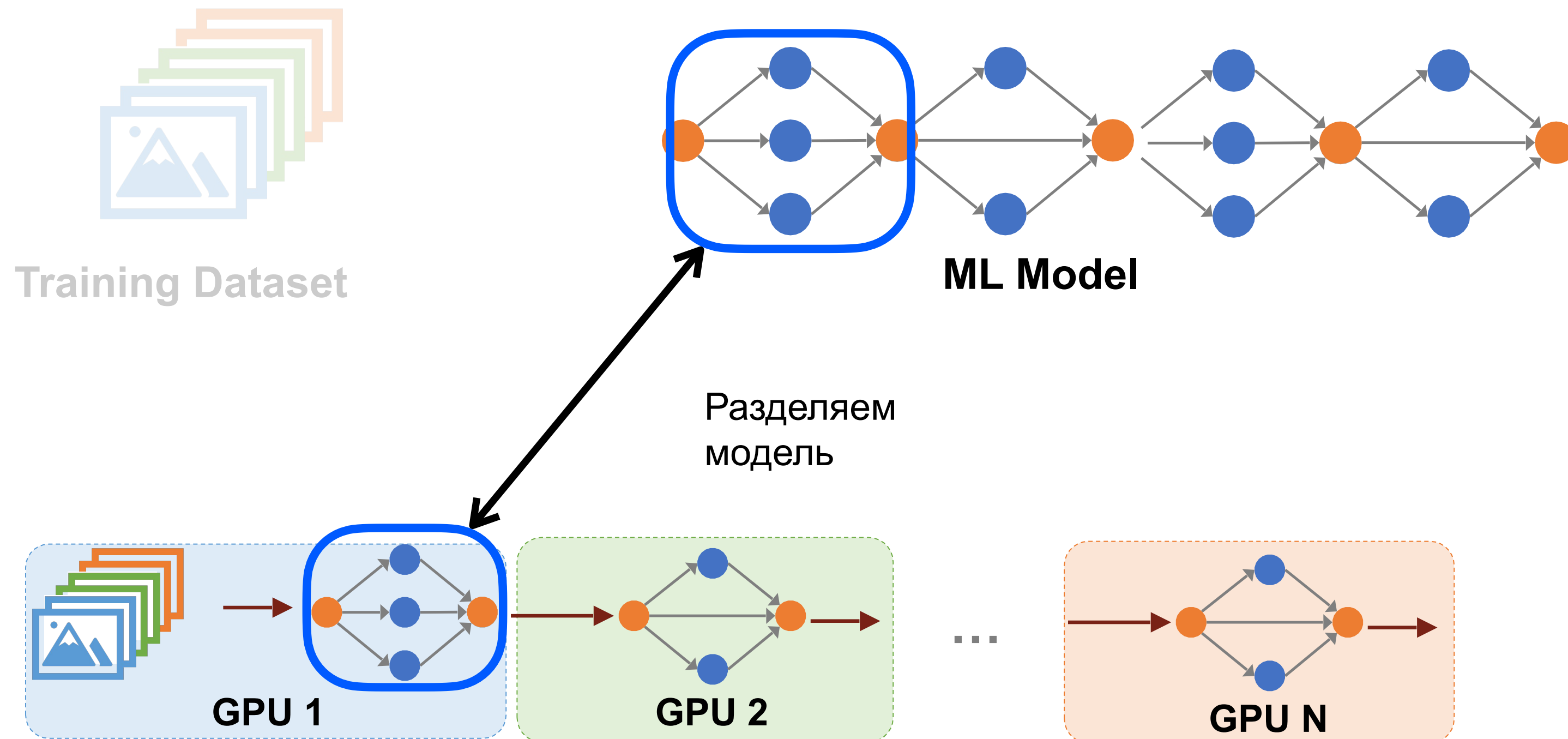
Виды параллелизма в распределенном обучении

Pipeline Parallelism



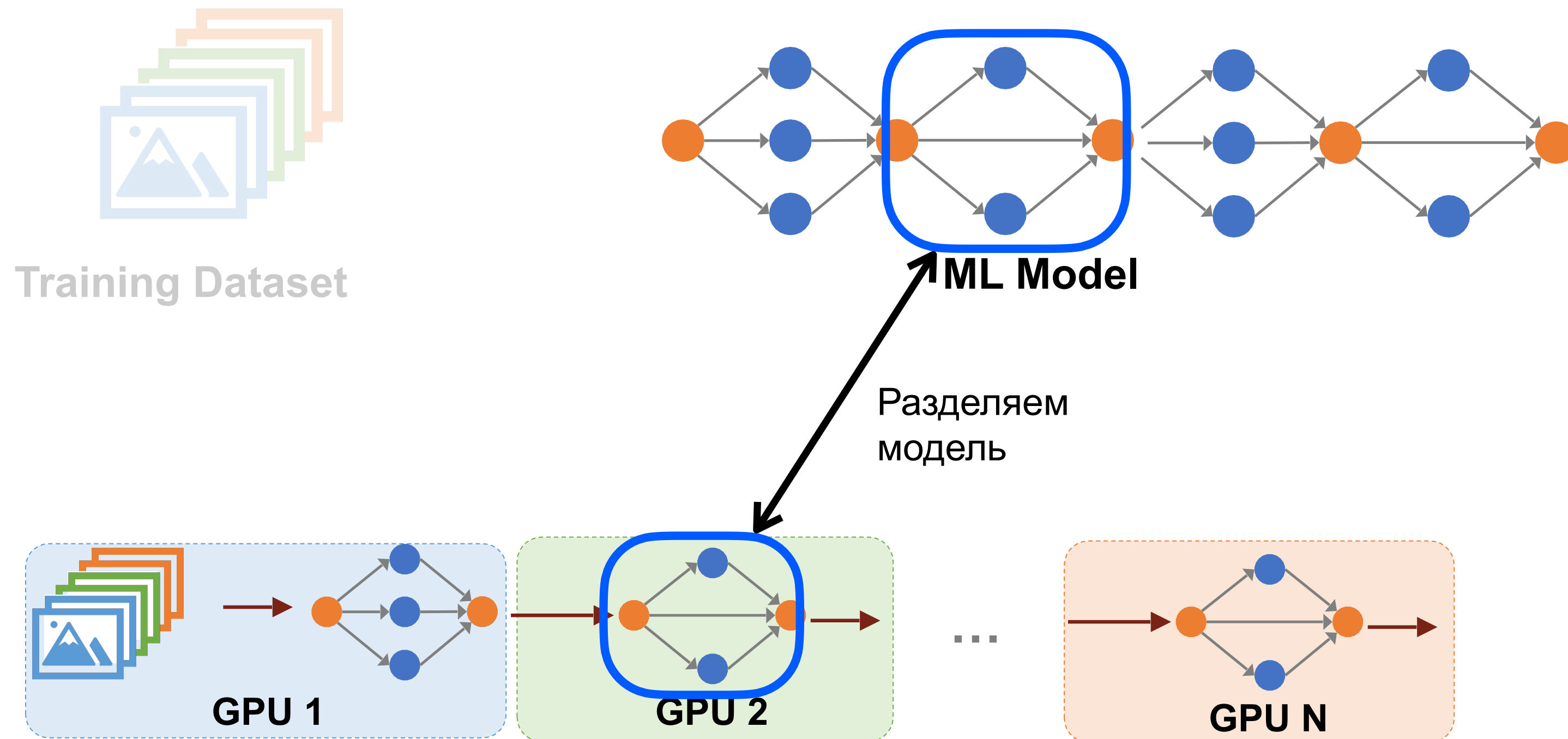
Виды параллелизма в распределенном обучении

Pipeline Parallelism



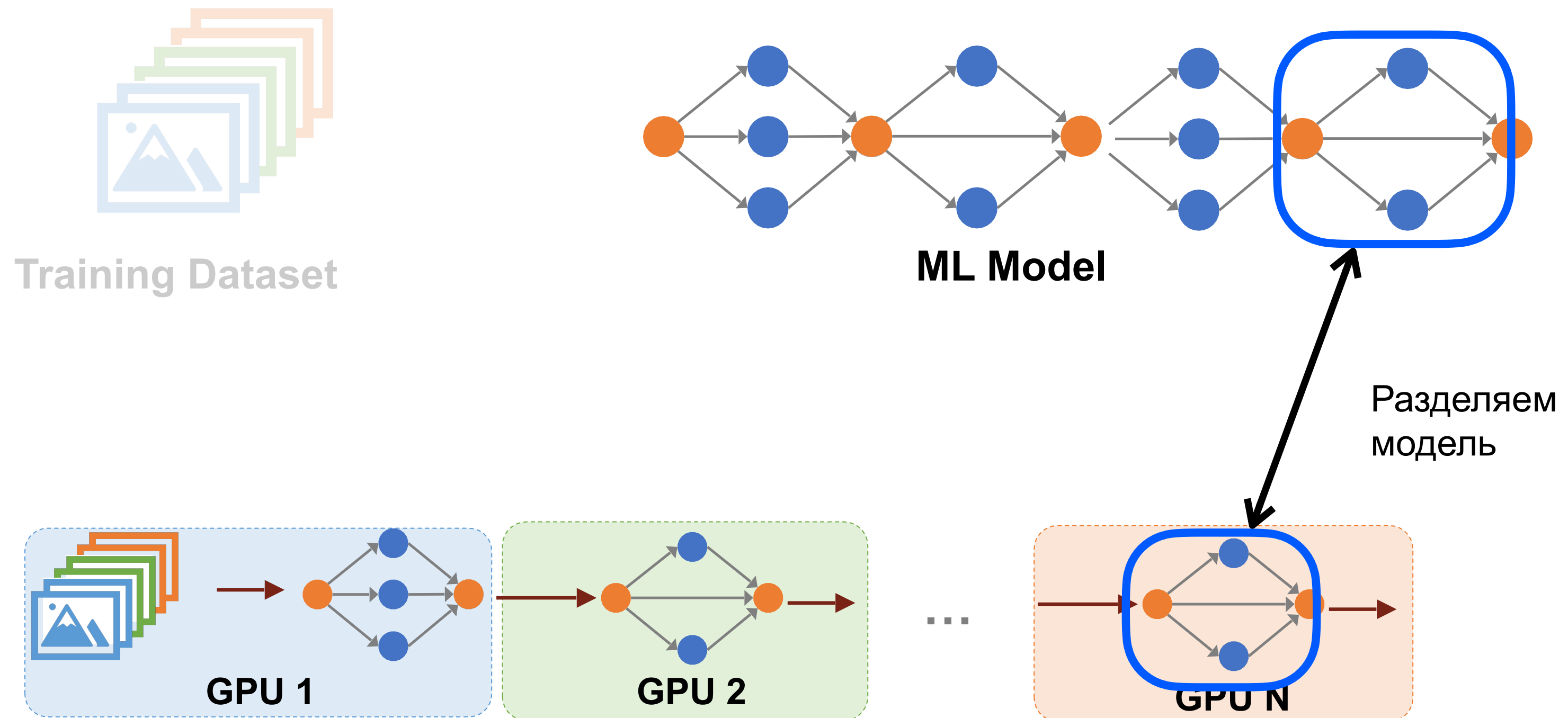
Виды параллелизма в распределенном обучении

Pipeline Parallelism



Виды параллелизма в распределенном обучении

Pipeline Parallelism



Виды параллелизма в распределенном обучении

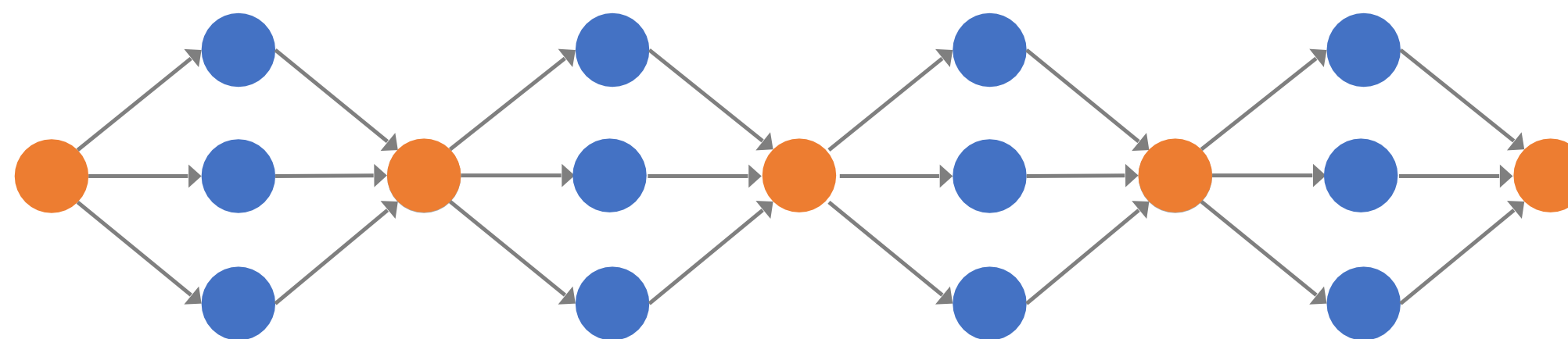
- Data Parallelism
- Pipeline Parallelism
- **Tensor Parallelism**
- Sequence Parallelism

Виды параллелизма в распределенном обучении

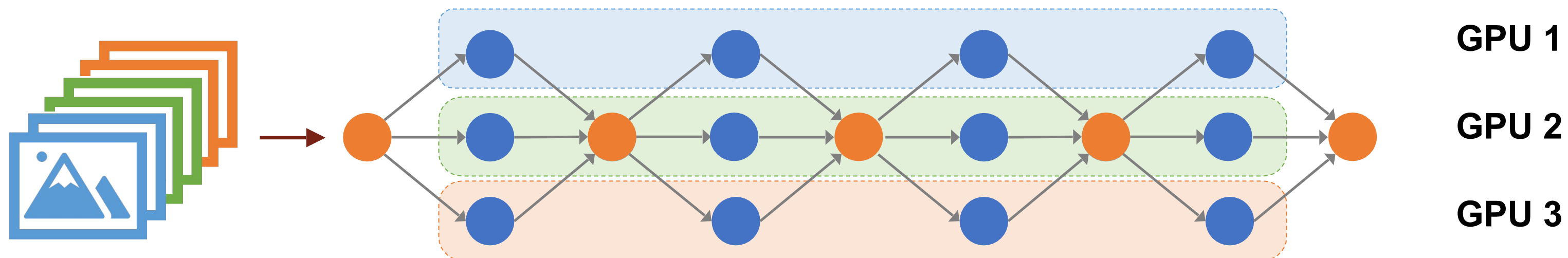
Tensor Parallelism



Training Dataset



ML Model

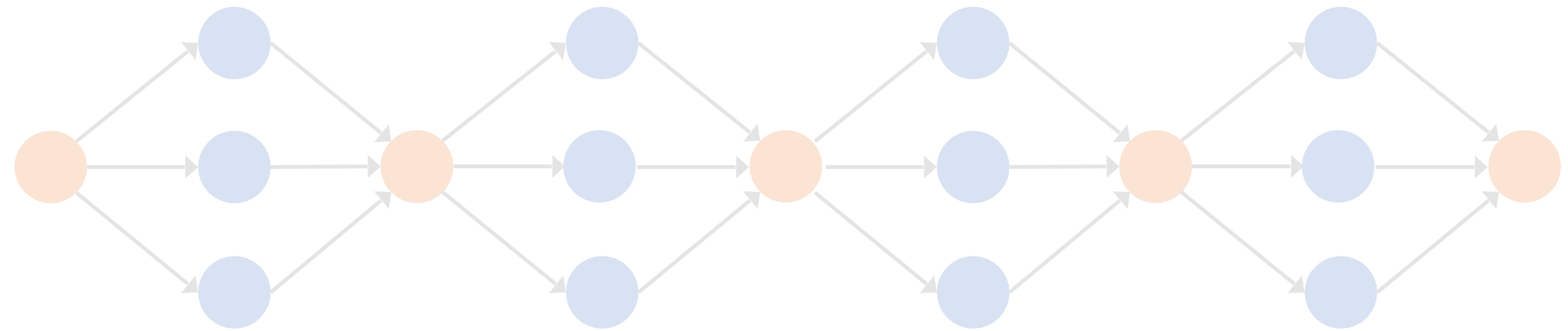


Виды параллелизма в распределенном обучении

Tensor Parallelism

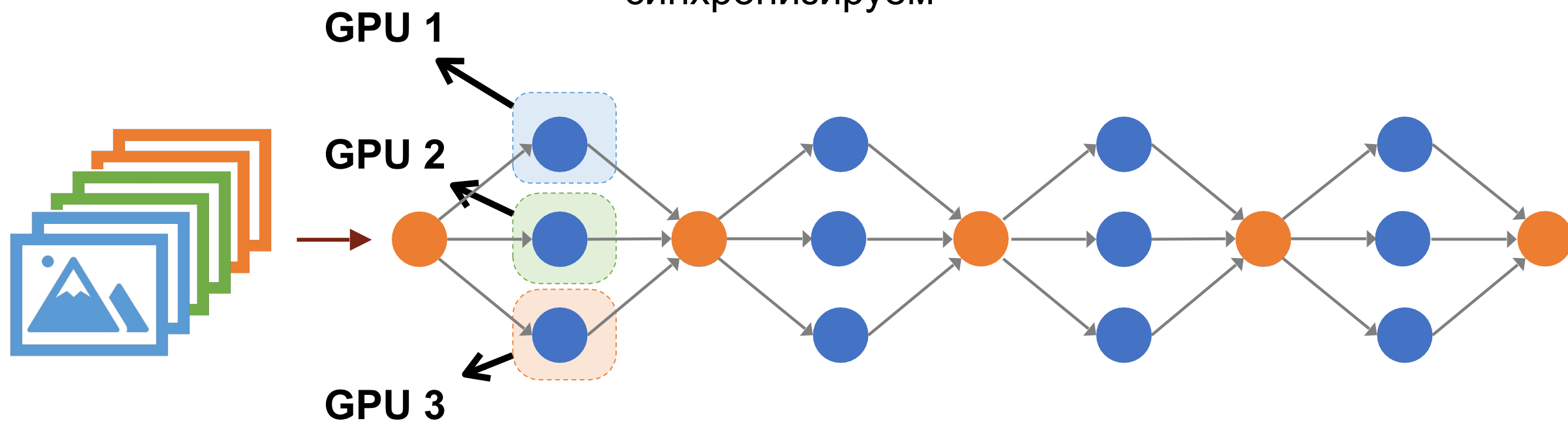


Training Dataset



ML Model

Отдельно считаем активации, затем синхронизируем

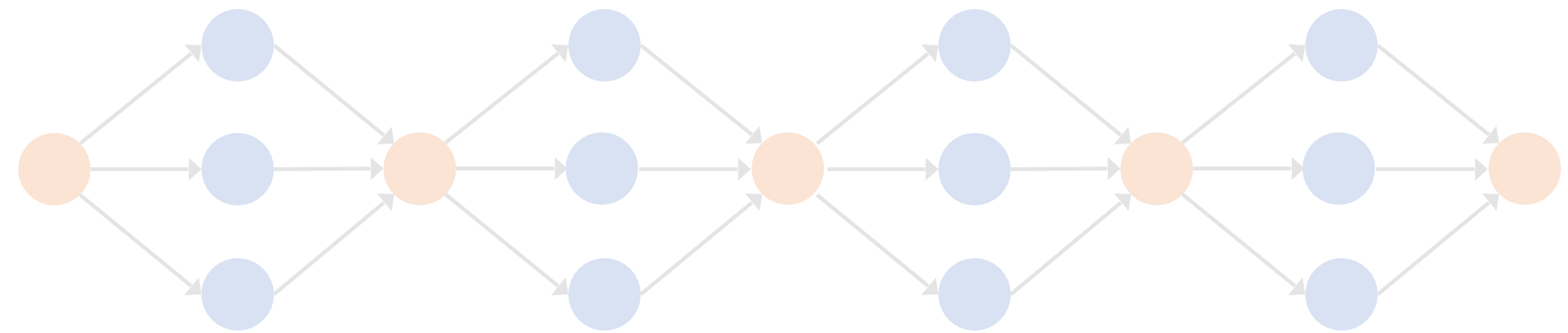


Виды параллелизма в распределенном обучении

Tensor Parallelism

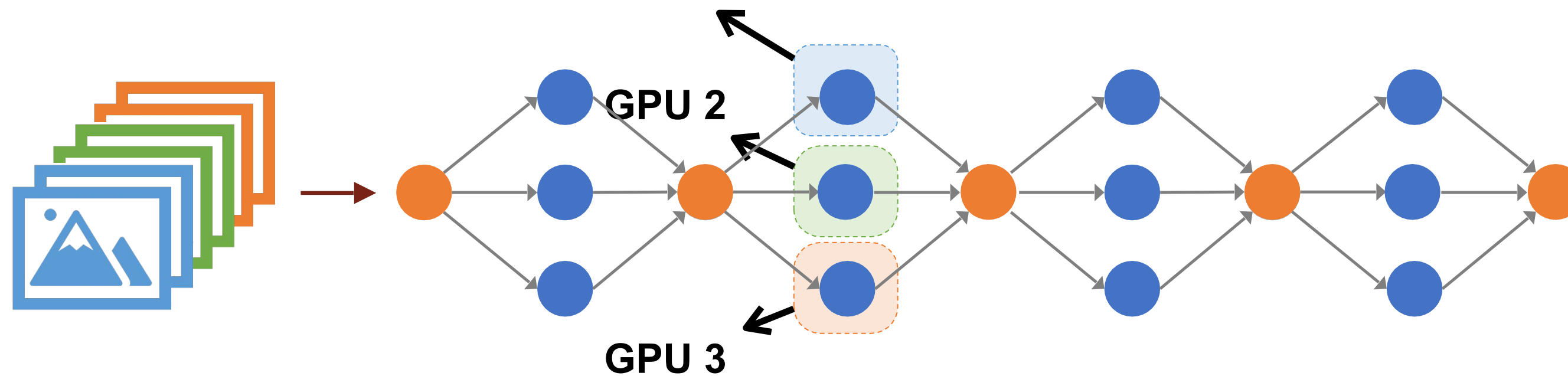


Training Dataset



ML Model

Отдельно считаем активации, затем синхронизируем

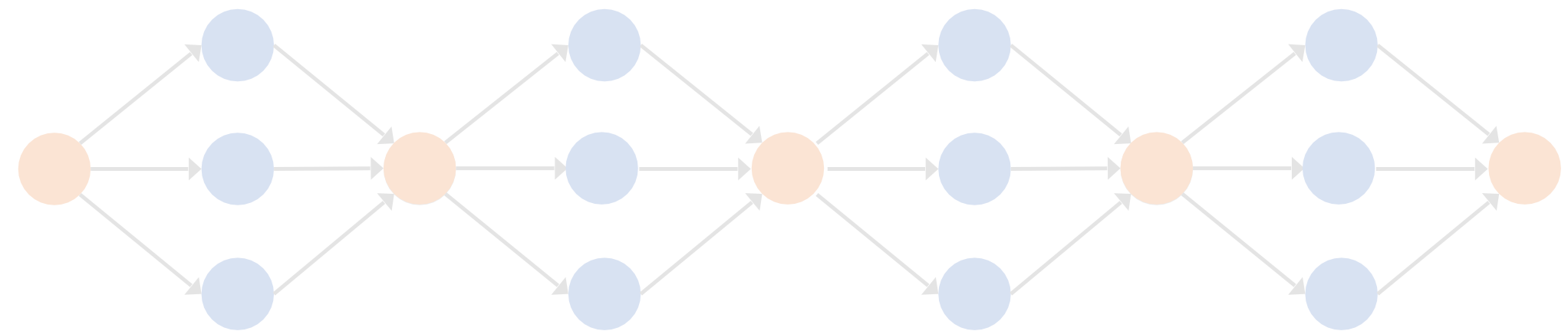


Виды параллелизма в распределенном обучении

Tensor Parallelism

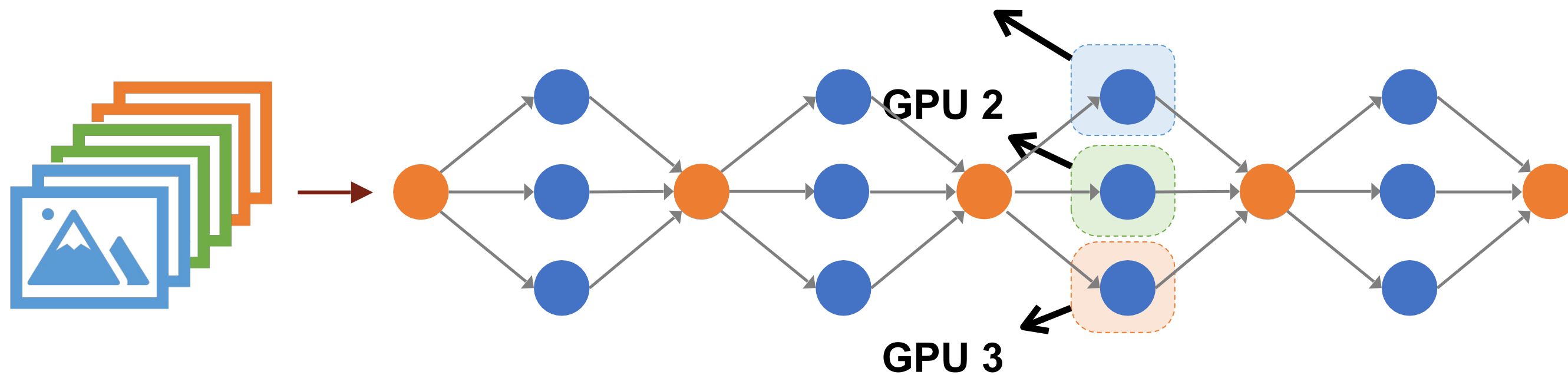


Training Dataset



ML Model

Отдельно считаем активации, затем синхронизируем

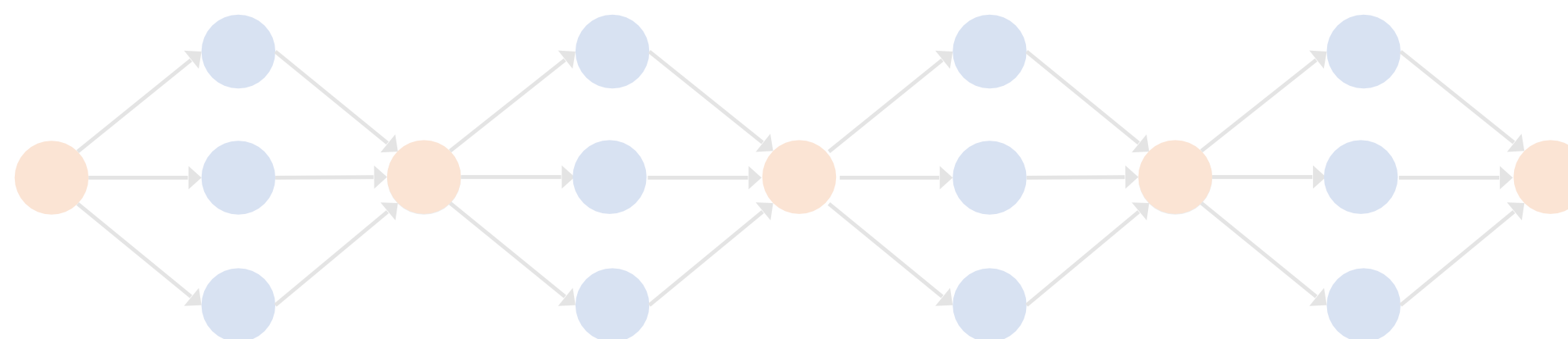


Виды параллелизма в распределенном обучении

Tensor Parallelism

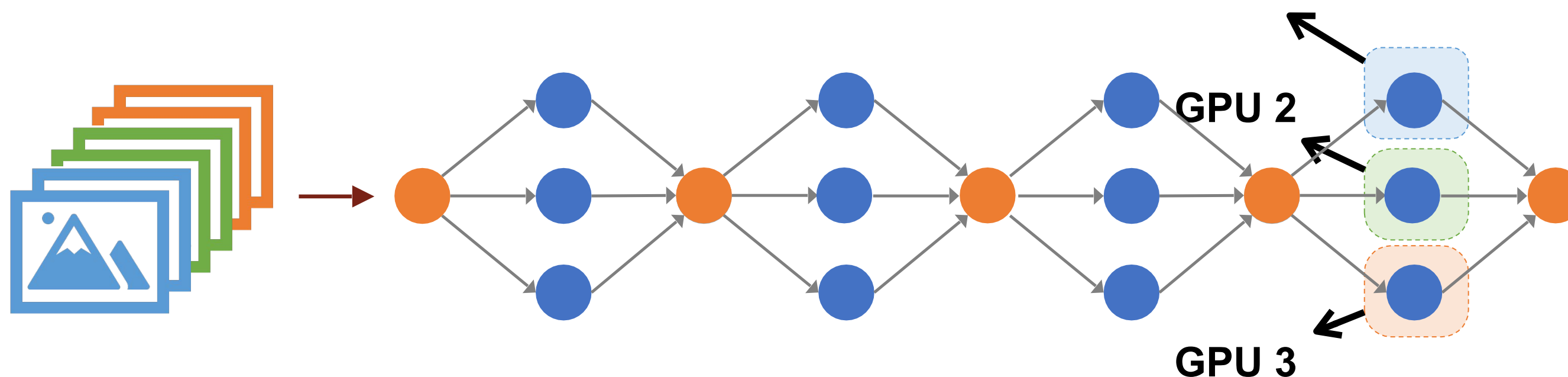


Training Dataset



ML Model

Отдельно считаем активации, затем синхронизируем

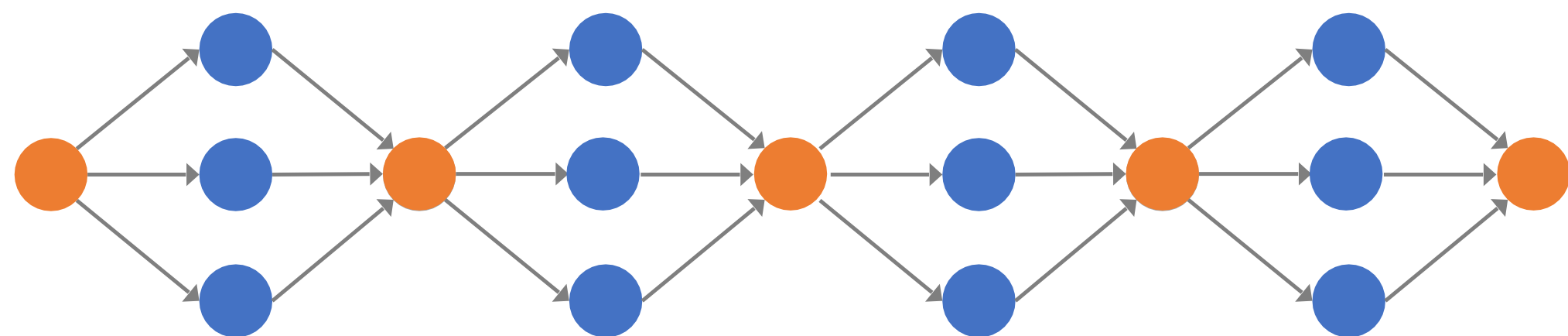


Виды параллелизма в распределенном обучении

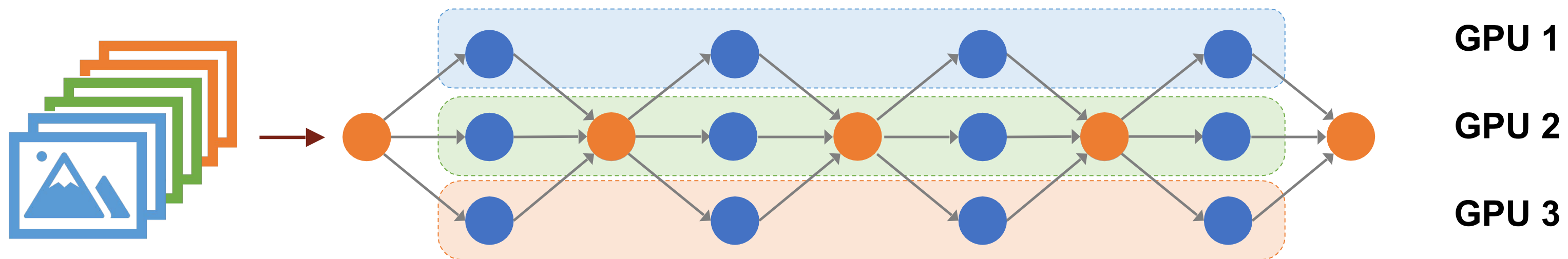
Tensor Parallelism



Training Dataset



ML Model

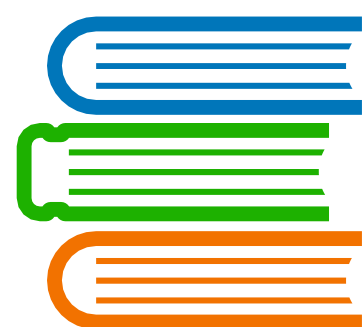


Виды параллелизма в распределенном обучении

- Data Parallelism
- Pipeline Parallelism
- Tensor Parallelism
- **Sequence Parallelism**

Виды параллелизма в распределенном обучении

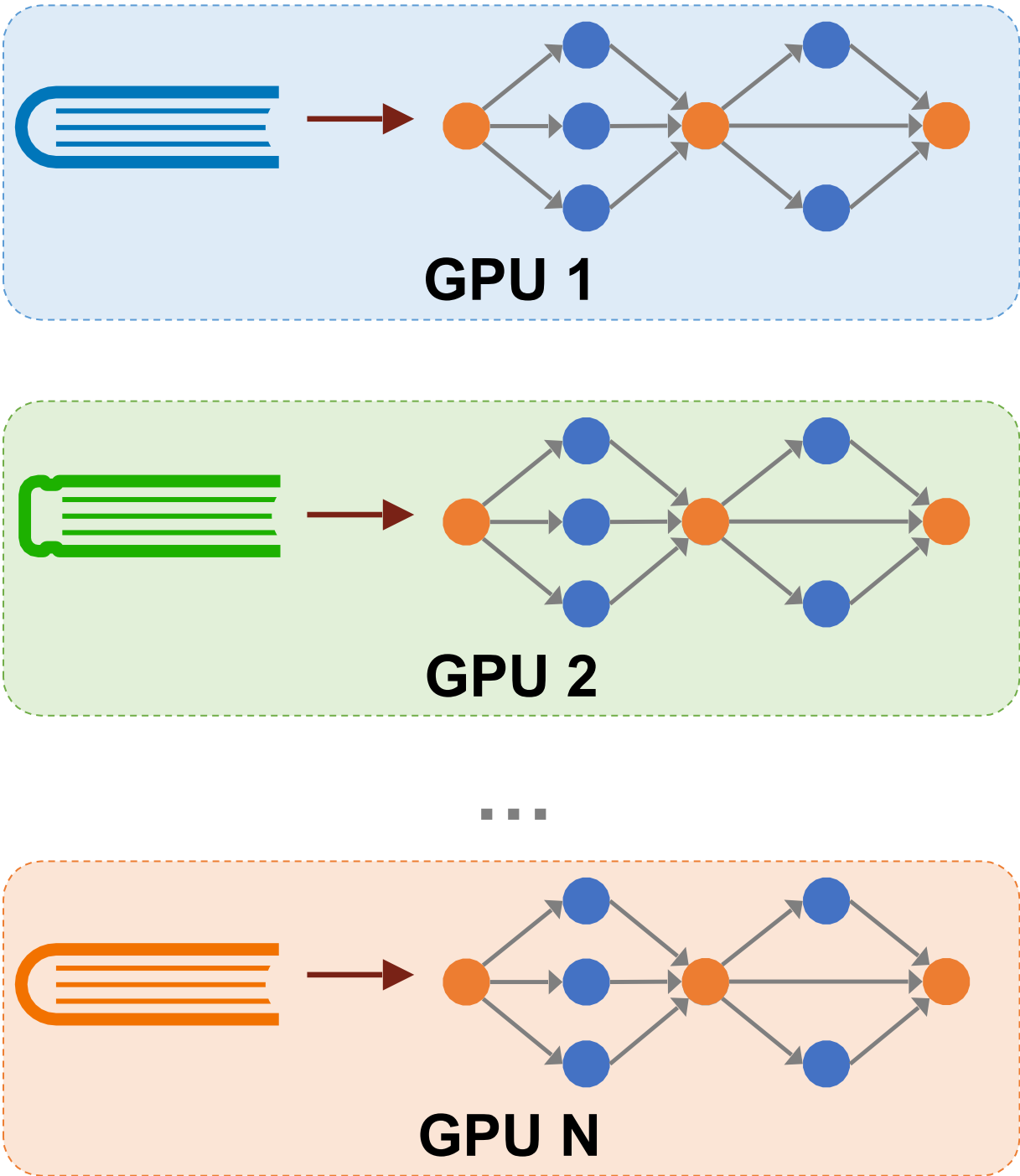
Sequence Parallelism



Long Text for Training

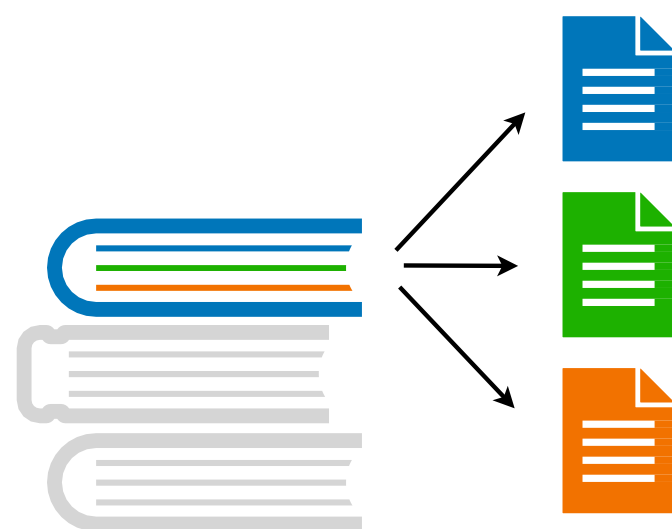
[Batch Size, Tokens, Hidden Dimension]

↑
Data Parallelism
Разделяем батч



Виды параллелизма в распределенном обучении

Sequence Parallelism

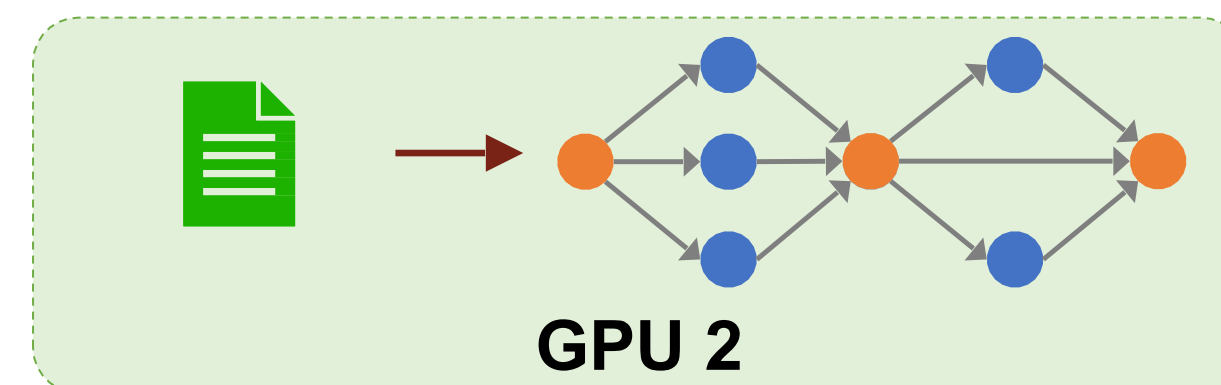
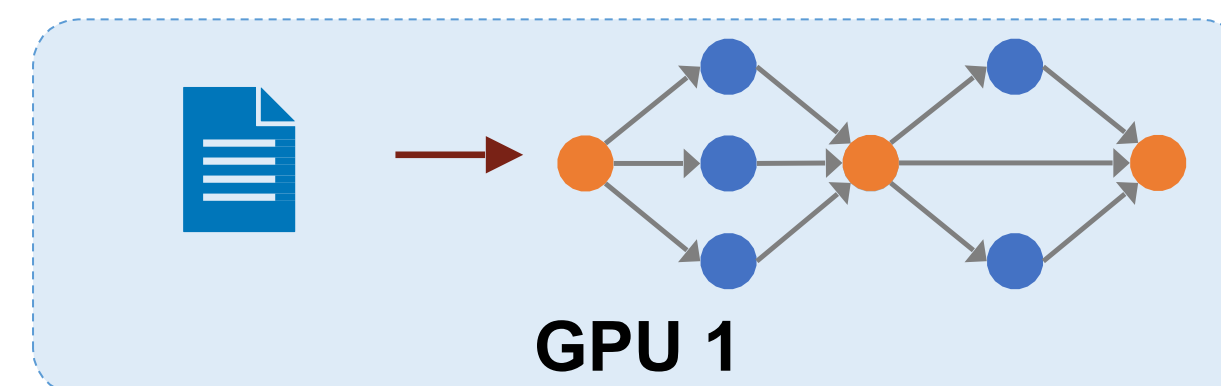


Long Text for Training

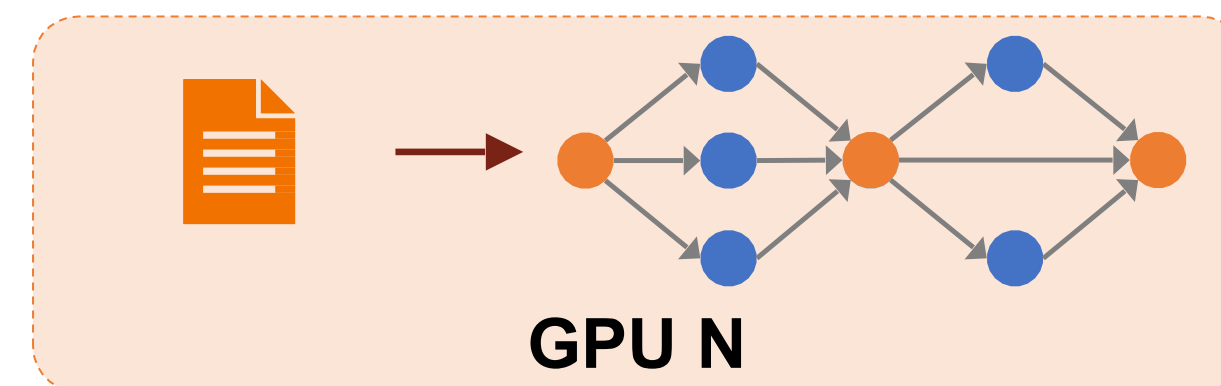
может быть >100K

[Batch Size, **Tokens**, Hidden Dimension]

↑
Sequence Parallelism
Разделяем токены



...



План лекции

01

Бэкграунд и мотивация

Зачем нам распределенное обучение?

02

Виды параллелизма

Обзорно: DP, PP, TP, SP

03

Data Parallelism

Разбираем Parameter Server

04

Коллективные операции

Эффективный All-Reduce

05

Оптимизация памяти в DP

ZeRO-1/2/3 и FSDP

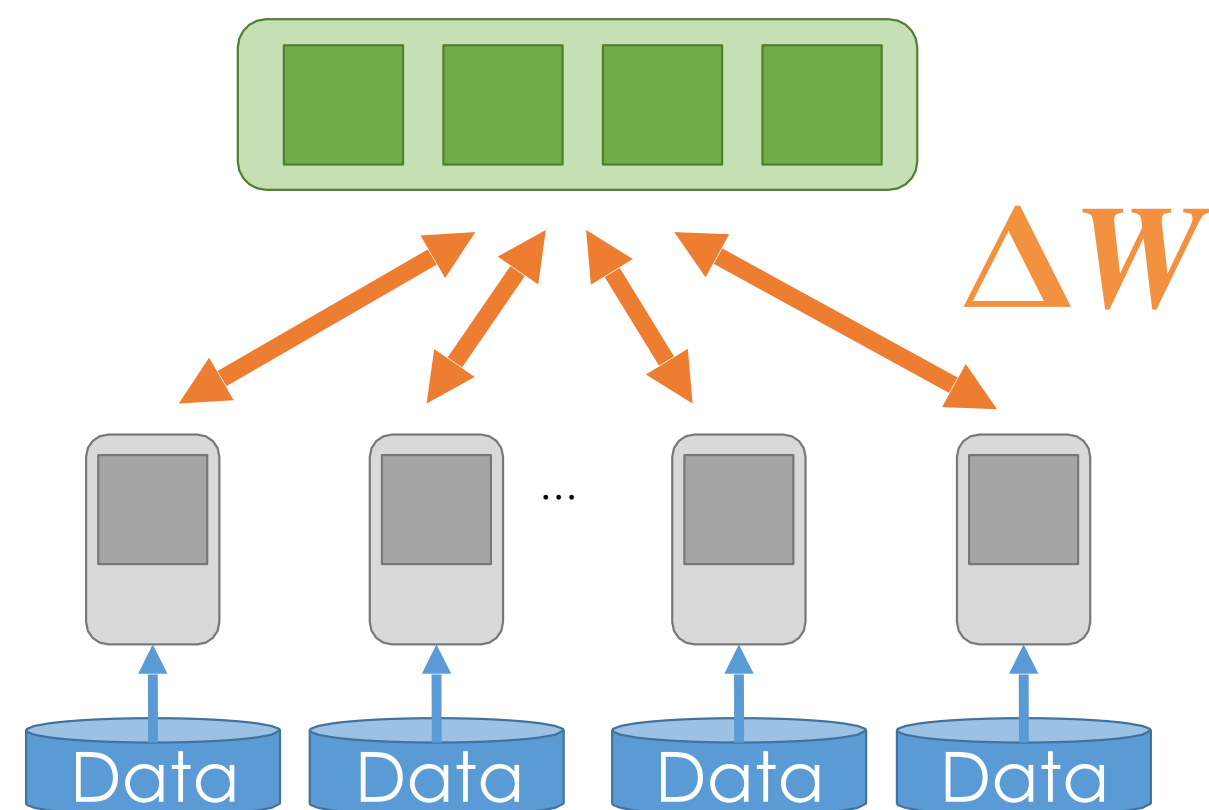


Data Parallelism

Scaling Distributed Machine Learning with the Parameter Server

Parameter Server

Центральный контроллер всего процесса обучения



Worker nodes

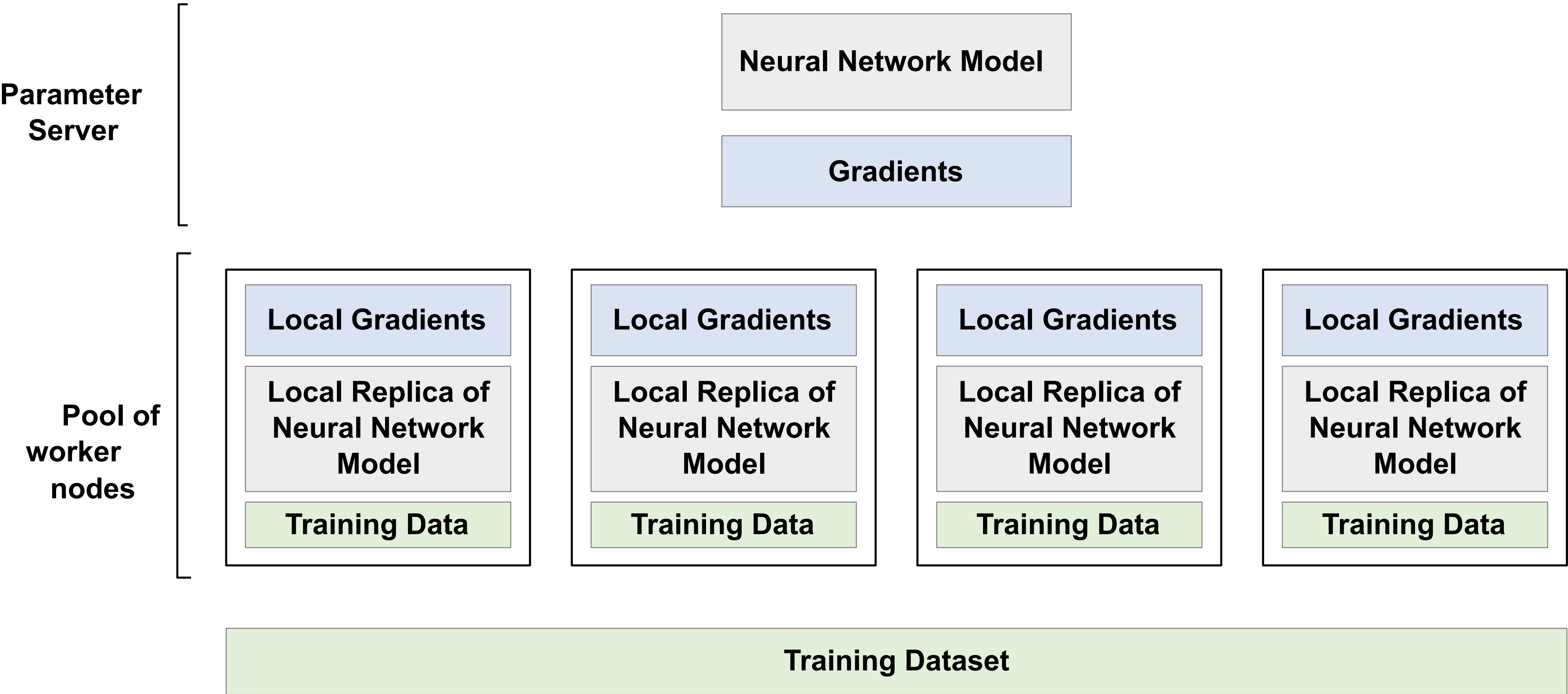
Аппаратные ускорители и хранилище данных

Две разные роли:

- **Parameter Server**: получает градиенты от воркеров и отправляет назад новые веса модели
- **Workers**: вычисляют градиенты на части датасета и отправляют их серверу

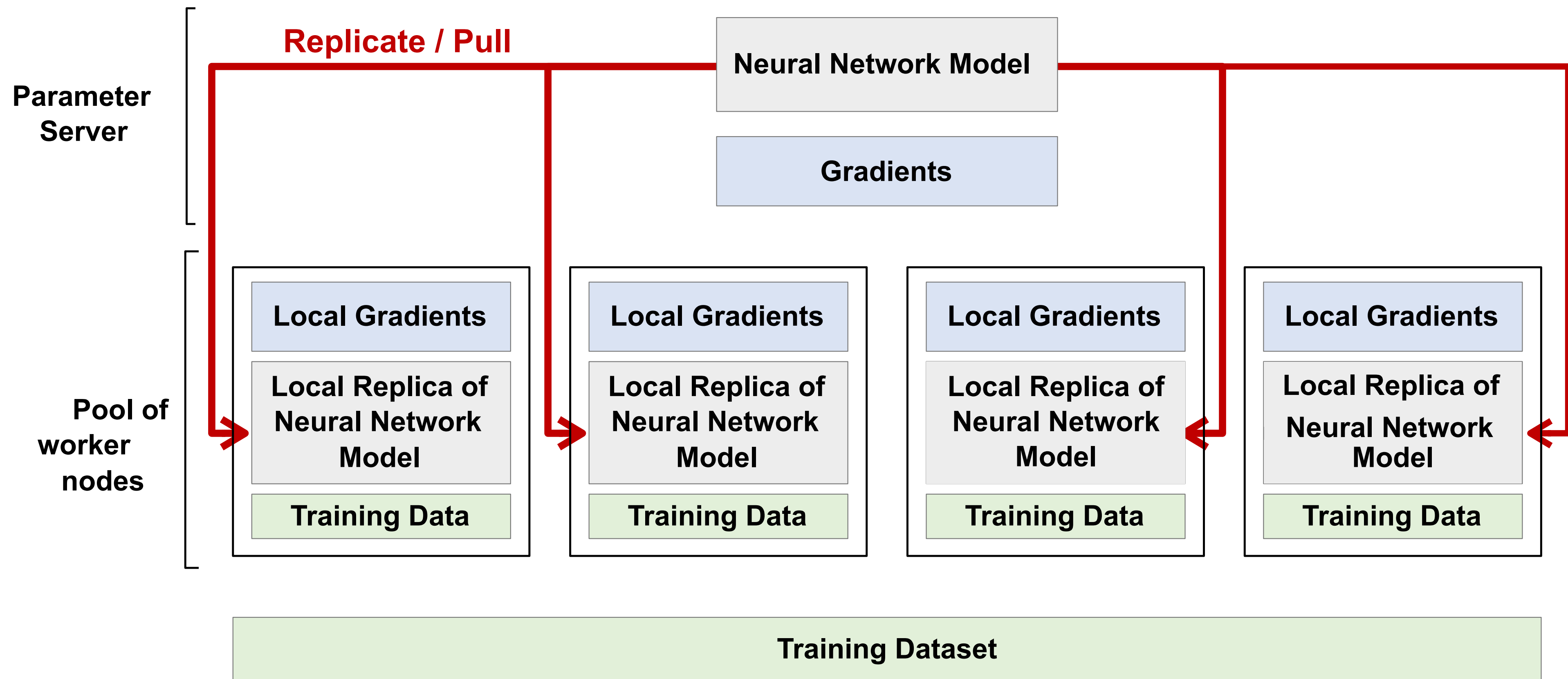
Data Parallelism

Разберем по шагам



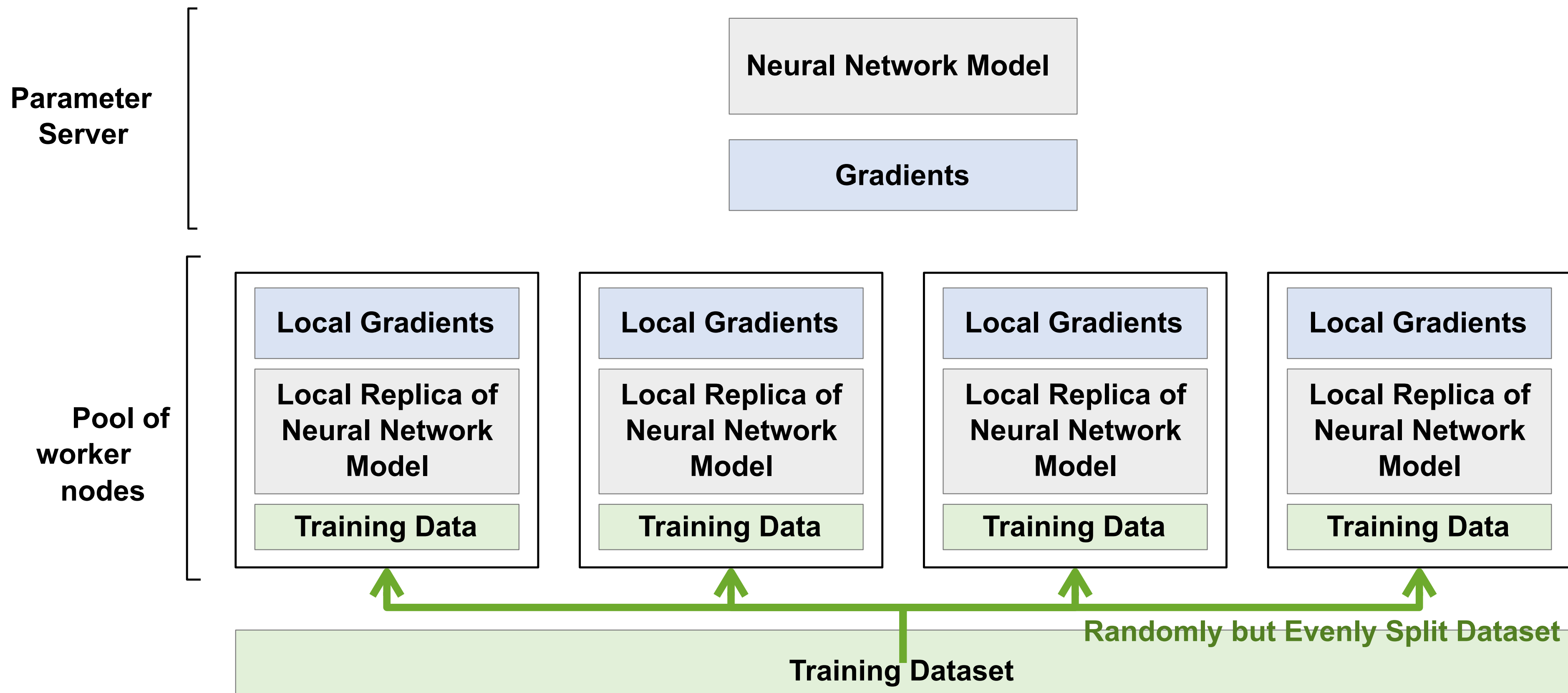
Data Parallelism

1 — копируем модель на каждый воркер



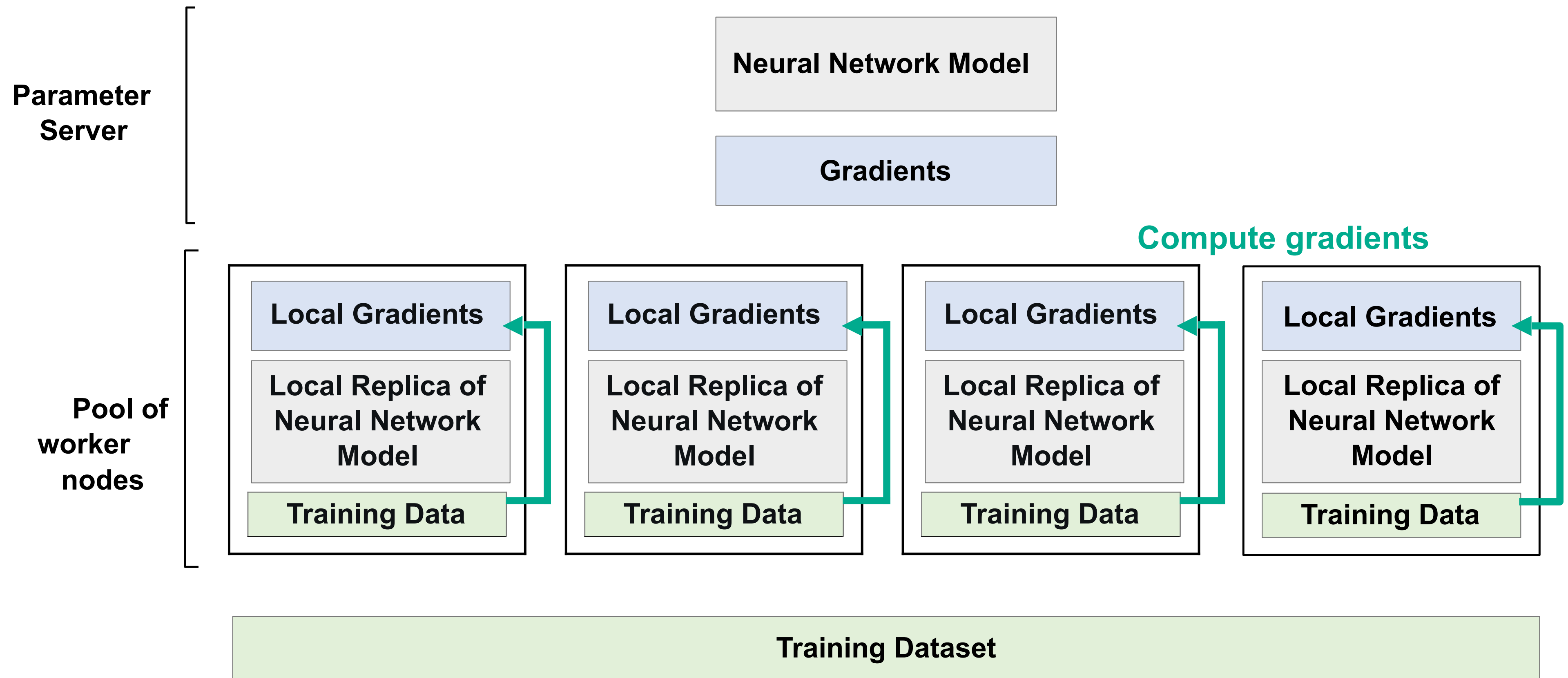
Data Parallelism

2 — разделяем трейн по воркерам



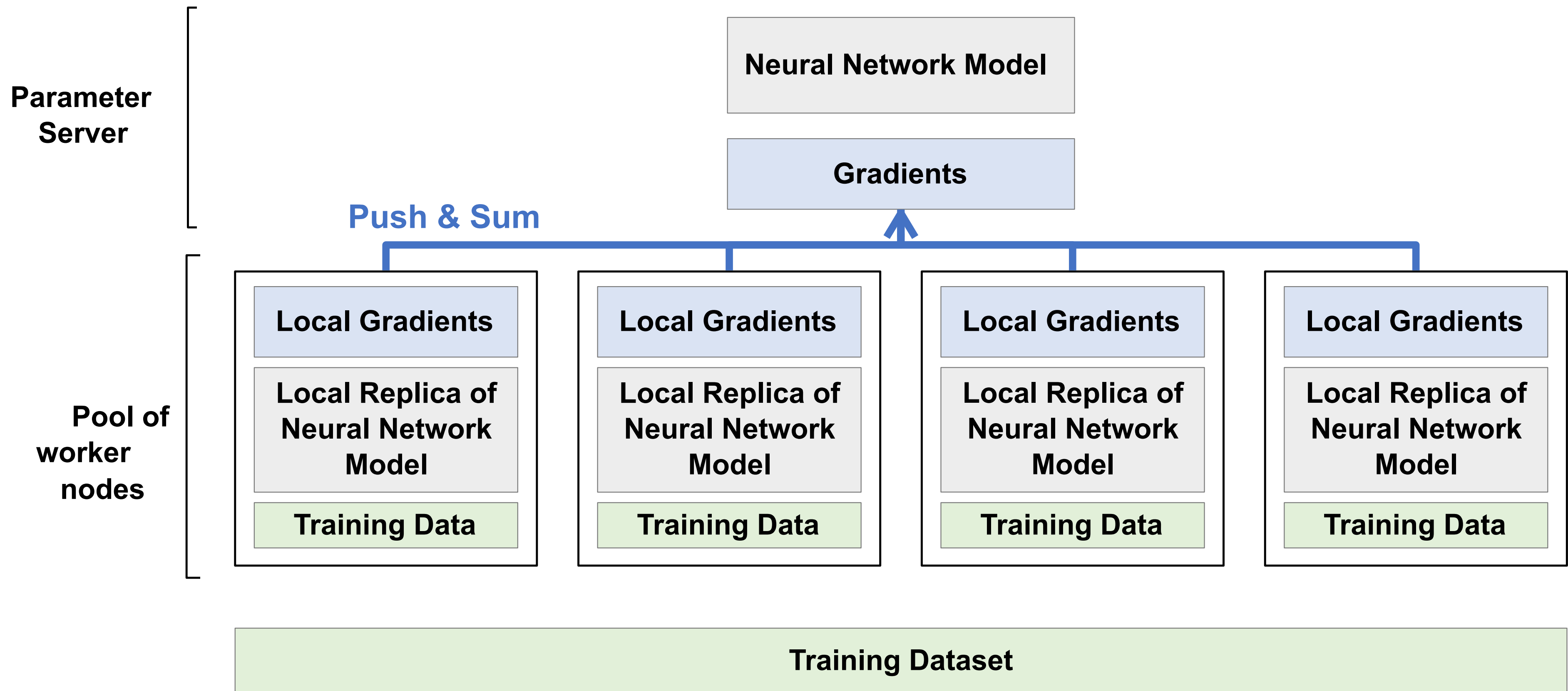
Data Parallelism

3 — считаем градиенты



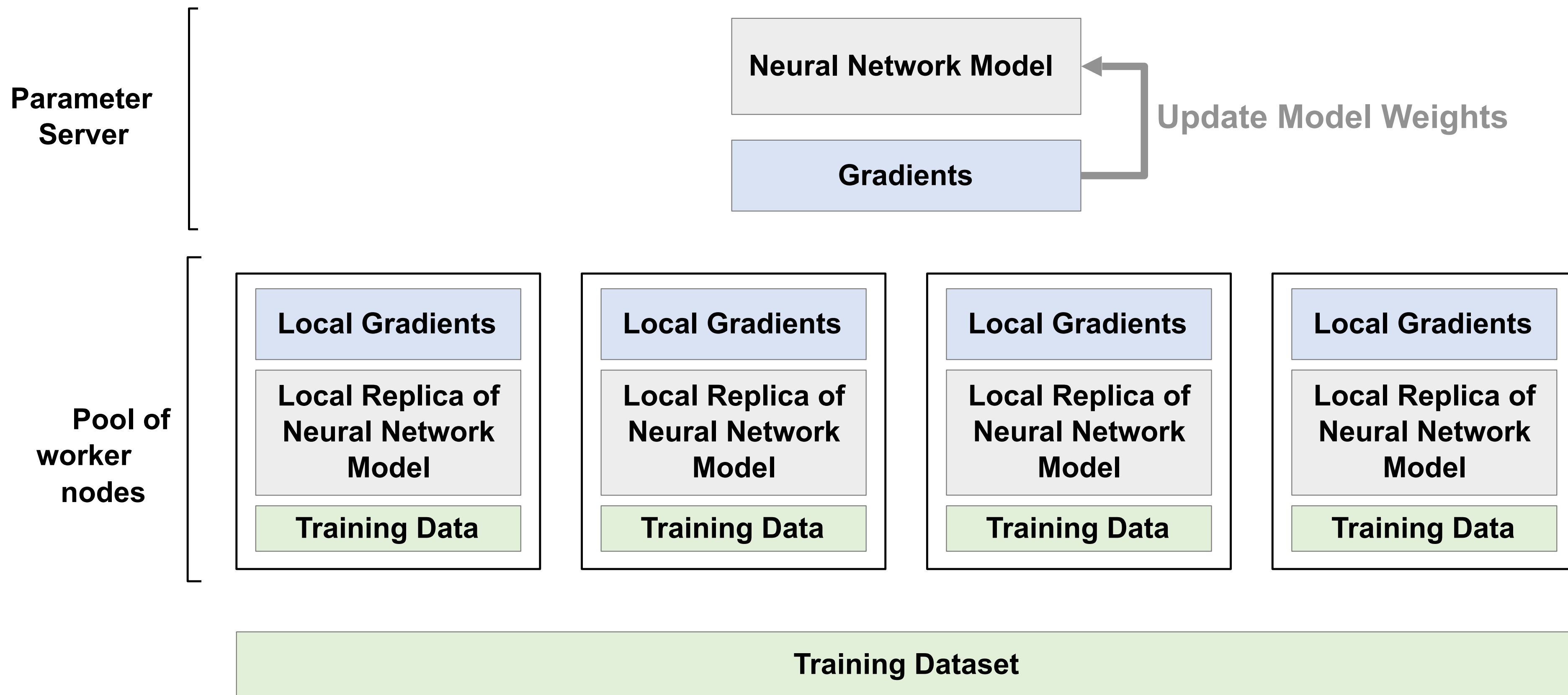
Data Parallelism

4 — агрегируем градиенты



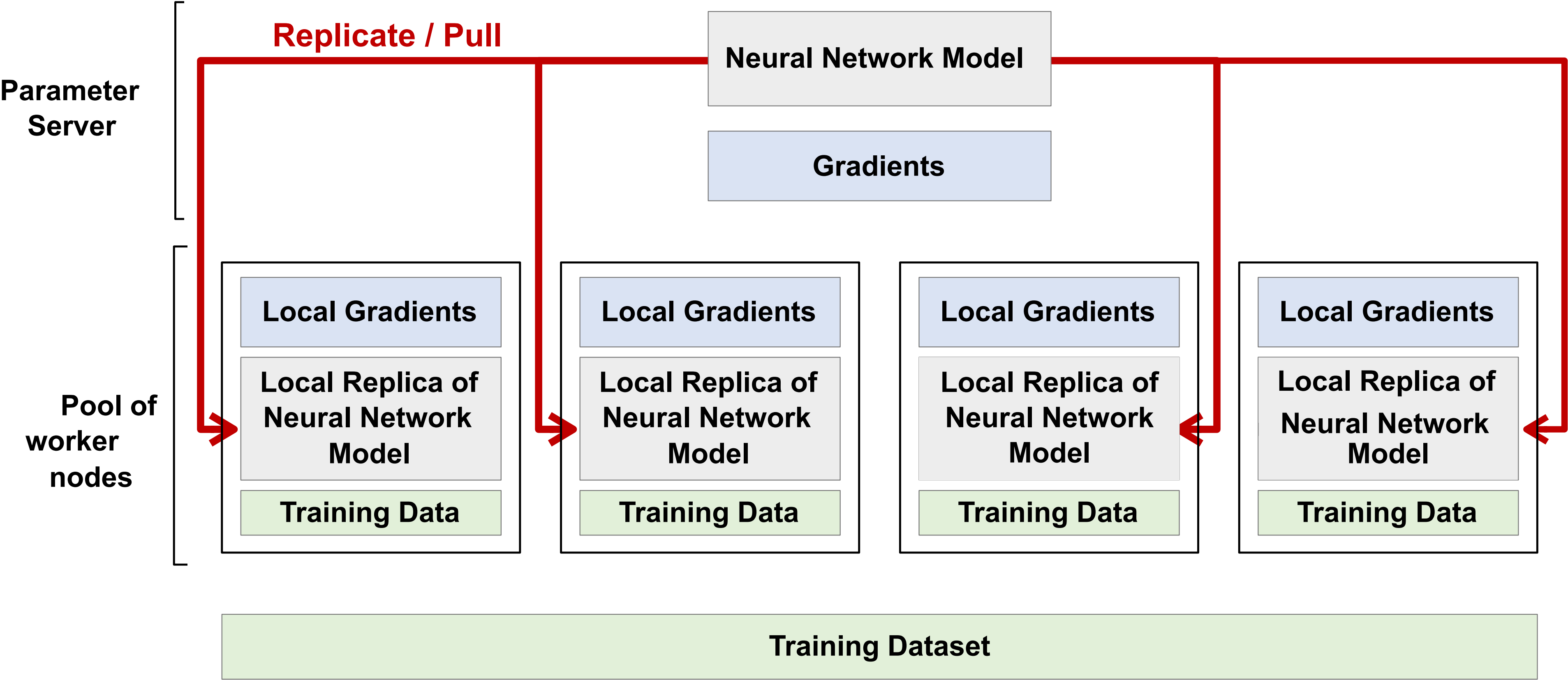
Data Parallelism

5 — выполняем шаг оптимизации и обновляем веса



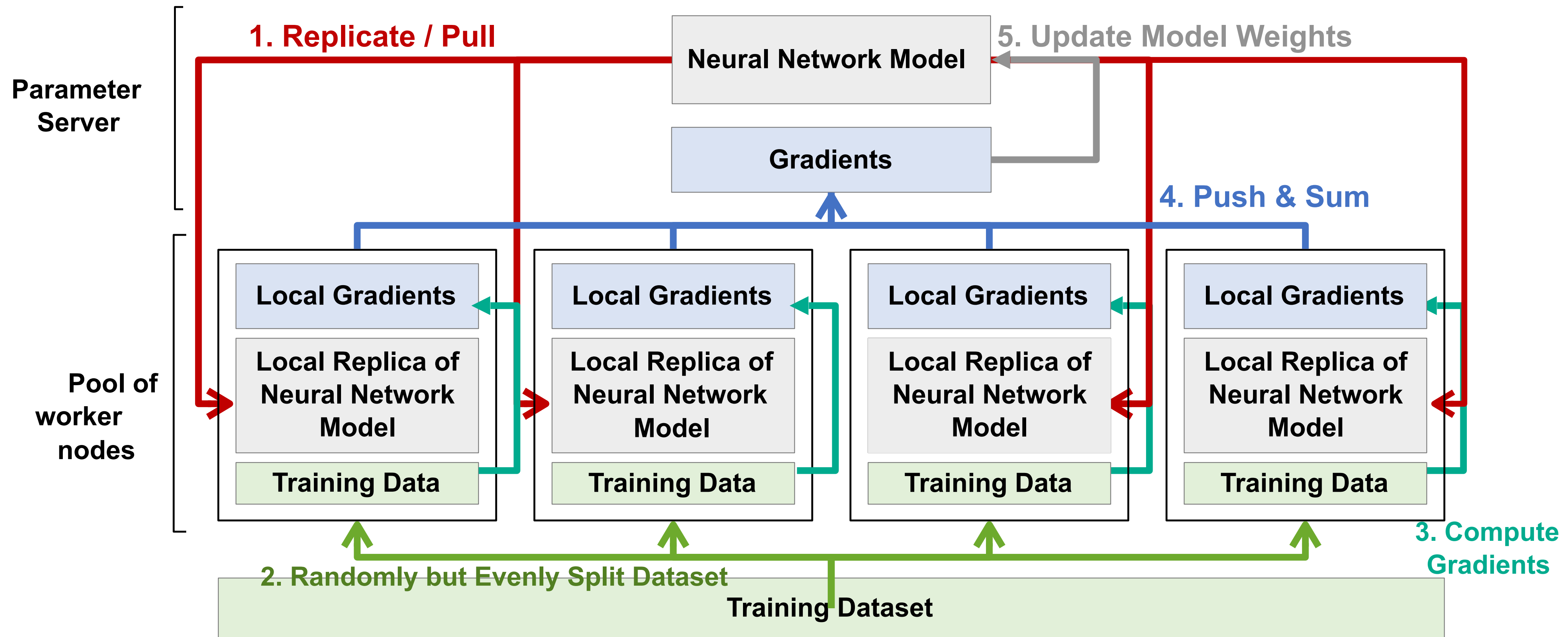
Data Parallelism

Затем повторяем



Data Parallelism

Replicate -> Split -> Compute Gradients -> Push & Sum -> Update



Обучение Single Node vs Distributed

Single Node Training:

For iteration i in $0 \dots T$,

1. Sample data from datasets $x_{(i)}, y_{(i)}$
2. Compute gradients $\nabla w_{(i)} = f(x_{(i)}, y_{(i)}; w_{(i)})$
3. Perform gradient step $w_{(i)} = w_{(i)} - \eta \nabla w_{(i)}$

Distributed Training:

For iteration i in $0 \dots T$,

1. Replicate / pull weights from parameter server
2. Sample data from datasets $x_{(ij)}, y_{(ij)}$
3. Compute gradients $\nabla w_{(i)} = f(x_{(i)}, y_{(i)}; w_{(i)})$
4. Push $\nabla w_{(ij)}$ to parameter server and sum
5. Perform gradient step $w_{(ij)} = w_{(ij)} - \eta \overline{\nabla w_{(i)}}$ at parameter server

два этапа коммуникаций между GPU

План лекции

01

Бэкграунд и мотивация

Зачем нам распределенное обучение?

02

Виды параллелизма

Обзорно: DP, PP, TP, SP

03

Data Parallelism

Разбираем Parameter Server

04

Коллективные операции

Эффективный All-Reduce

05

Оптимизация памяти в DP

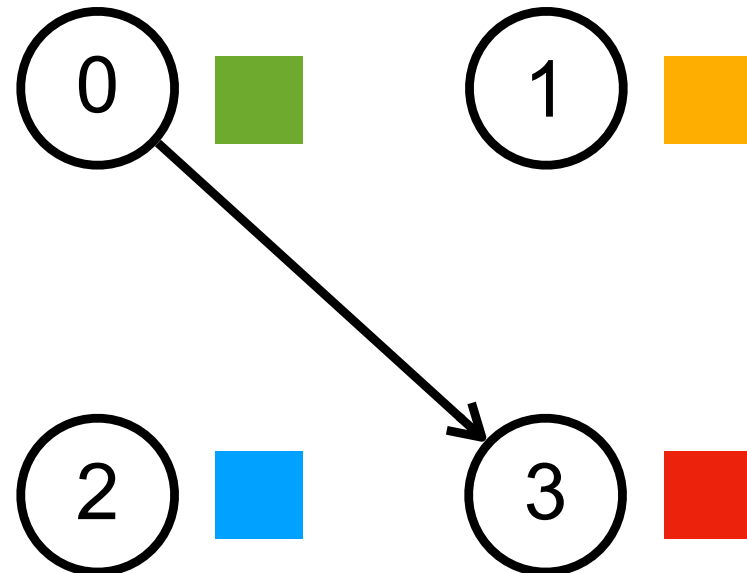
ZeRO-1/2/3 и FSDP



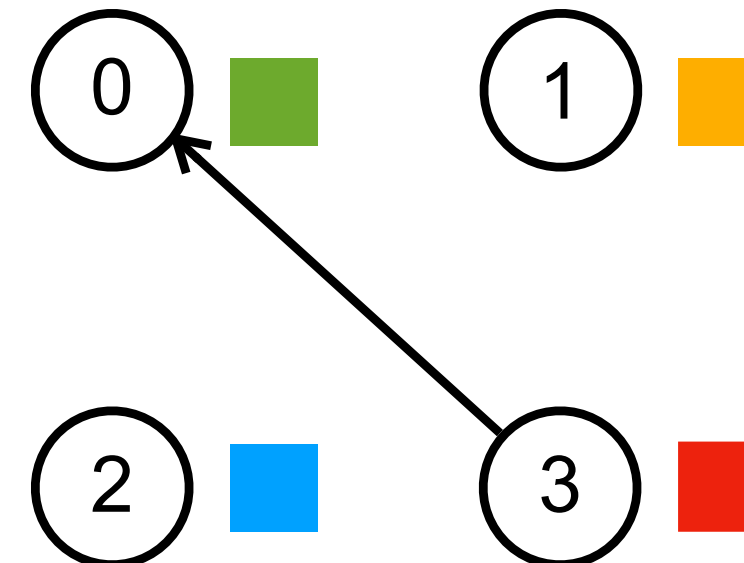
Коллективные операции

One-to-One: Send и Recv

Send: n0 -> n3



Recv: n0 -> n3



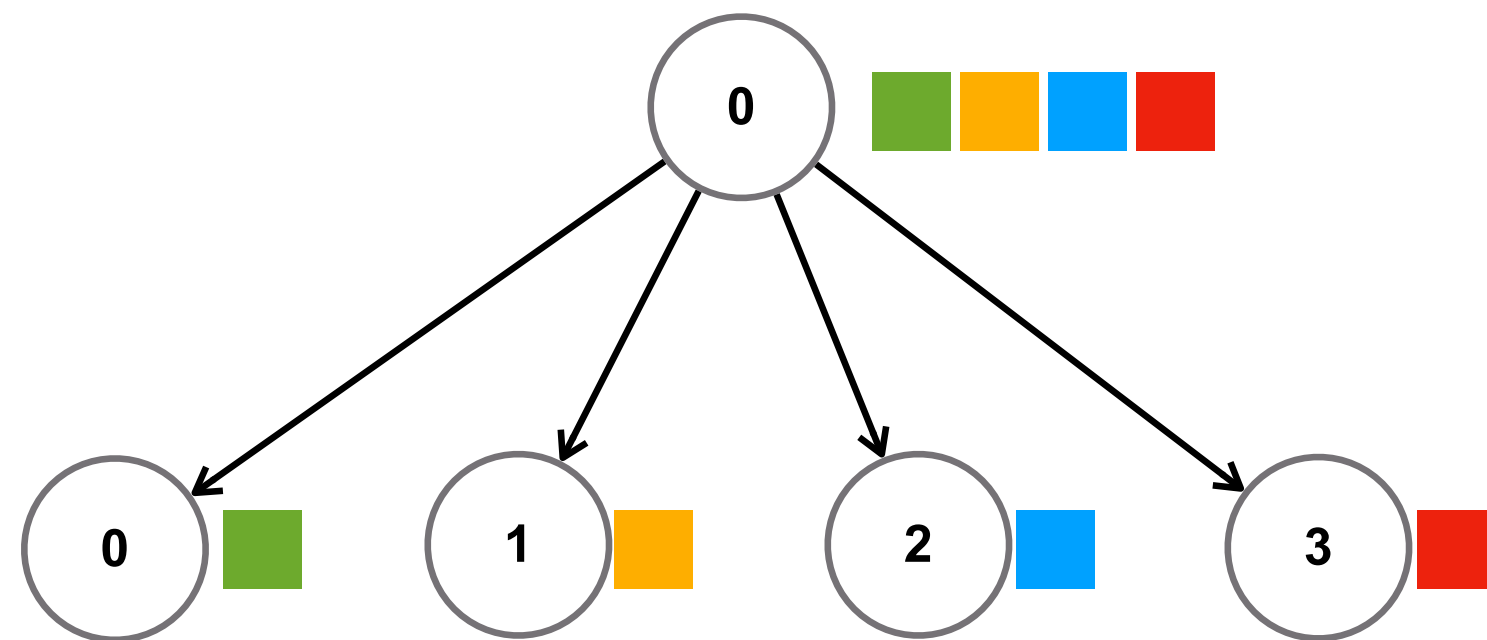
One-to-one коммуникация: передача данных из одного процесса в другой

- Send & Receive являются наиболее распространенными схемами распределенной коммуникации
- Реализованы в MPI / NCCL

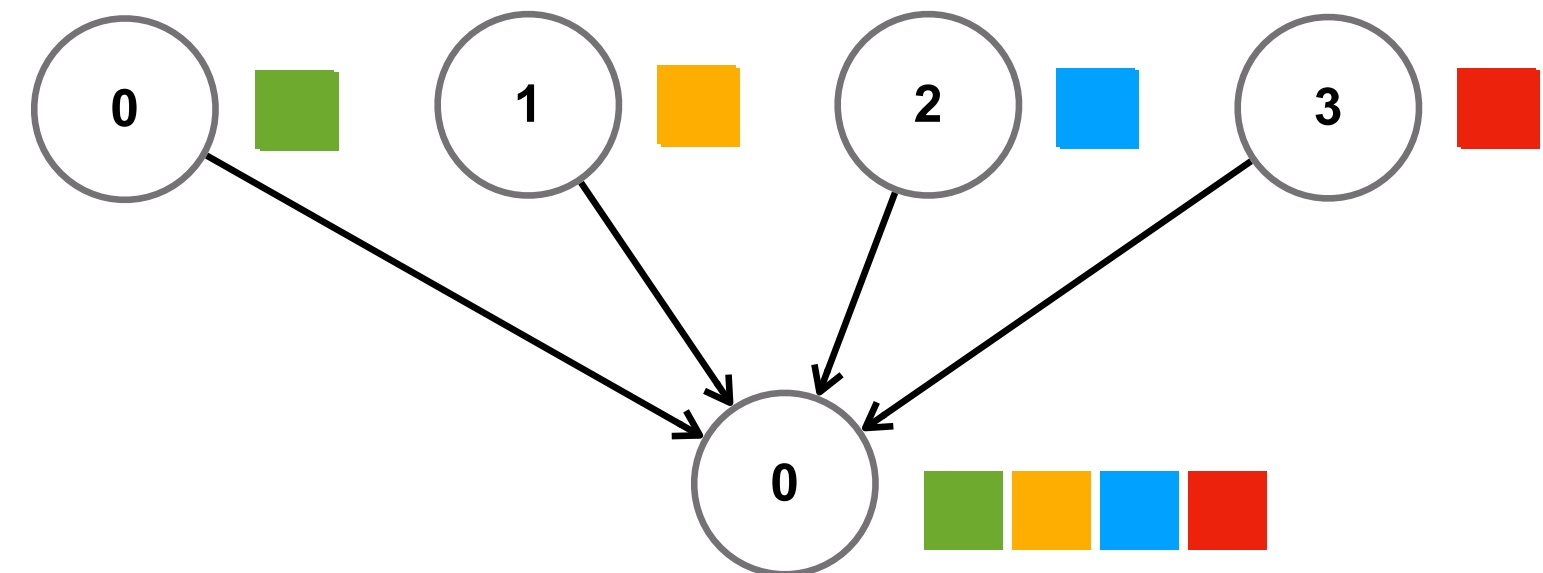
Коллективные операции

One-to-Many: Scatter и Gather

Scatter



Gather



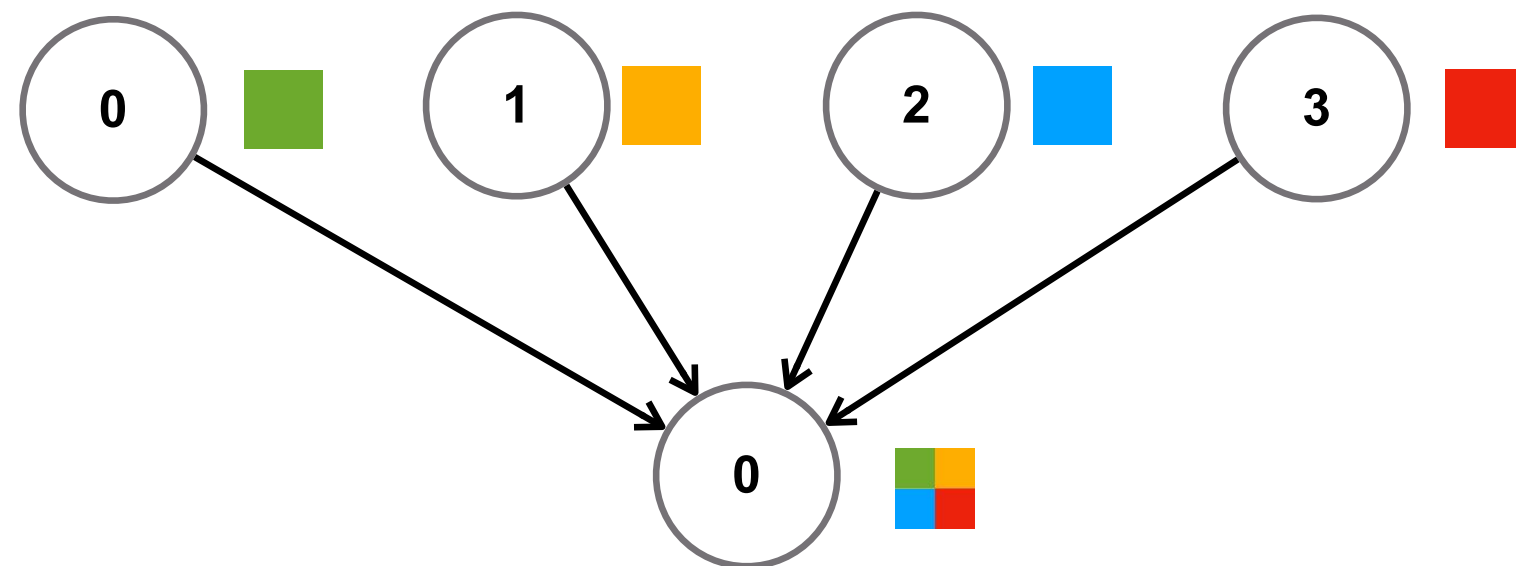
One-to-many коммуникация: такие операции выполняются всеми воркерами

- Scatter: рассылка тензоров всем остальным воркерам
- Gather: сбор тензоров от всех остальных воркеров

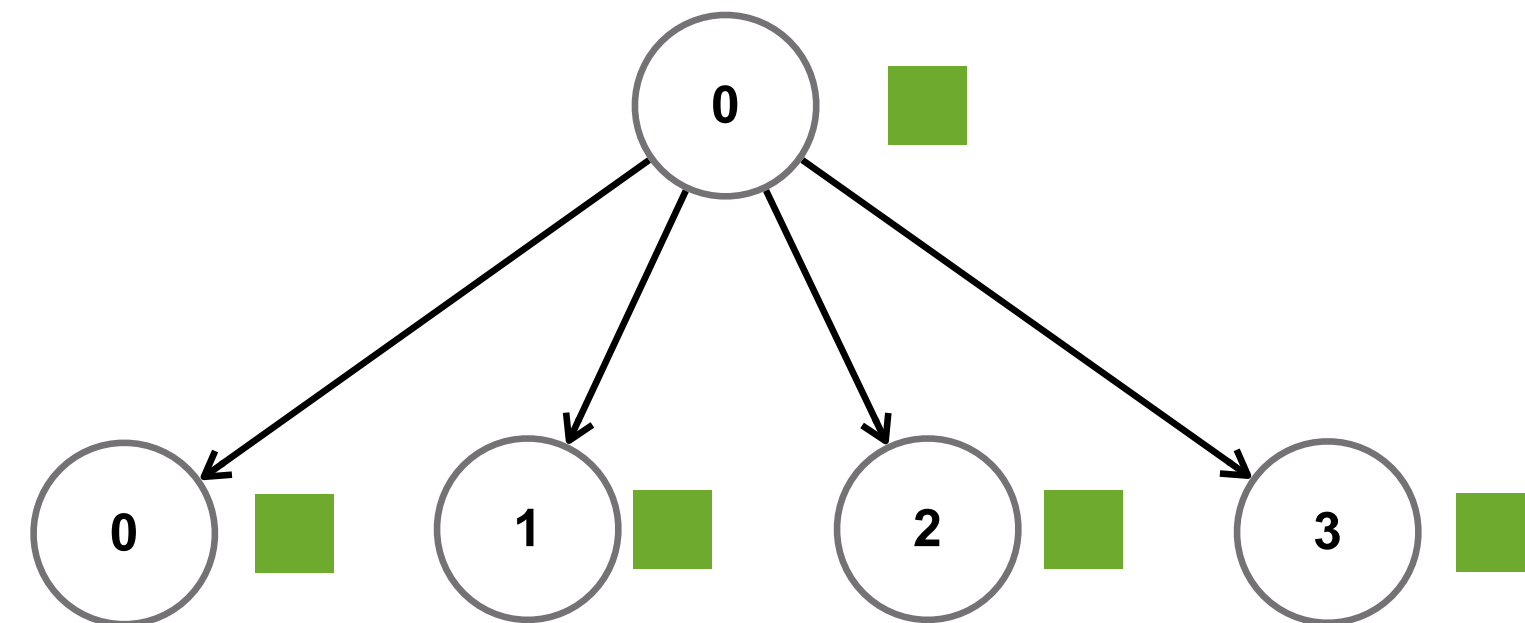
Коллективные операции

One-to-Many: Reduce и Broadcast

Reduce



Broadcast



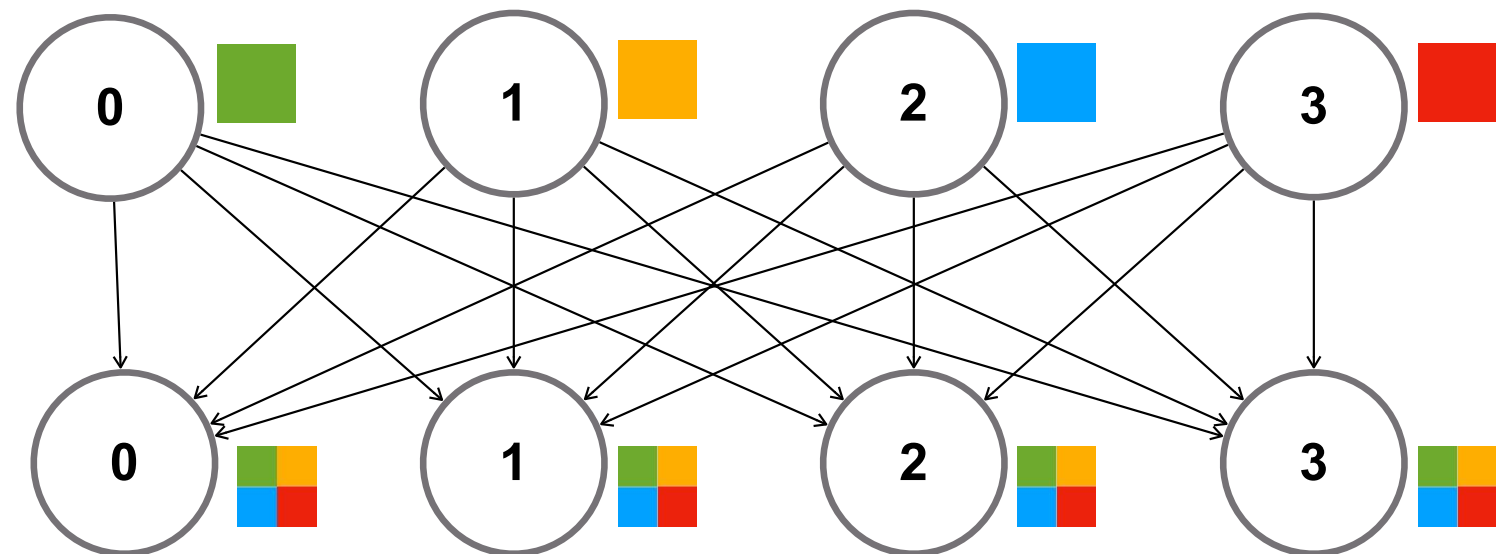
One-to-many коммуникация: такие операции выполняются всеми воркерами

- Reduce: похож на gather, но выполняет усреднение / суммирование во время агрегации
 - [1] [2] [3] [4] —(gather)—> [1,2,3,4]
 - [1] [2] [3] [4] —(reduce)—> [1 + 2 + 3 + 4] = [10]
- Broadcast: рассылка идентичных копий всем остальным воркерам

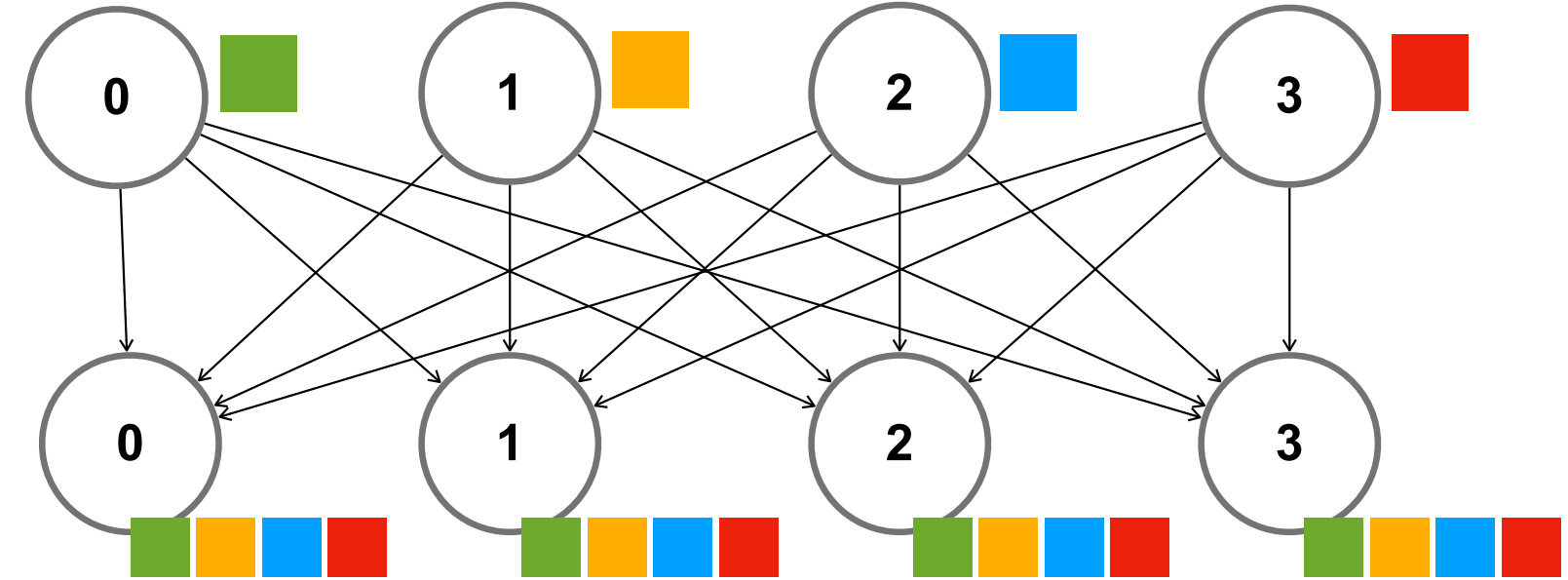
Коллективные операции

Many-to-Many: All Reduce и All Gather

All-Reduce



All-Gather

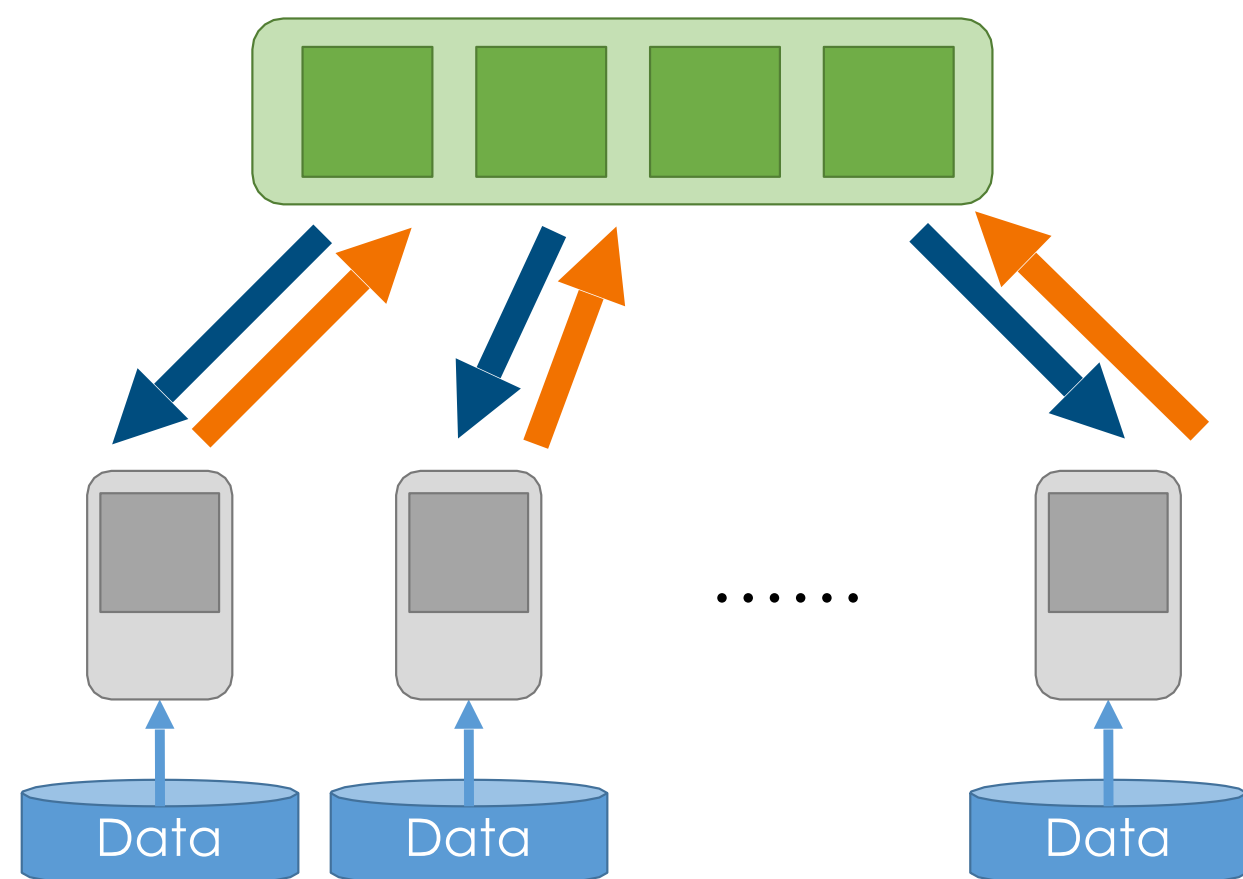


Many-to-many коммуникация:

- All-Reduce: *Reduce* выполняется на всех воркерах
- All-Gather: *Gather* выполняется на всех воркерах

Коллективные операции

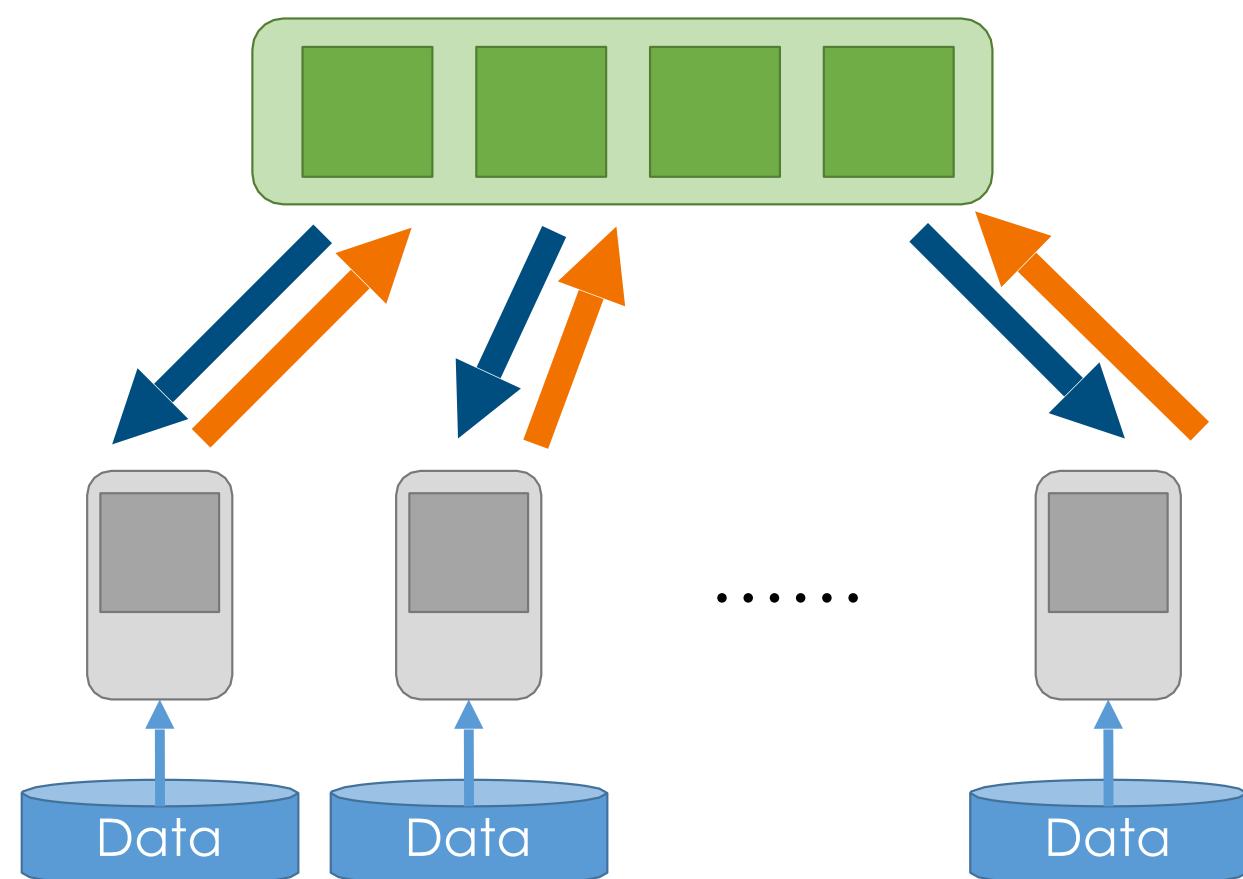
Схемы коммуникаций в Parameter Server



Коллективные операции

Схемы коммуникаций в Parameter Server

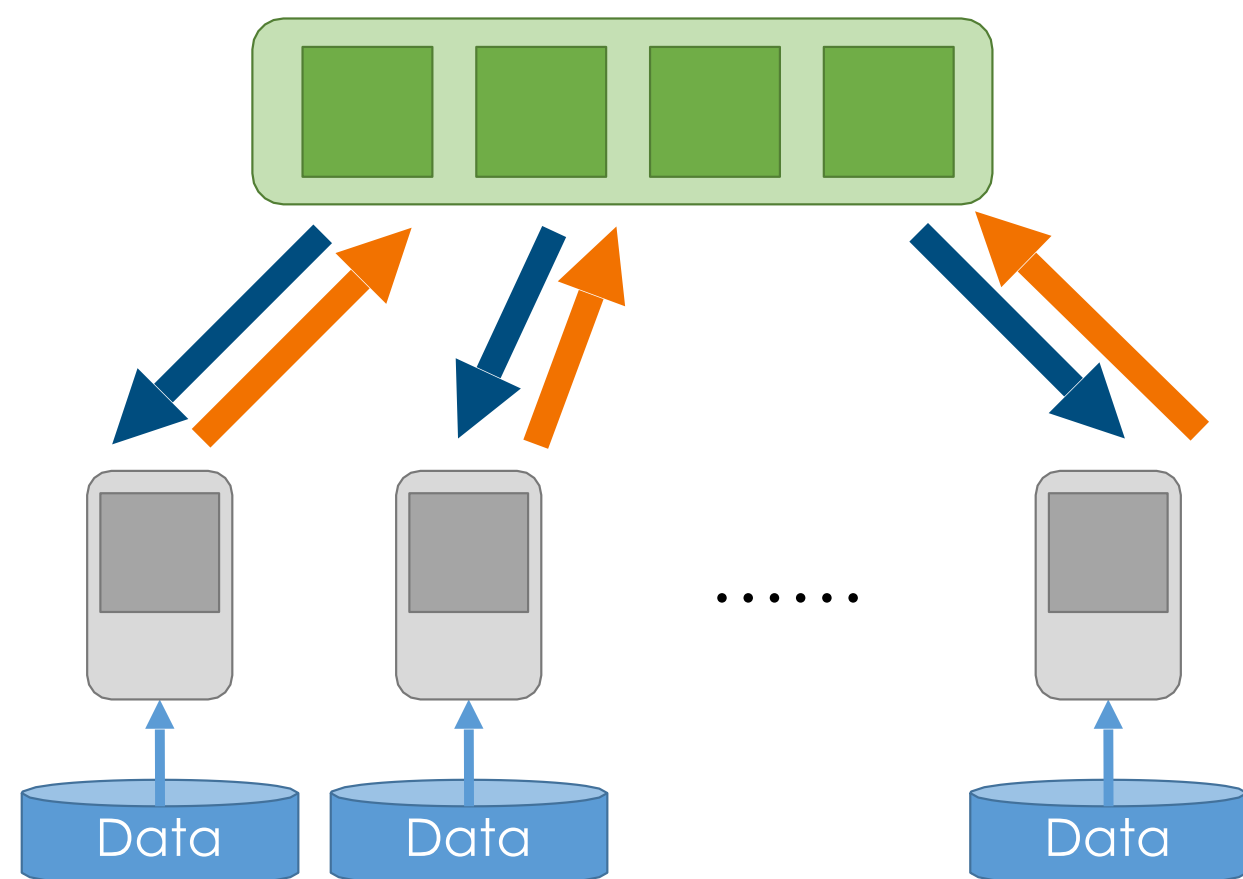
- Какая схема используется?
 - **Replicate & Pull**: ?
 - **Push & Sum**: ?



Коллективные операции

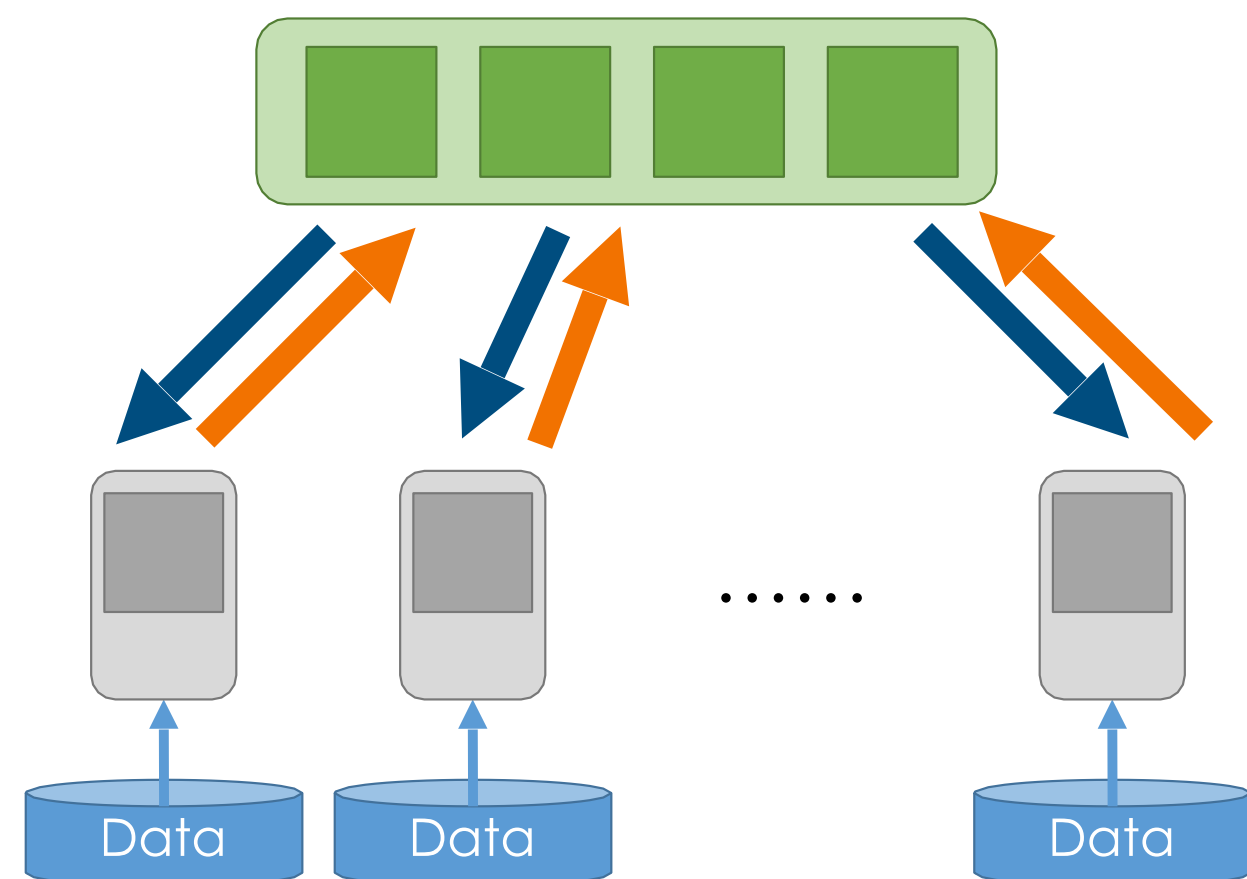
Схемы коммуникаций в Parameter Server

- Какая схема используется?
 - **Replicate & Pull**: Broadcast
 - Parameter Server -> всем воркерам
 - **Push & Sum**: Reduce
 - Усреднение со всех воркеров -> на Parameter Server



Коллективные операции

Схемы коммуникаций в Parameter Server

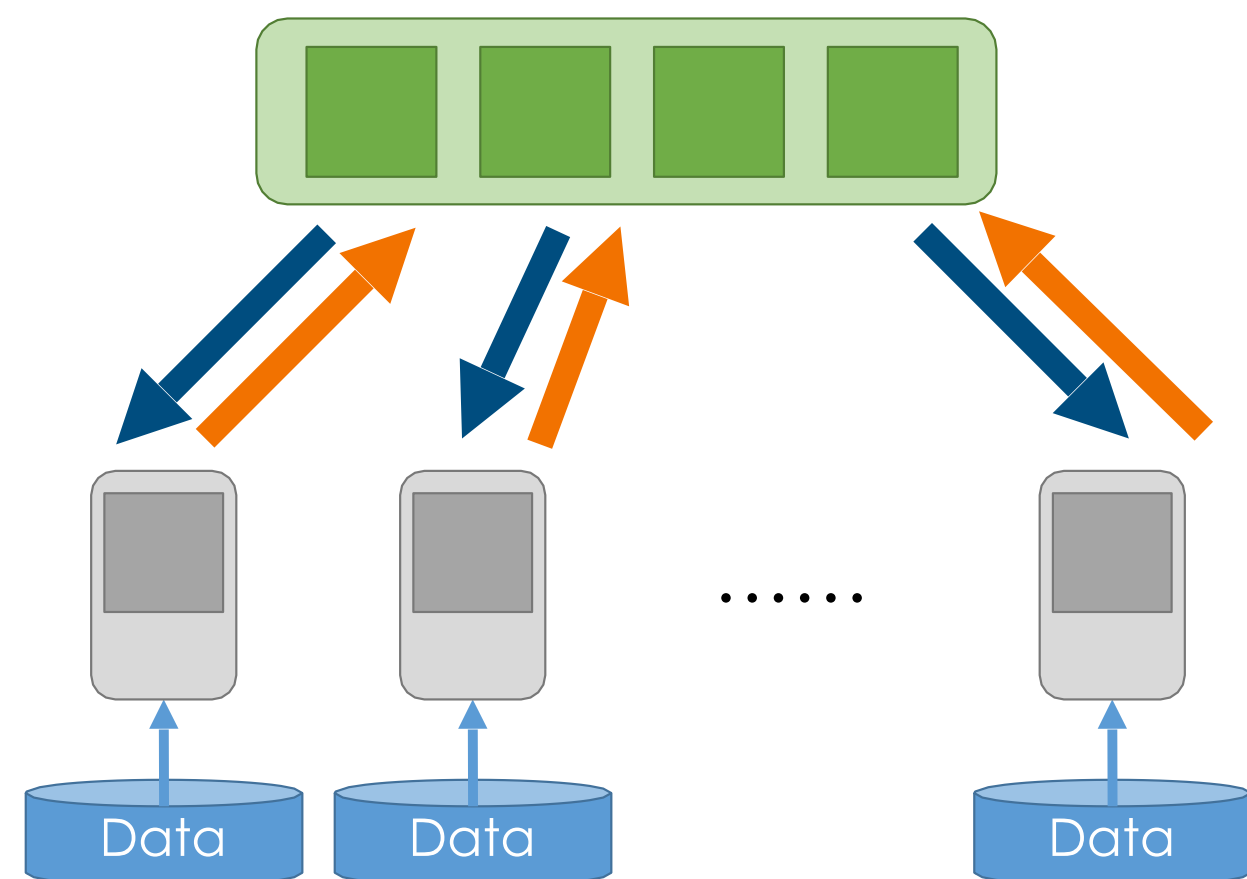


Note: N — число воркеров

- Какая схема используется?
 - **Replicate & Pull:** Broadcast
 - Parameter Server -> всем воркерам
 - **Push & Sum:** Reduce
 - Усреднение со всех воркеров -> на Parameter Server
- Какие требования по пропускной способности в терминах $O(N)$?
 - **Replicate & Pull:**
 - Worker: ?
 - Parameter Server: ?
 - **Push & Sum:**
 - Worker: ?
 - Parameter Server: ?

Коллективные операции

Схемы коммуникаций в Parameter Server

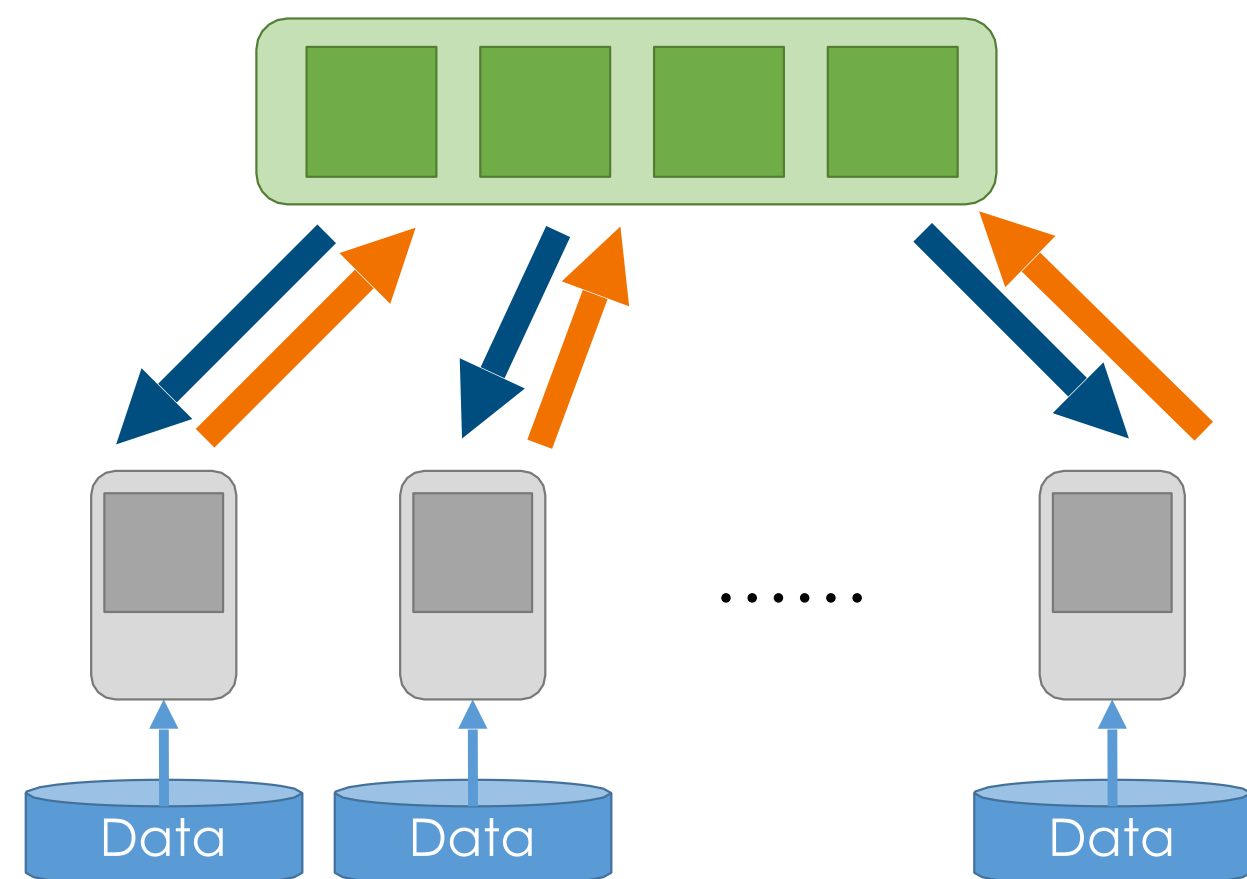


Note: N — число воркеров

- Какая схема используется?
 - **Replicate & Pull:** Broadcast
 - Parameter Server $\rightarrow$ всем воркерам
 - **Push & Sum:** Reduce
 - Усреднение со всех воркеров $\rightarrow$ на Parameter Server
- Какие требования по пропускной способности в терминах $O(N)$?
 - **Replicate & Pull:**
 - Worker: $O(1)$
 - Parameter Server: $O(N)$
 - **Push & Sum:**
 - Worker: $O(1)$
 - Parameter Server: $O(N)$

Коллективные операции

Схемы коммуникаций в Parameter Server



Note: N — число воркеров

- **Replicate & Pull:**

- Worker: $O(1)$
- Parameter Server: $O(N)$

- **Push & Sum:**

- Worker: $O(1)$
- Parameter Server: $O(N)$

Требования к пропускной способности центрального Parameter Server растут линейно при росте числа воркеров. Это может стать **узким местом**, когда воркеров много

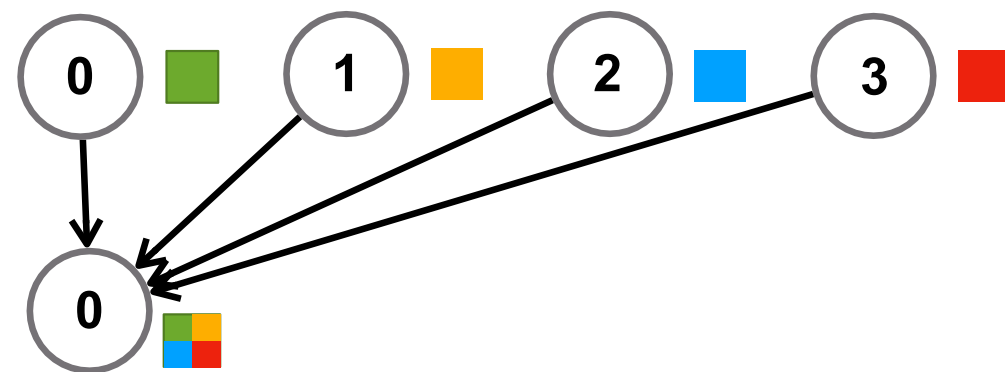
Можем ли мы выполнить агрегацию без центрального сервера?

All-Reduce!

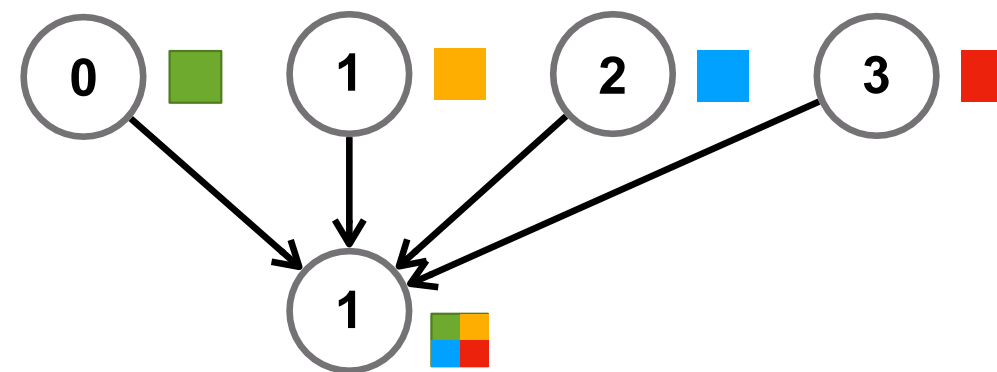
Коллективные операции

Наивная реализация All-Reduce — Sequential

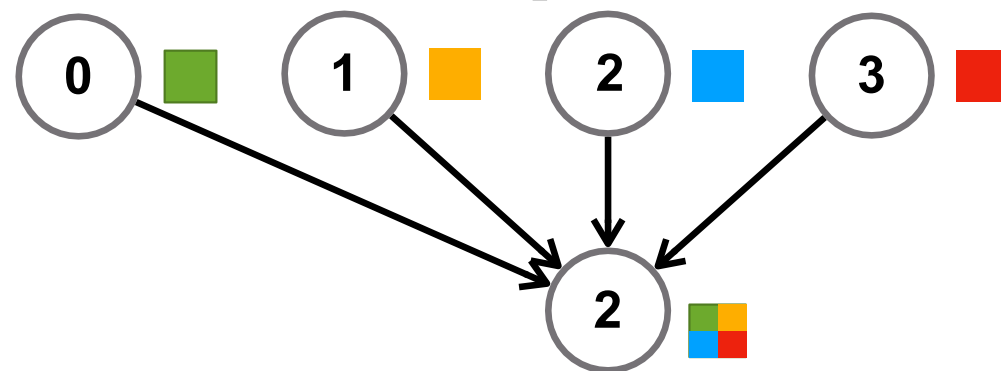
Step 1



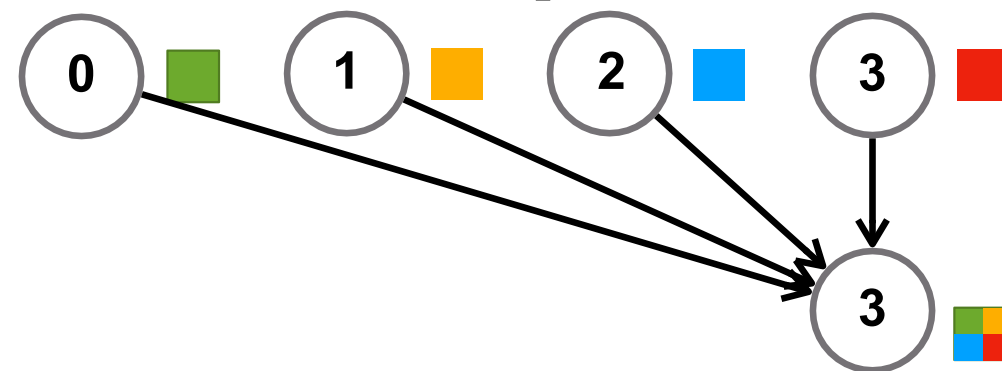
Step 2



Step 3



Step 4



Time: $O(N)$

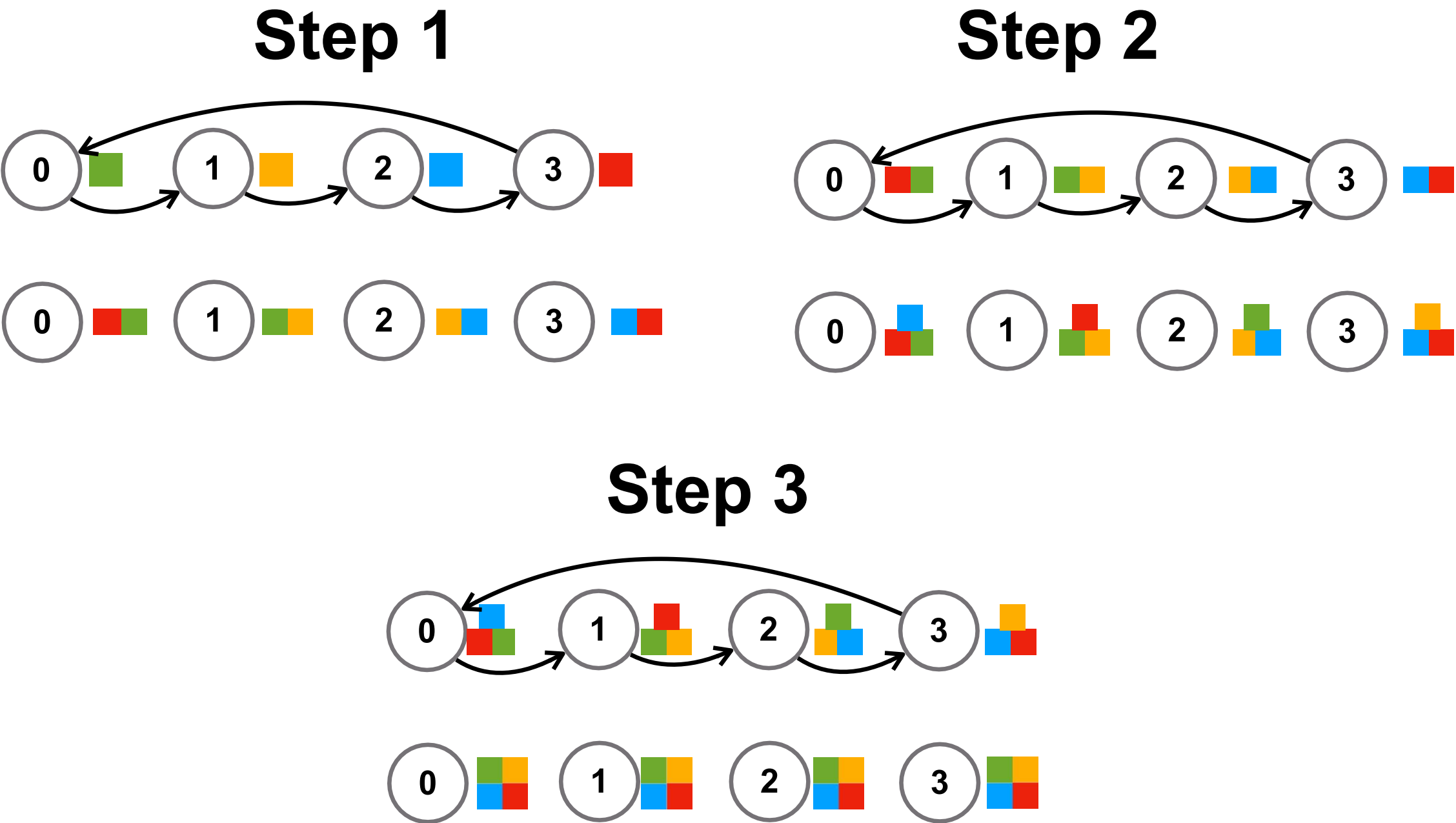
Bandwidth: $O(N)$

На каждом шаге выполняется **одна** операция Reduce

Note: N — число воркеров

Коллективные операции

Ring All-Reduce



Time: $O(N)$

Bandwidth: $O(1)$ <- лучше

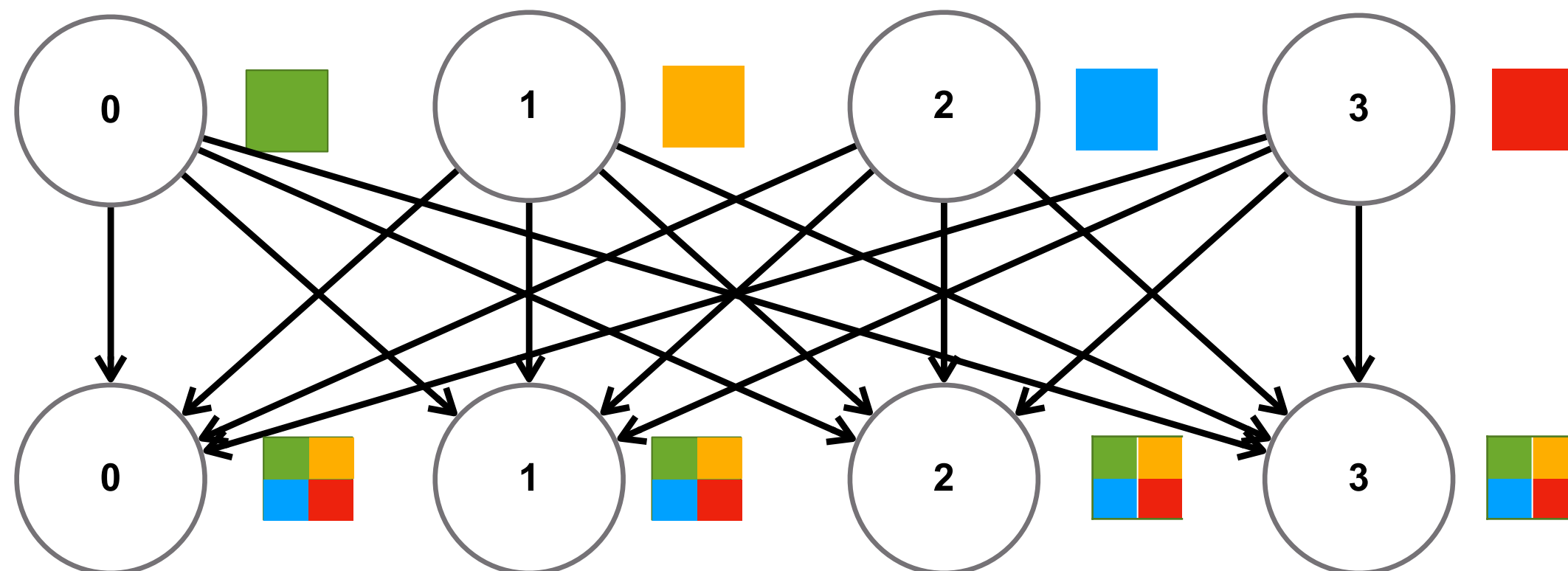
На каждом шаге выполняется одна операция Send и Merge

Note: N — число воркеров

Коллективные операции

Наивная реализация All-Reduce — Parallel Reduce

Step 1



Time: $O(1)$ <- лучше

Bandwidth: $O(N^2)$ <- хуже

Все Reduce операции выполняются одновременно

Note: N — число воркеров

Коллективные операции

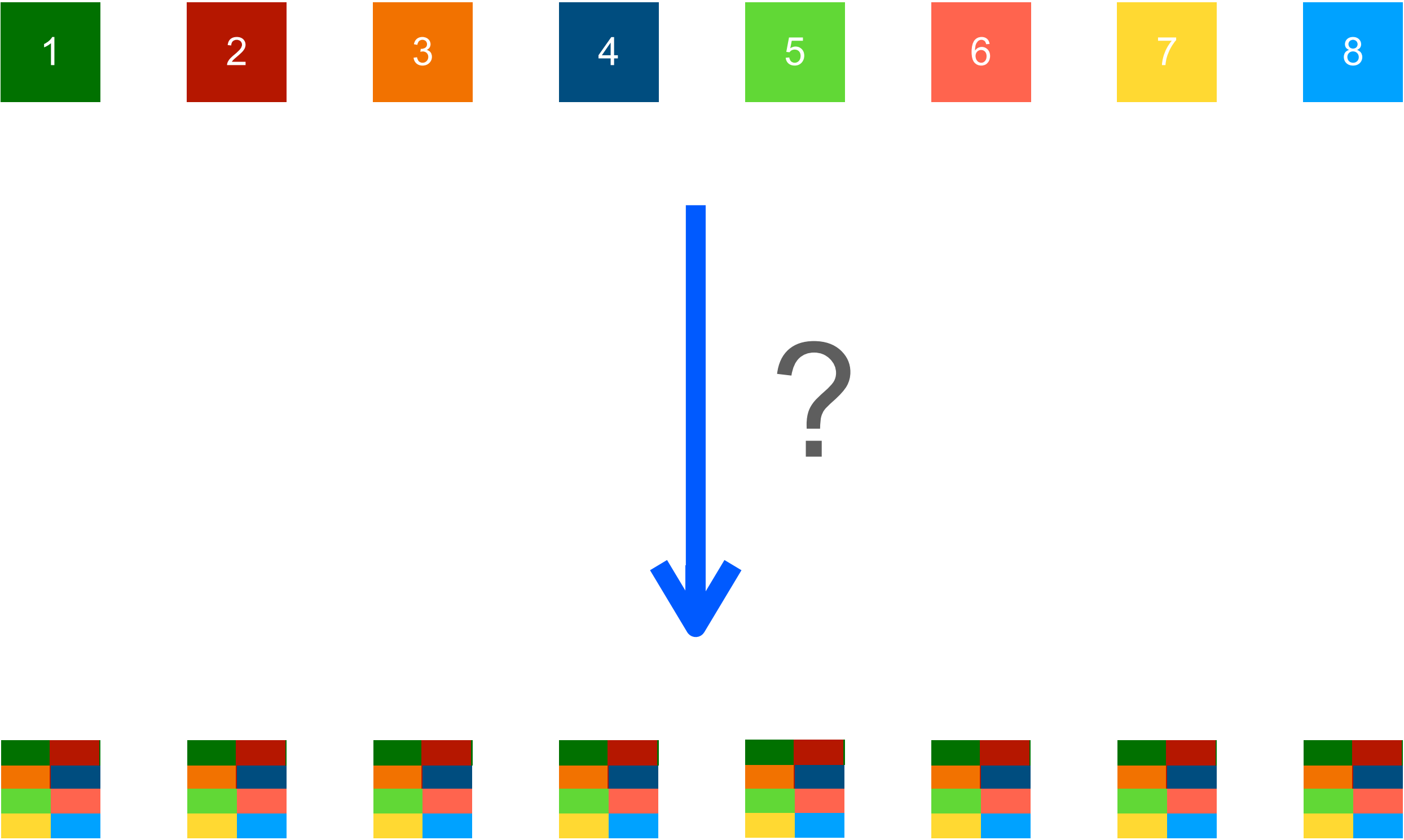
Сравнение разных распределенных методов агрегации

	Time	Peak Node Bandwidth	Total Bandwidth
Parameter Server	$O(1)$	$O(N)$	$O(N)$
All-Reduce - Sequential	$O(N)$	$O(N)$	$O(N)$
All-Reduce - Ring	$O(N)$	$O(1)$	$O(N)$
All-Reduce - Parallel	$O(1)$	$O(N)$	$O(N^2)$

Можем ли мы получить преимущества Ring и Parallel одновременно?

Коллективные операции

Recursive Halving All-Reduce



Коллективные операции

Recursive Halving All-Reduce



Коллективные операции

Recursive Halving All-Reduce

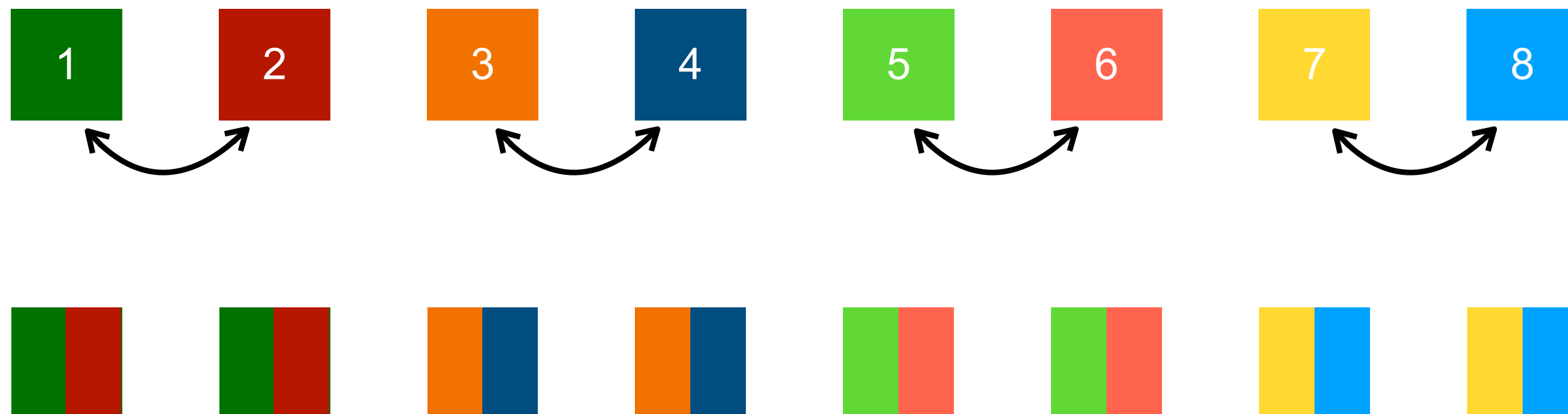
Step 1 - Каждая нода обменивается с соседом со сдвигом 1



Коллективные операции

Recursive Halving All-Reduce

Step 1 - Каждая нода обменивается с соседом со сдвигом 1



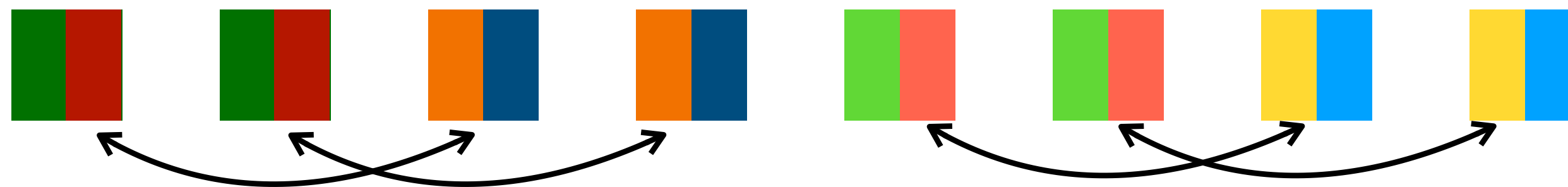
Коллективные операции

Recursive Halving All-Reduce

Step 1 - Каждая нода обменивается с соседом со сдвигом 1



Step 2 - Каждая нода обменивается с соседом со сдвигом 2



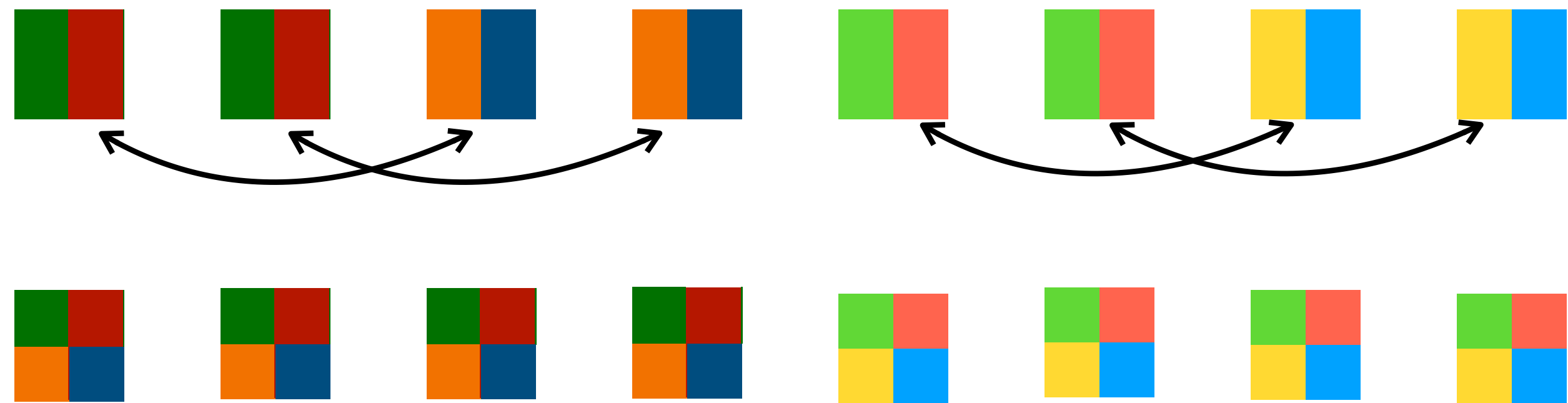
Коллективные операции

Recursive Halving All-Reduce

Step 1 - Каждая нода обменивается с соседом со сдвигом 1



Step 2 - Каждая нода обменивается с соседом со сдвигом 2



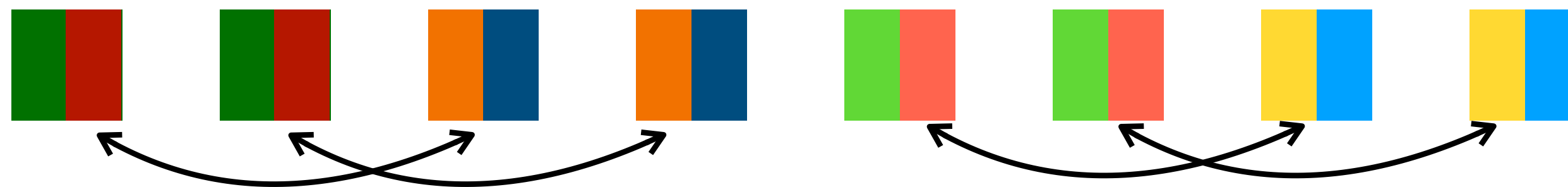
Коллективные операции

Recursive Halving All-Reduce

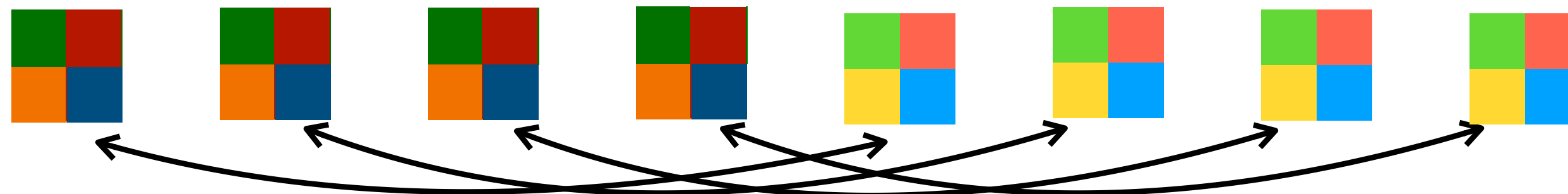
Step 1 - Каждая нода обменивается с соседом со сдвигом 1



Step 2 - Каждая нода обменивается с соседом со сдвигом 2



Step 3 - Каждая нода обменивается с соседом со сдвигом 4



Коллективные операции

Recursive Halving All-Reduce

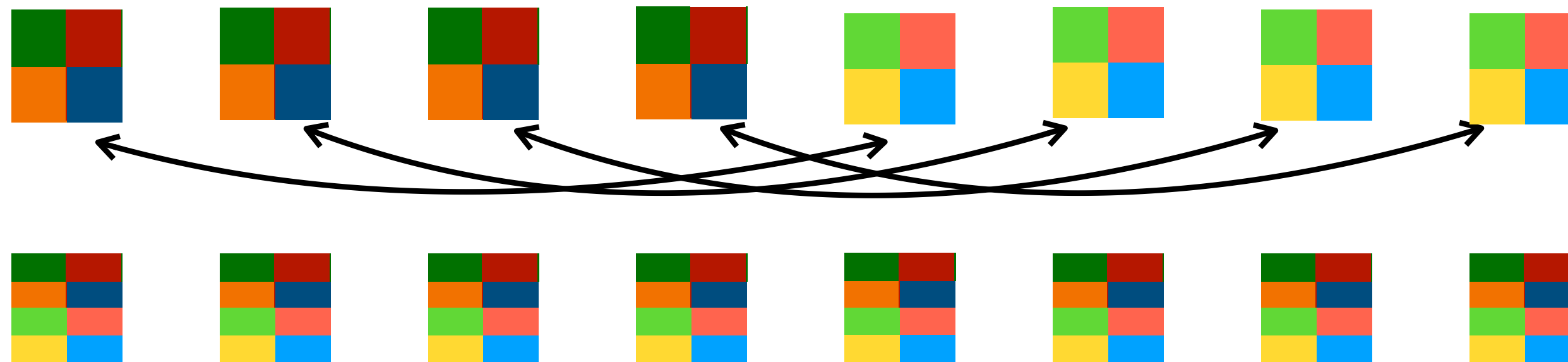
Step 1 - Каждая нода обменивается с соседом со сдвигом 1



Step 2 - Каждая нода обменивается с соседом со сдвигом 2



Step 3 - Каждая нода обменивается с соседом со сдвигом 4



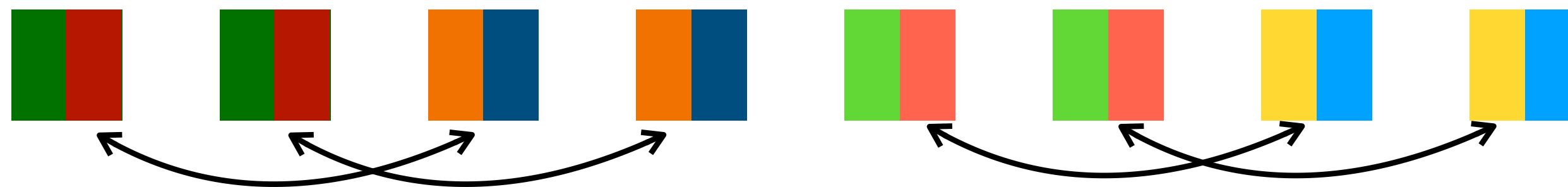
Коллективные операции

Recursive Halving All-Reduce

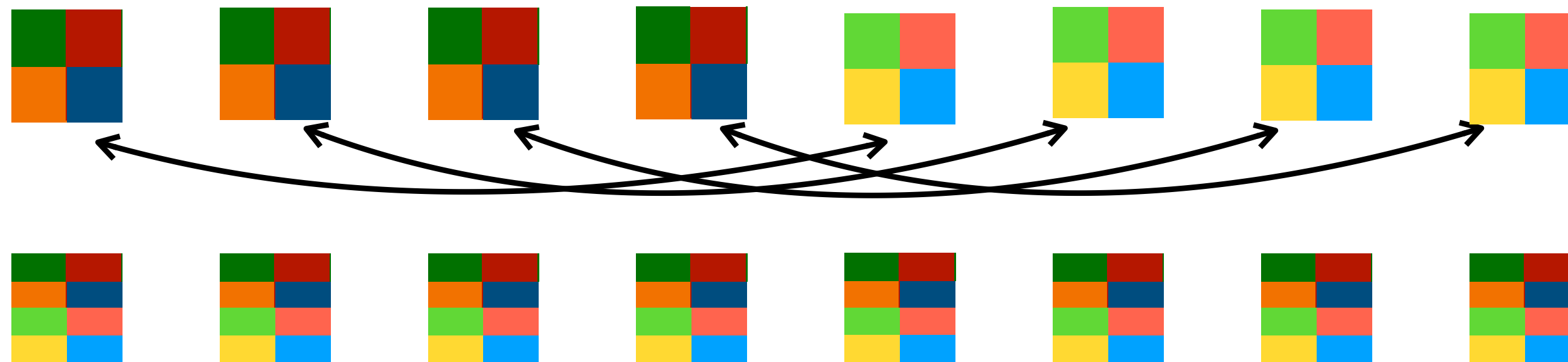
Step 1 - Каждая нода обменивается с соседом со сдвигом 1



Step 2 - Каждая нода обменивается с соседом со сдвигом 2



Step 3 - Каждая нода обменивается с соседом со сдвигом 4



Для N воркеров,
All-Reduce занимает
 $\log(N)$ шагов

План лекции

01

Бэкграунд и мотивация

Зачем нам распределенное обучение?

02

Виды параллелизма

Обзорно: DP, PP, TP, SP

03

Data Parallelism

Разбираем Parameter Server

04

Коллективные операции

Эффективный All-Reduce

05

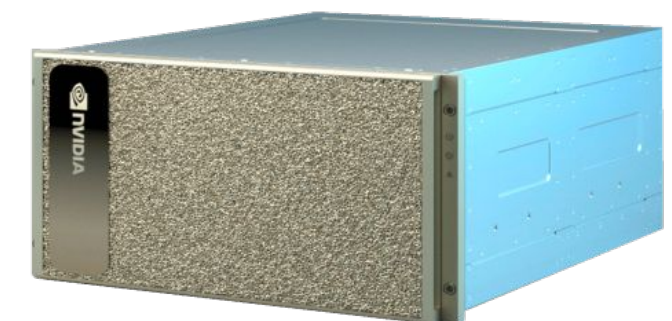
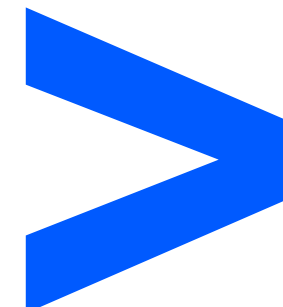
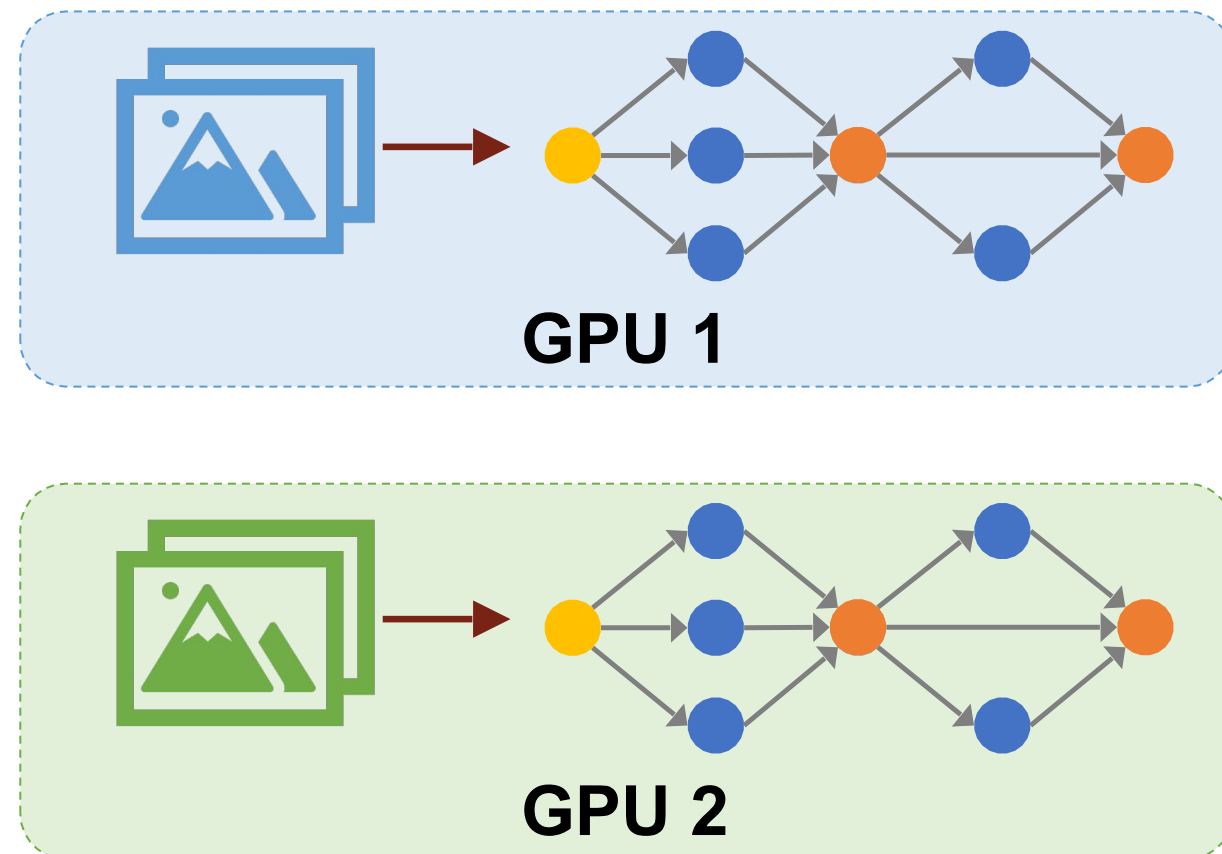
Оптимизация памяти в DP

ZeRO-1/2/3 и FSDP



ZeRO и FSDP

Учим большую модель, напр. GPT-3 175B:



$175B * 2 \text{ Bytes (fp16)}$
 $= 350GB \text{ Memory}$

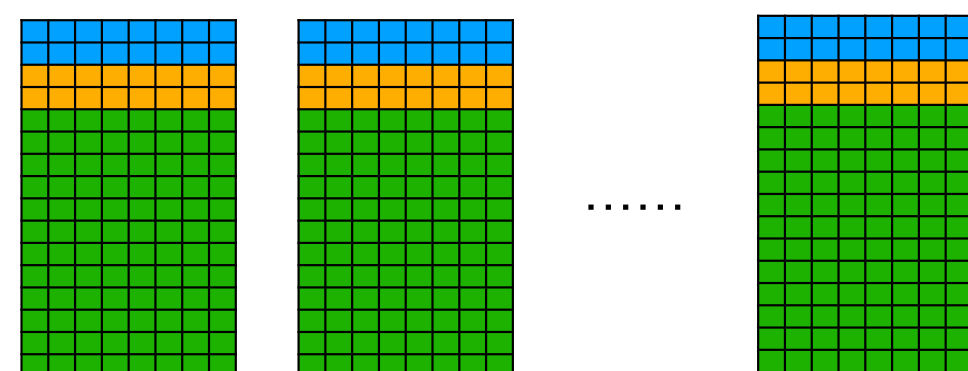
Nvidia H100 80GB

Даже лучшие GPU **не могут** вместить веса в память!

Более того, для обучения еще нужно хранить градиенты и состояние оптимизатора

ZeRO и FSDP

DeepSpeed Zero для оптимизации памяти



Baseline

■: weights, 2 bytes

■: gradients, 2 bytes

■: optimizer states, K bytes for Adam

* K=12 from ZeRO: “fp32 copy of the parameters, momentum and variance”

N : number of GPUs ψ : number of parameters

- Наивный DP имеет N копий весов, градиентов и стейтов оптимизатора
- Это избыточно и легко приводит к OOM при обучении больших моделей

$$\underbrace{(2\text{bytes})}_{\text{weights}} + \underbrace{(2\text{bytes})}_{\text{gradients}} + \underbrace{(12\text{bytes})}_{\text{optim states}} \psi$$

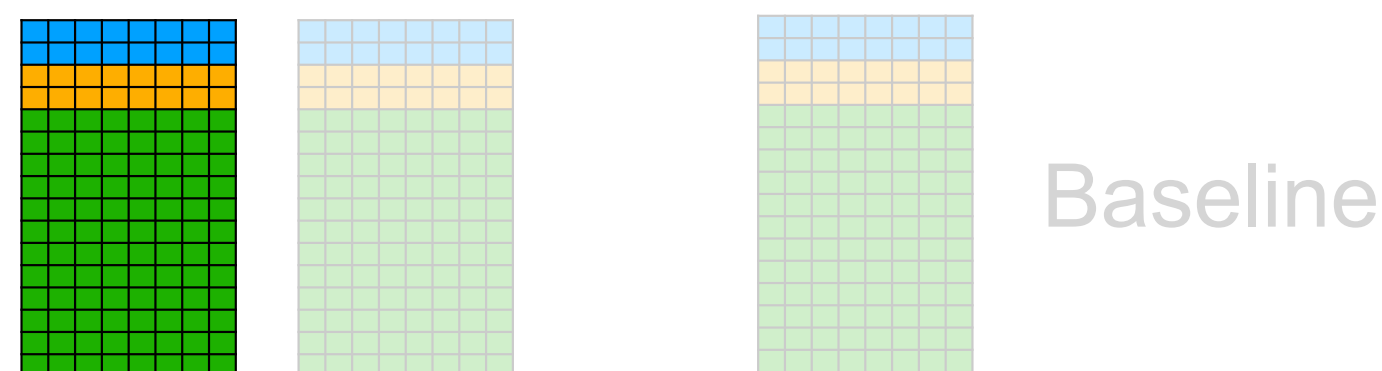
- Даже имея H100 с памятью в 80GB, самая большая обучаемая модель это

$$80\text{GB}/16\text{Bytes} = 5.0B$$

далековато от 175B!

ZeRO и FSDP

DeepSpeed Zero для оптимизации памяти



Baseline

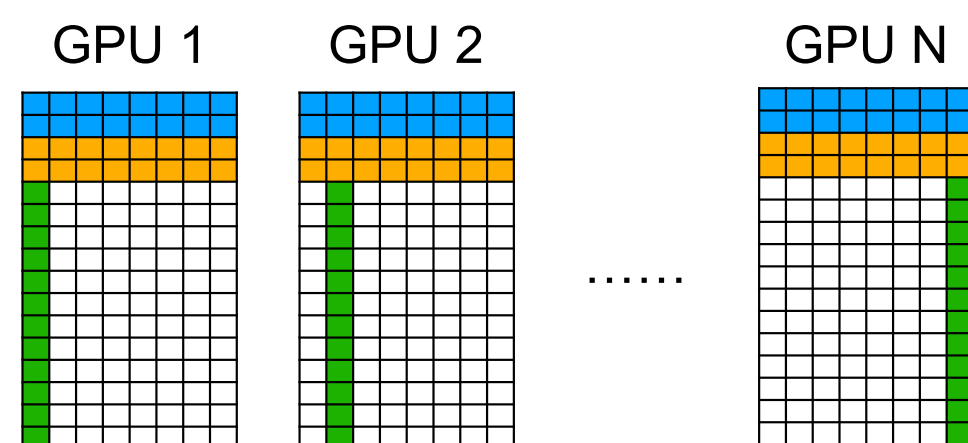
: weights, 2 bytes

: gradients, 2 bytes

: optimizer states, K bytes for Adam

* K=12 from ZeRO: “fp32 copy of the parameters, momentum and variance”

N : number of GPUs ψ : number of parameters



ZeRO-1

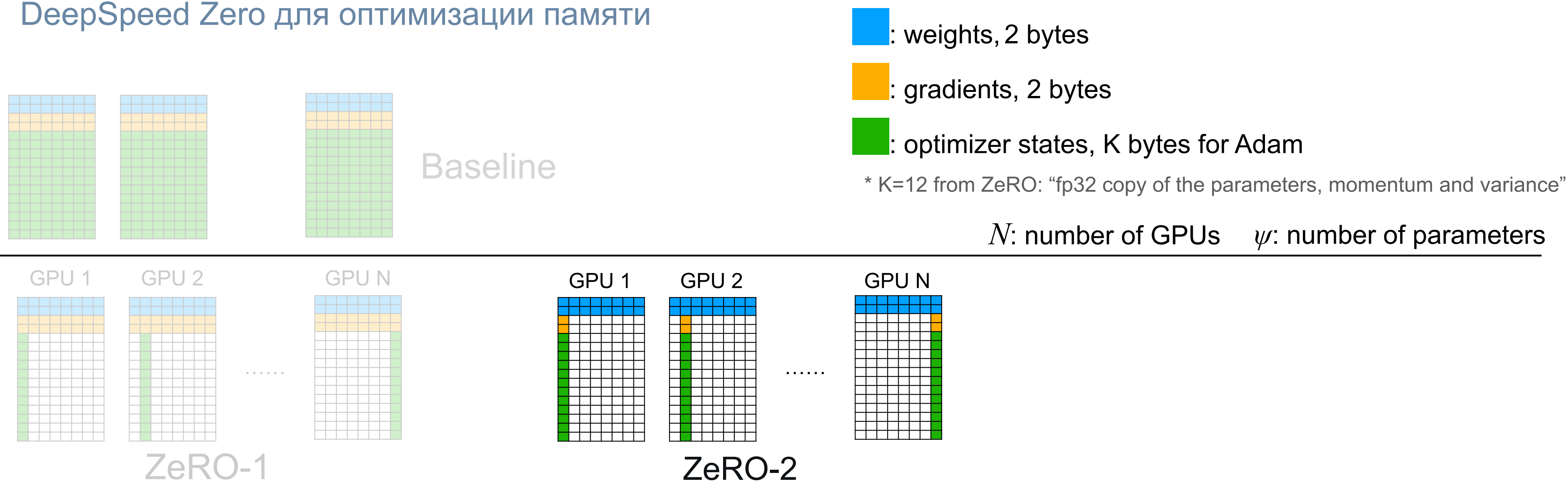
- ZeRO-1: шардируем **стейты оптимизатора**

Потребление памяти самой большой обучаемой модели (N=64)

$$\left(\underbrace{2\text{bytes}}_{\text{weights}} + \underbrace{2\text{bytes}}_{\text{gradients}} + \underbrace{12/N \text{ bytes}}_{\text{optim states}} \right) \psi \quad 80\text{GB}/4.2\text{Bytes} = 19B$$

ZeRO и FSDP

DeepSpeed Zero для оптимизации памяти



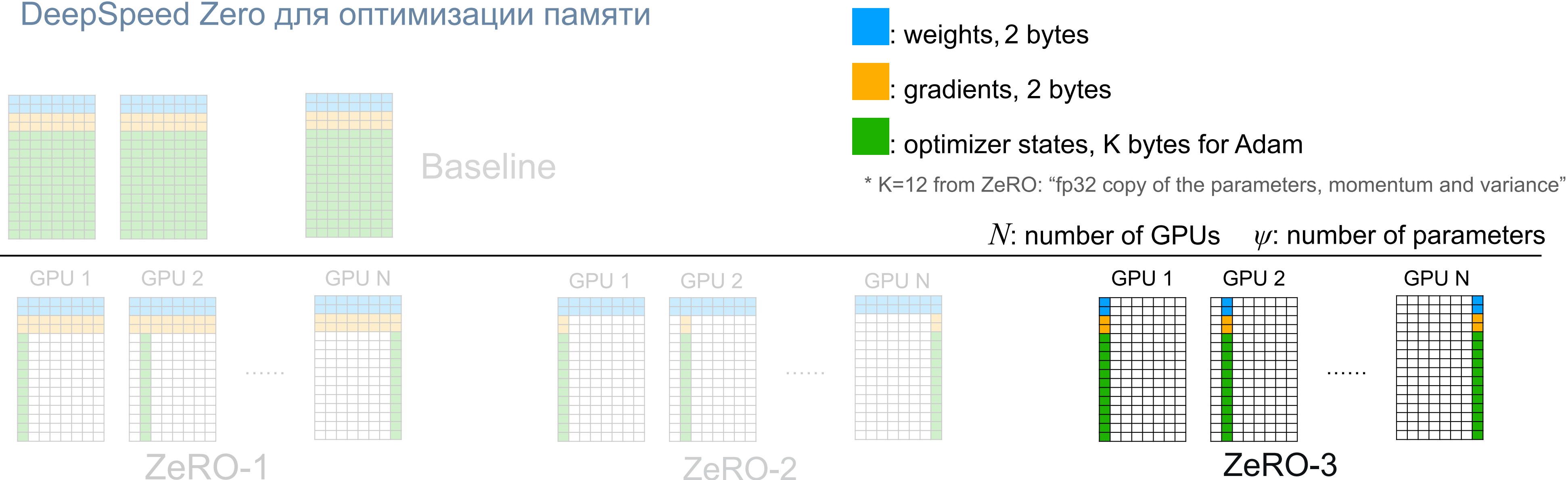
- ZeRO-2: шардируем **стейты оптимизатора** и **градиенты**

Потребление памяти самой большой обучаемой модели ($N=64$)

$$\left(\underbrace{2\text{bytes}}_{\text{weights}} + \underbrace{\frac{2}{N}\text{ bytes}}_{\text{gradients}} + \underbrace{\frac{12}{N}\text{ bytes}}_{\text{optim states}} \right) \psi \quad 80\text{GB}/2.2\text{Bytes} = 36B$$

ZeRO и FSDP

DeepSpeed Zero для оптимизации памяти



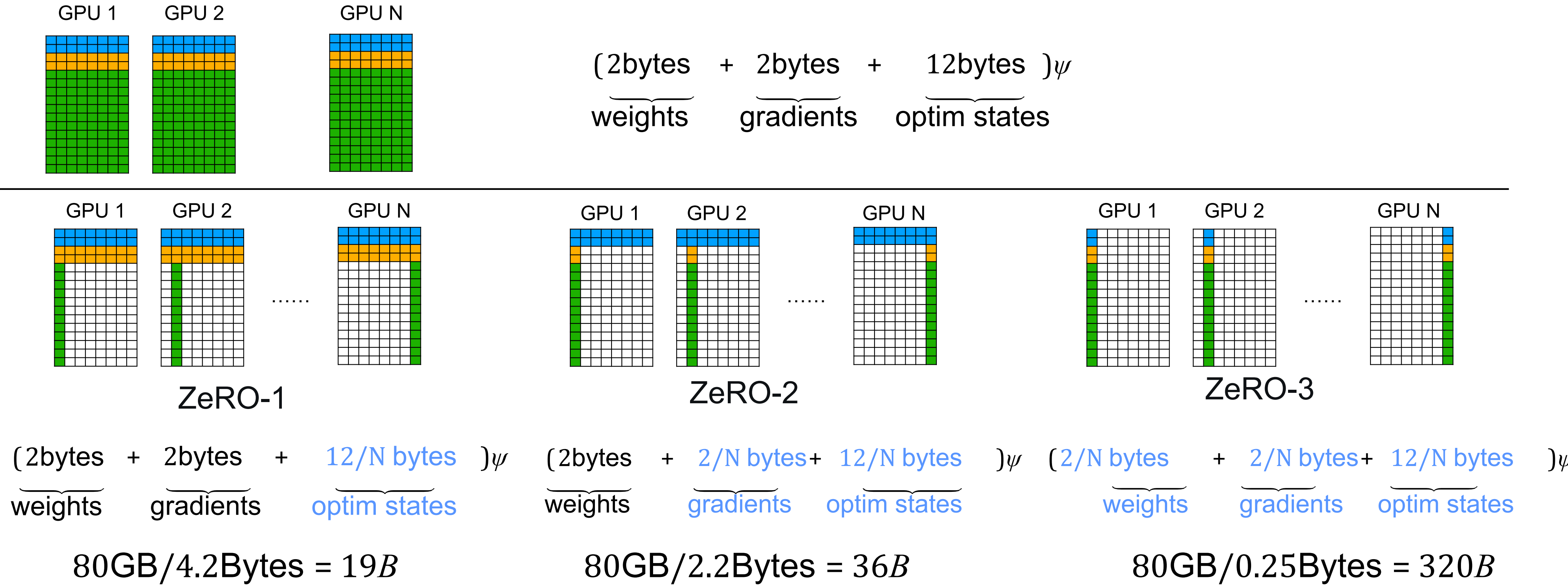
- ZeRO-3: шардируем **стейты оптимизатора, градиенты и веса**

Потребление памяти самой большой обучаемой модели ($N=64$)

$$\left(\underbrace{2/N \text{ bytes}}_{\text{weights}} + \underbrace{2/N \text{ bytes}}_{\text{gradients}} + \underbrace{12/N \text{ bytes}}_{\text{optim states}} \right) \psi \quad 80\text{GB}/0.25\text{Bytes} = 320B$$

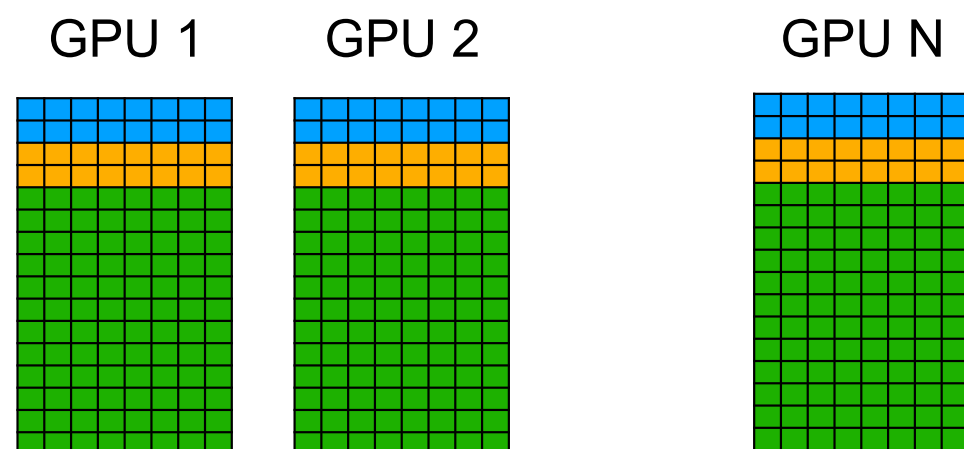
ZeRO и FSDP

Сравниваем ZeRO-1/2/3; FSDP

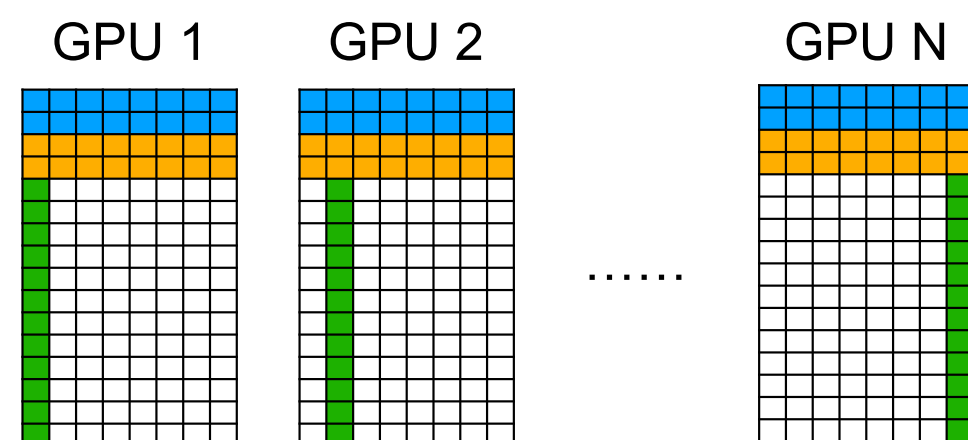


ZeRO и FSDP

Сравниваем ZeRO-1/2/3; FSDP



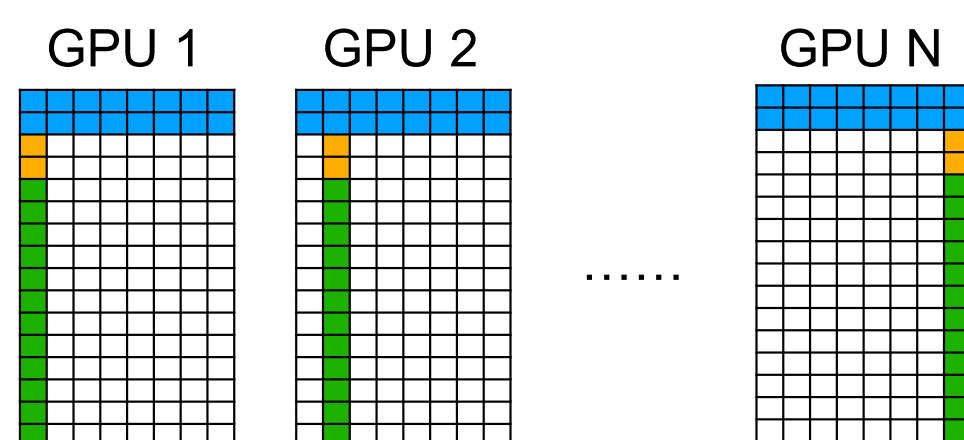
$$\underbrace{(2\text{bytes})}_{\text{weights}} + \underbrace{(2\text{bytes})}_{\text{gradients}} + \underbrace{(12\text{bytes})}_{\text{optim states}} \psi$$



ZeRO-1

$$\underbrace{(2\text{bytes})}_{\text{weights}} + \underbrace{(2\text{bytes})}_{\text{gradients}} + \underbrace{(12/N \text{ bytes})}_{\text{optim states}} \psi$$

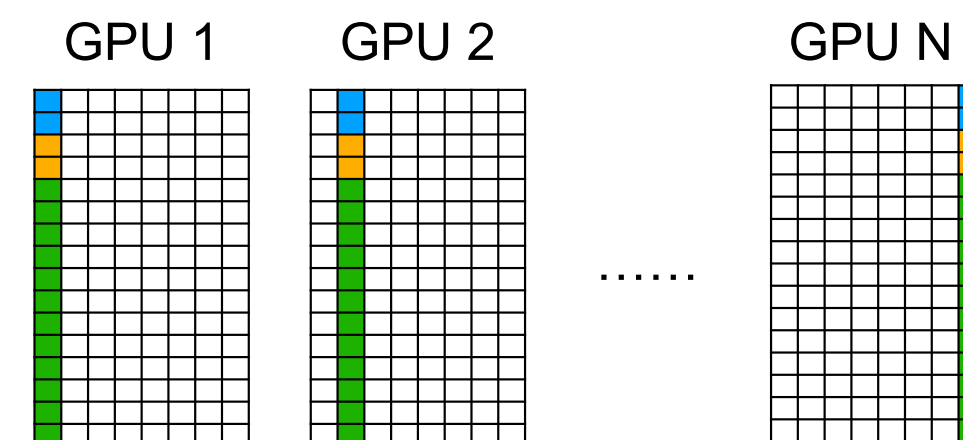
$$80\text{GB}/4.2\text{Bytes} = 19B$$



ZeRO-2

$$\underbrace{(2\text{bytes})}_{\text{weights}} + \underbrace{(2/N \text{ bytes})}_{\text{gradients}} + \underbrace{(12/N \text{ bytes})}_{\text{optim states}} \psi$$

$$80\text{GB}/2.2\text{Bytes} = 36B$$



ZeRO-3

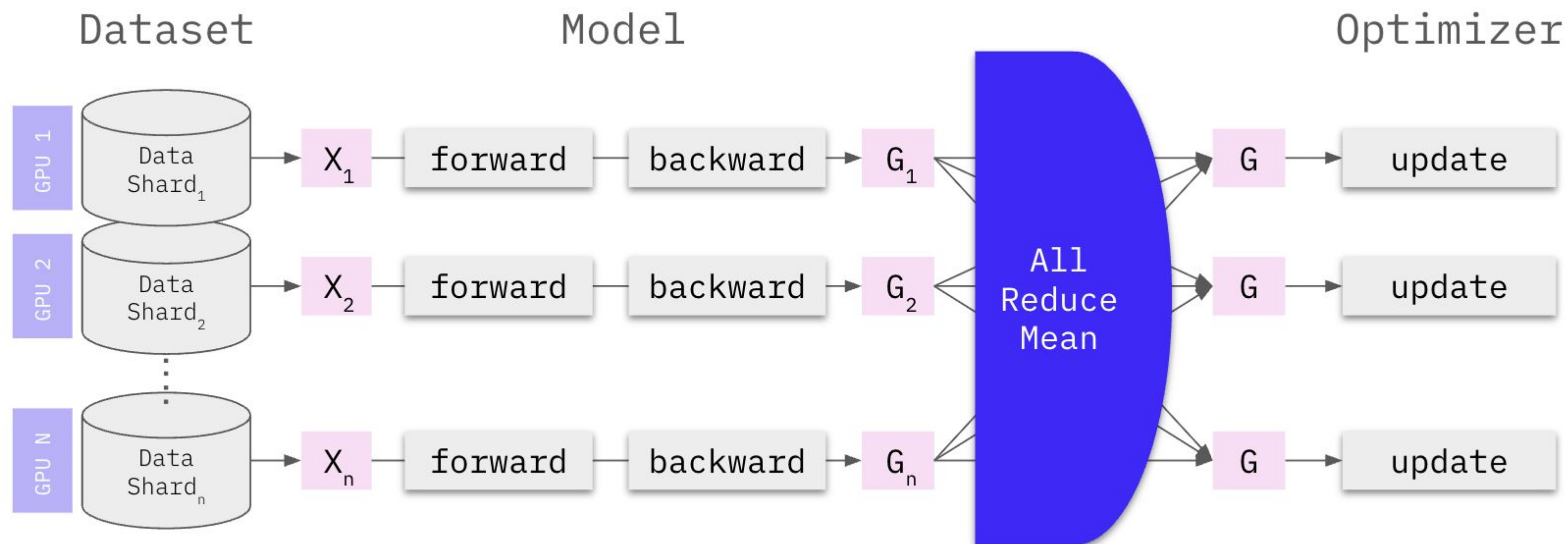
$$\underbrace{(2/N \text{ bytes})}_{\text{weights}} + \underbrace{(2/N \text{ bytes})}_{\text{gradients}} + \underbrace{(12/N \text{ bytes})}_{\text{optim states}} \psi$$

$$80\text{GB}/0.25\text{Bytes} = 320B$$

В PyTorch, ZeRO-3 реализован в [FullyShardedDataParallel](#), или же FSDP

ZeRO и FSDP

Подход DDP

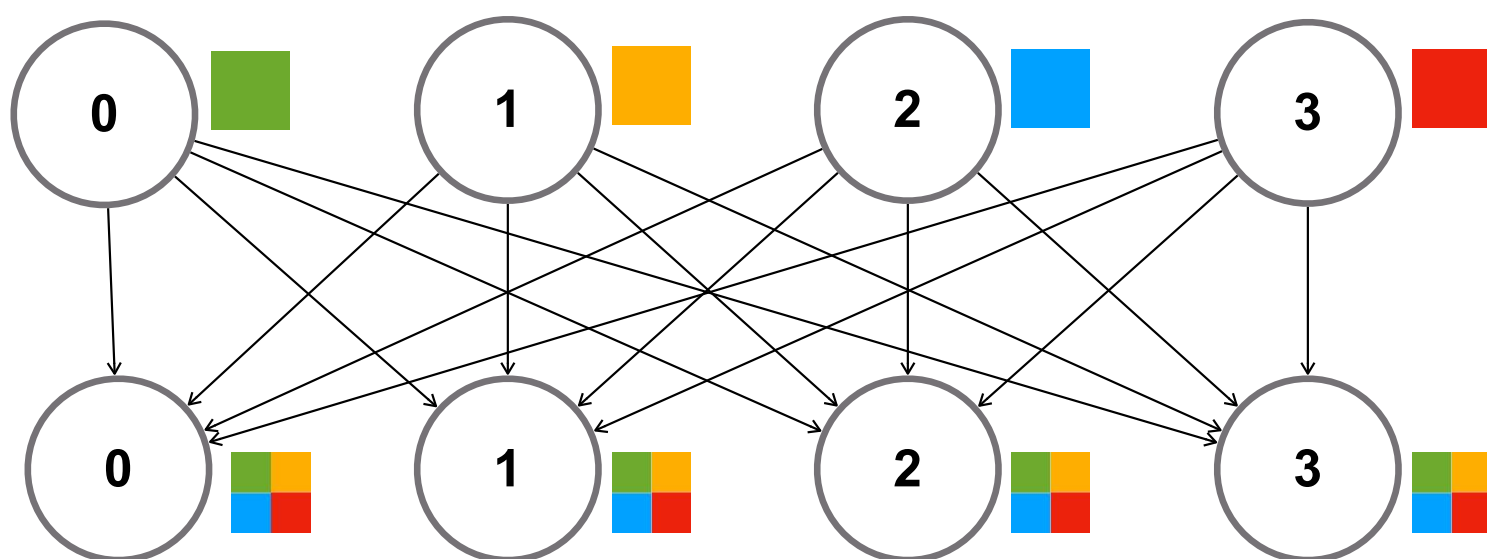


ZeRO и FSDP

Проблема DDP

- Синхронизация градиентов в DDP — это All-Reduce
- Можем получить OOM !

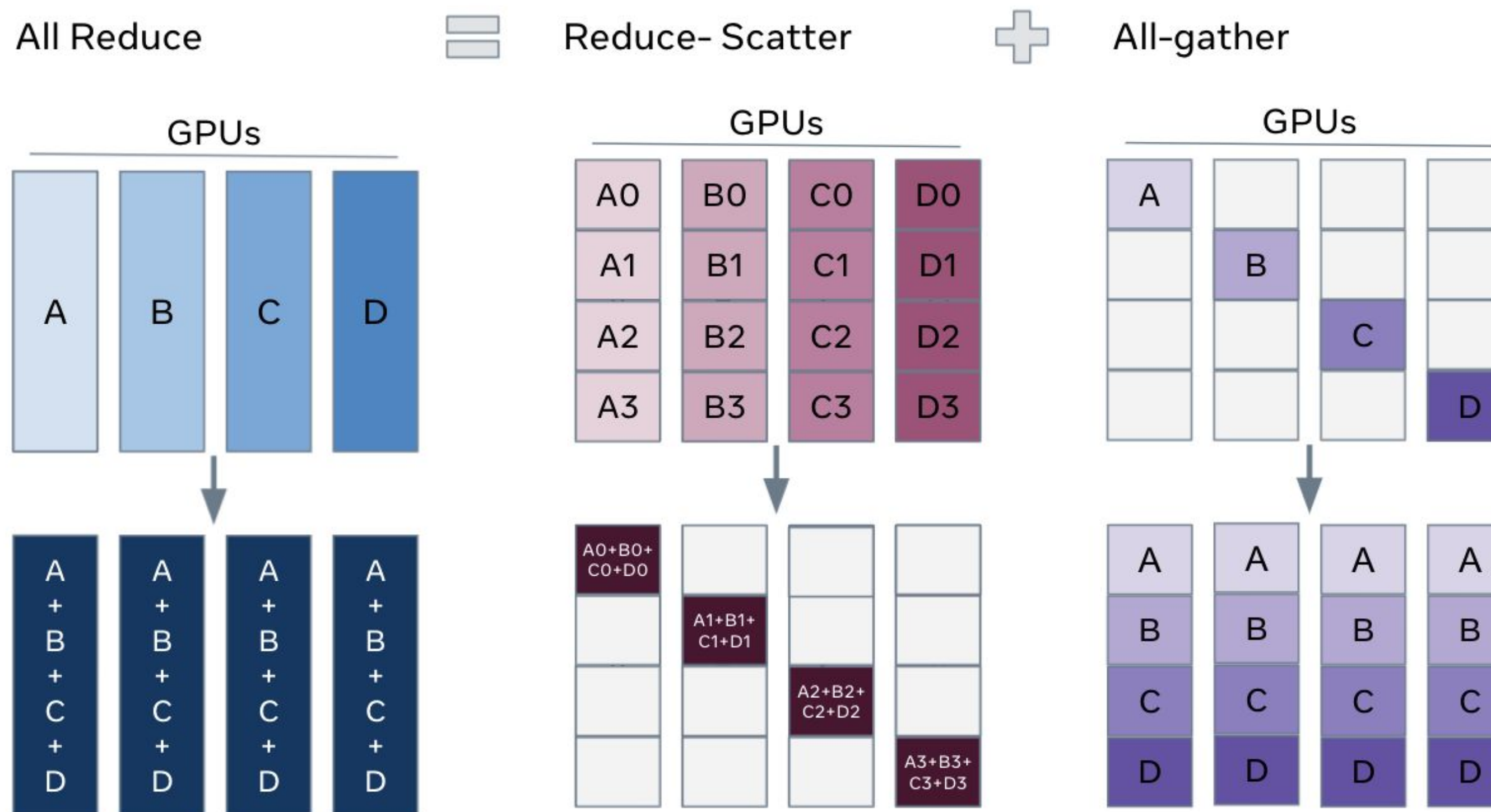
All-Reduce



```
def demo_basic(rank, world_size):  
    print(f"Running basic DDP example on rank {rank}.")  
    setup(rank, world_size)  
  
    # create model and move it to GPU with id rank  
    model = ToyModel().to(rank)  
    ddp_model = DDP(model, device_ids=[rank])  
  
    loss_fn = nn.MSELoss()  
    optimizer = optim.SGD(ddp_model.parameters(), lr=0.001)  
  
    optimizer.zero_grad()  
    outputs = ddp_model(torch.randn(20, 10))  
    labels = torch.randn(20, 5).to(rank)  
    loss_fn(outputs, labels).backward()  
    optimizer.step()  
  
    cleanup()  
    print(f"Finished running basic DDP example on rank {rank}.")
```

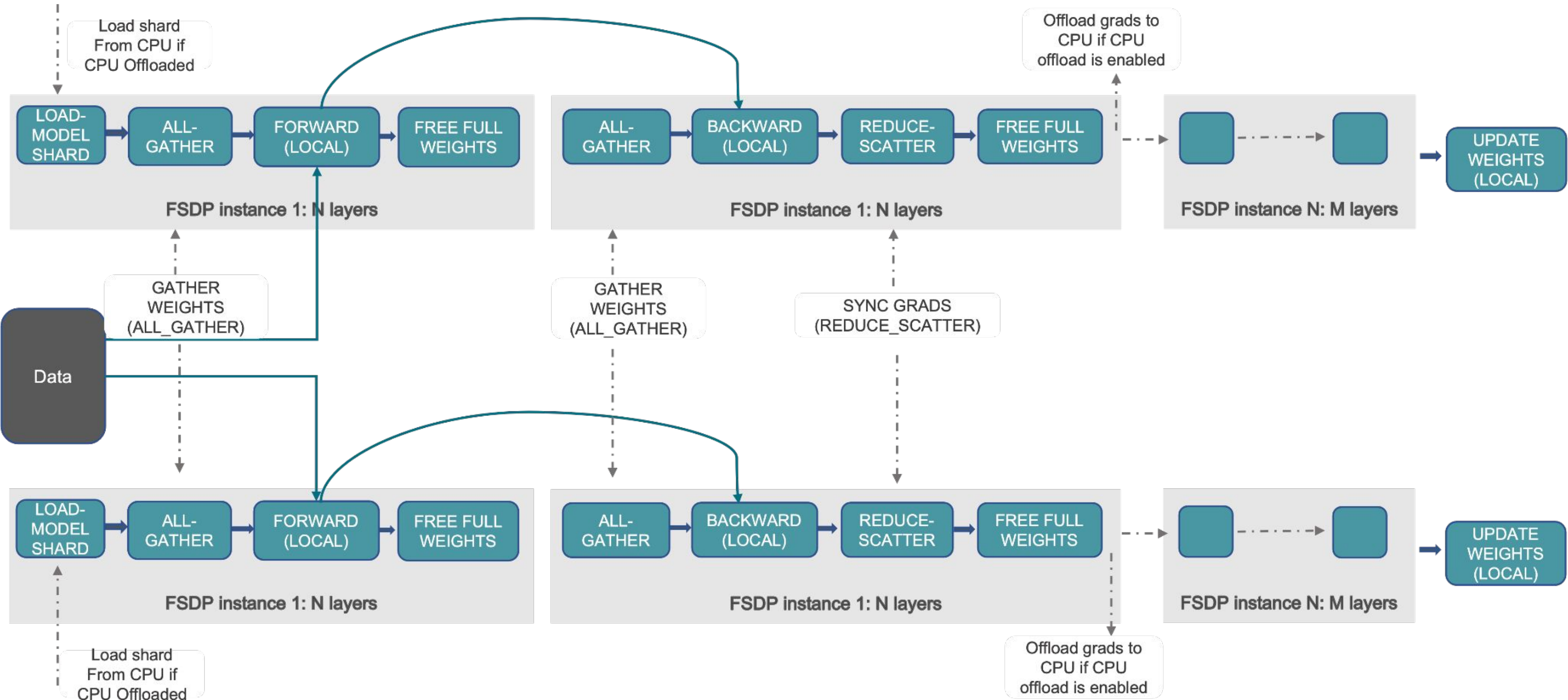
ZeRO и FSDP

Декомпозиция All-Reduce



ZeRO и FSDP

FSDP Workflow

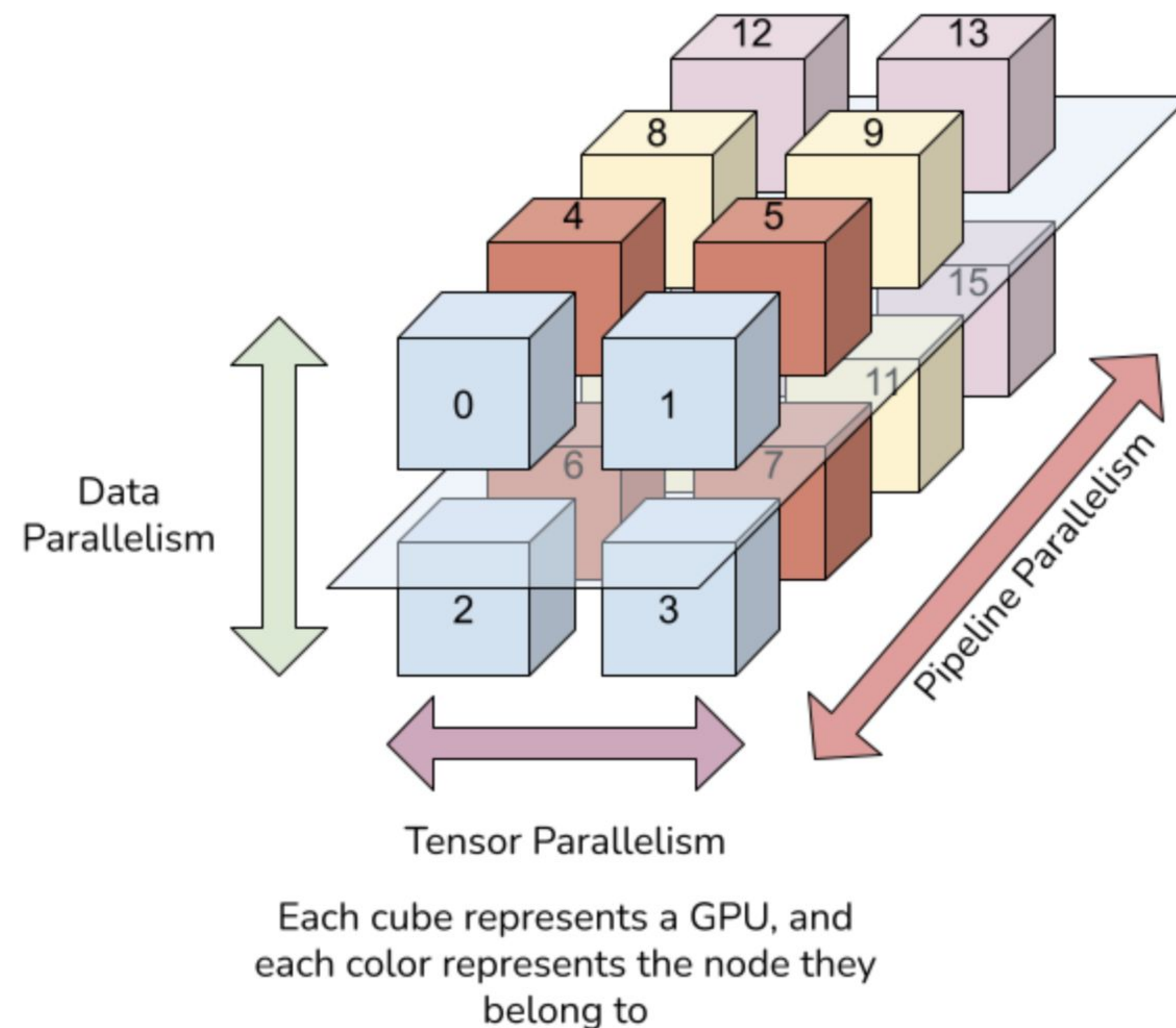


ZeRO и FSDP

Иерархия шардирования: Module-level vs Parameter-level

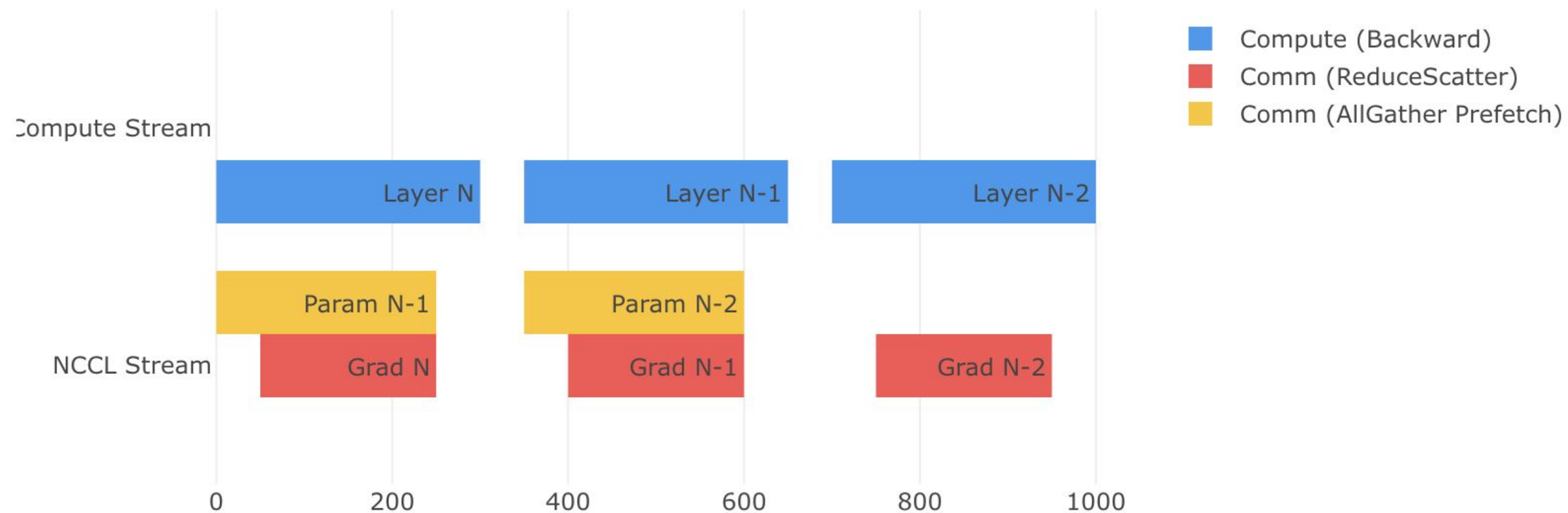
Как именно мы шардируем?

- Parameter-level sharding (Tensor Parallelism): разрезание конкретных матриц весов и изменение математики умножения
- Module-level sharding (FSDP): FSDP ничего не знает про внутренности слоя. Он работает на уровне nn.Module
- FSDP перехватывает параметры перед forward() и после backward() у всего модуля, подменяя шардированные FlatParameter на полные и обратно



ZeRO и FSDP

Асинхронность FSDP



ZeRO и FSDP

Auto-Wrap Policy в FSDP

- Если обернуть модель монолитно -> All-Gather попытается собрать 100% весов на одном GPU -> снова OOM !
- auto_wrap_policy: рекурсивный обход дерева модулей по политике. Как только политика встречает нужный блок -> создает для него изолированный FSDP-юнит со своим FlatParameter

```
FullyShardedDataParallel(  
  (_fsdp_wrapped_module): FlattenParamsWrapper(  
    (_fpw_module): Net(  
      (conv1): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1))  
      (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1))  
      (dropout1): Dropout(p=0.25, inplace=False)  
      (dropout2): Dropout(p=0.5, inplace=False)  
      (fc1): FullyShardedDataParallel(  
        (_fsdp_wrapped_module): FlattenParamsWrapper(  
          (_fpw_module): Linear(in_features=9216, out_features=128, bias=True)  
        )  
      )  
      (fc2): Linear(in_features=128, out_features=10, bias=True)  
    )  
  )  
)
```

ZeRO и FSDP

Auto-Wrap Policy в FSDP

```
def get_llama_wrapper(model, mesh=None):  
    """  
    Configures FSDP wrapping for Llama architecture.  
    """  
  
    # Identify the repeating layer class  
    # For GPT-2 use GPT2Block, for BERT use BertLayer, etc.  
    llama_auto_wrap_policy = ModuleWrapPolicy({LlamaDecoderLayer})  
  
    wrapped_model = FSDP(  
        model,  
        auto_wrap_policy=llama_auto_wrap_policy,  
        device_id=torch.cuda.current_device(),  
        use_orig_params=True, # Required for torch.compile  
    )  
  
    return wrapped_model
```

ZeRO и FSDP

Sharding Strategies в FSDP

- `sharding_strategy` (`Union` [`ShardingStrategy` , `Literal` [`'FULL_SHARD'` , `'SHARD_GRAD_OP'` , `'NO_SHARD'` , `'HYBRID_SHARD'`]]) –

Select whether to shard model parameters, gradients, optimizer states, or a combination of them. Available values are:

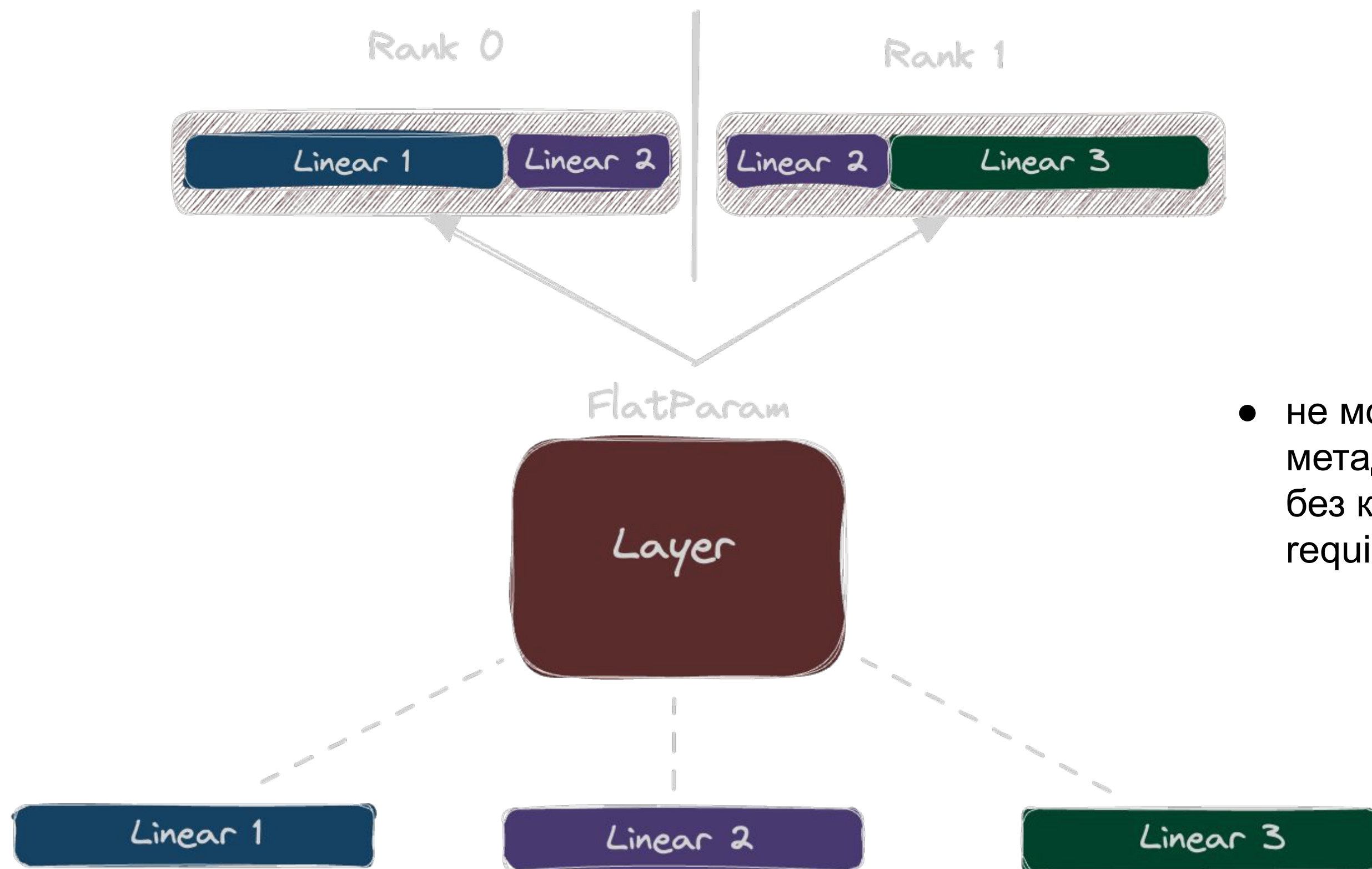
- `"FULL_SHARD"` : Shards model parameters, gradients, and optimizer states (default).
- `"SHARD_GRAD_OP"` : Shards gradients and optimizer states only. Model parameters get replicated.
- `"NO_SHARD"` : No sharding (identical to regular DDP).
- `"HYBRID_SHARD"` : Shards model parameters, gradients, and optimizer states within a single machine, but replicates across machines. See also the *device_mesh* parameter below.

Also accepts a `torch.distributed.fsdp.ShardingStrategy` enum value.

- `NO_SHARD`: ничего не шардируем; = DDP
- `SHARD_GRAD_OP`: полный аналог ZeRO Stage 2
- `FULL_SHARD` (Default): аналог ZeRO Stage 3, максимальная экономия VRAM

ZeRO и FSDP

Ограничения FlatParameter

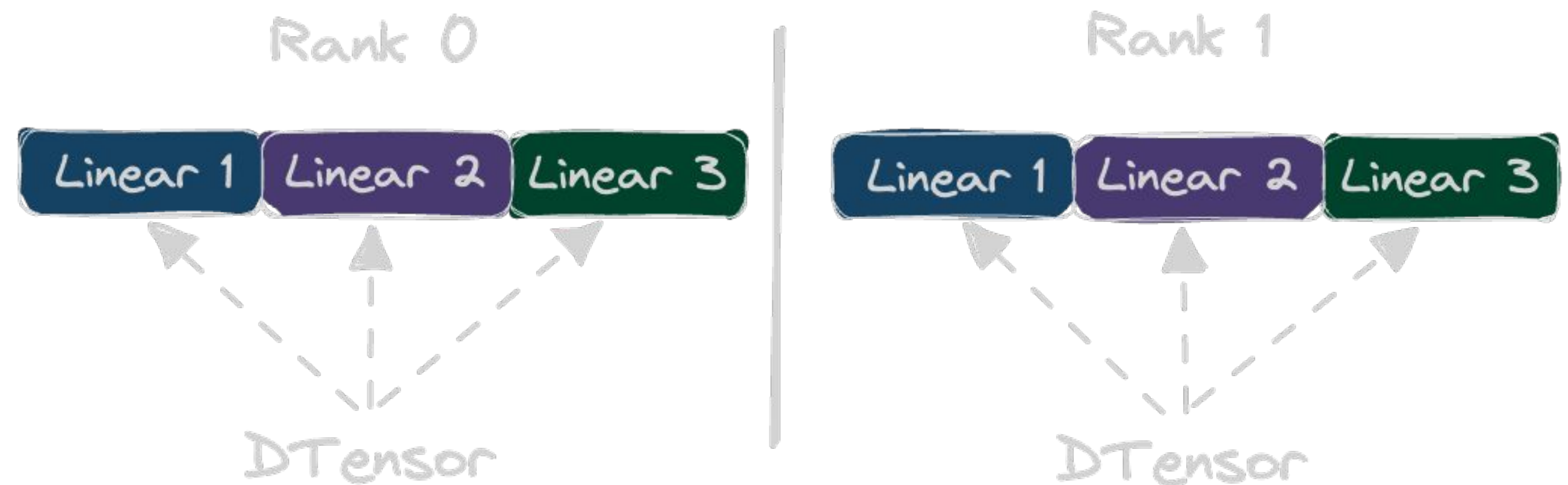


- не можем хранить метаданные у FlatParameter без костылей (напр. dtype, required_grad)

ZeRO и FSDP

Решение FSDP2: DTensor

- Новый механизм: DTensor, Distributed Tensor
- Отдельные слои снова могут иметь разные метаданные (e.g. `requires_grad`) -> можем учить лору !
- Gradient checkpointing работает из коробки, mixed precision работает из коробки



ZeRO и FSDP

API FSDP2

```
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
with torch.device("meta"):
    model = Transformer()
policy = ModuleWrapPolicy({TransformerBlock})
model = FSDP(model, auto_wrap_policy=policy)
def param_init_fn(module: nn.Module) -> None: ...
model = FSDP(model, auto_wrap_policy=policy, param_init_fn=param_init_fn)
```



```
with torch.device("meta"):
    model = Transformer()
for module in model.modules():
    if isinstance(module, TransformerBlock):
        fully_shard(module)
fully_shard(model)
for tensor in itertools.chain(model.parameters(), model.buffers()):
    assert tensor.device == torch.device("meta")

# Initialize the model after sharding
model.to_empty(device="cuda")
model.reset_parameters()
```

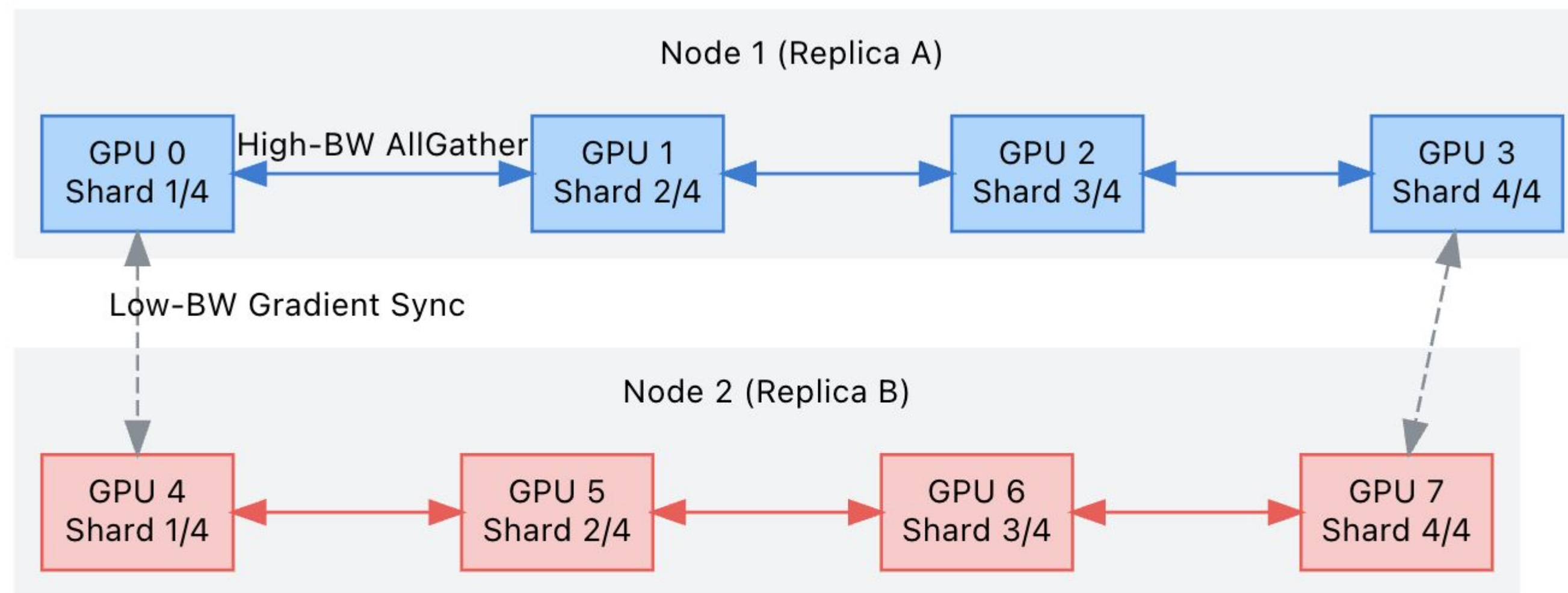
ZeRO и FSDP

FSDP vs DeepSpeed

FEATURE	FSDP	<u>ZERO STAGE 3 (DEEPSPEED)</u>
Integration	Native PyTorch	Requires DeepSpeed installation
Parameter sharding	Yes	Yes
Gradient sharding	Yes	Yes
Optimizer state sharding	Yes	Yes
CPU offloading	Limited	Full ZeRO-Infinity support
Ease of use	High	Moderate
Communication optimization	Good	More configurable

ZeRO и FSDP

Гибридное шардирование, HSDP

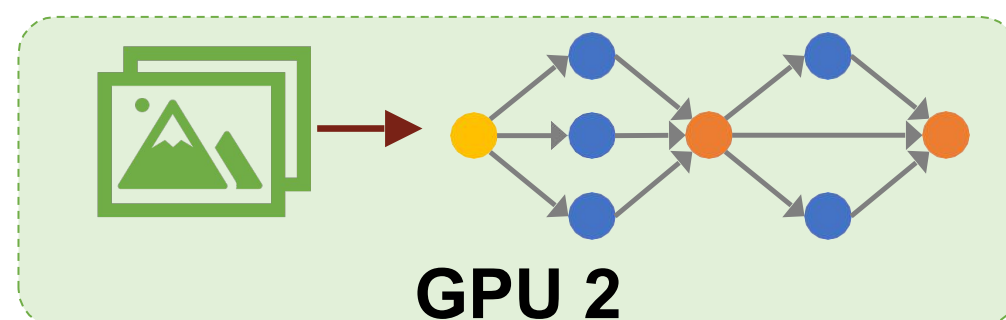
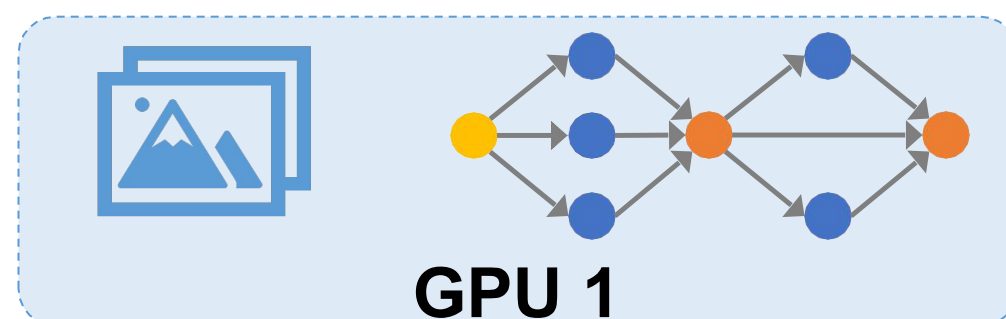


HSDP isolates heavy parameter gathering to high-speed intra-node links while using slower inter-node links only for gradient reduction.

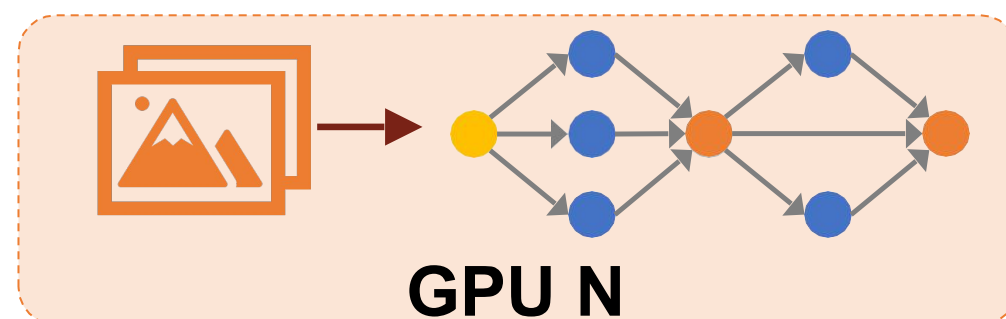
Резюме сегодняшней лекции

Data Parallelism & Sequence Parallelism:

- Разделяем данные

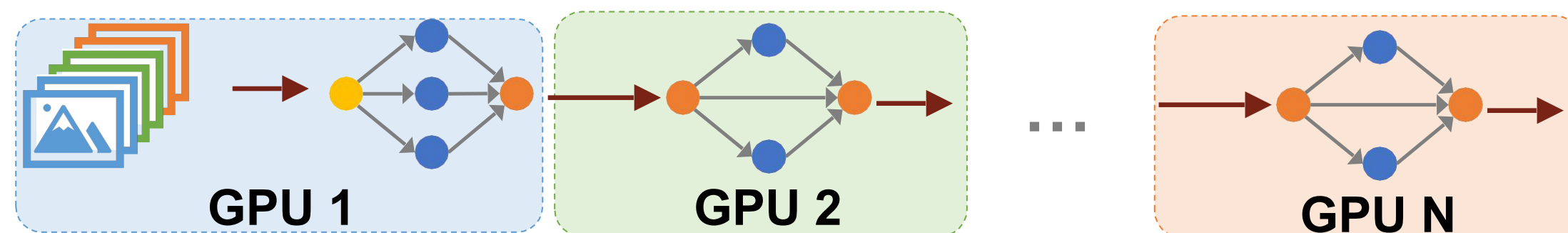


...



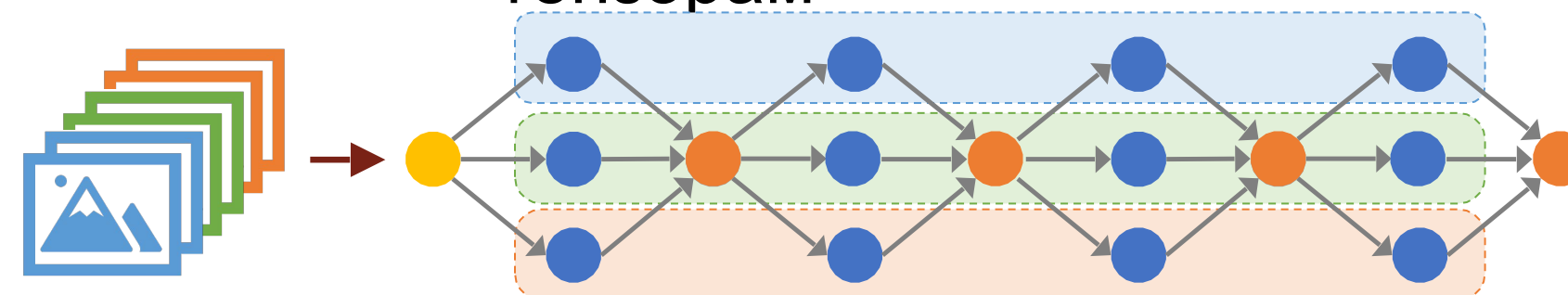
Pipeline Parallelism:

- Разделяем модель по слоям



Tensor Parallelism:

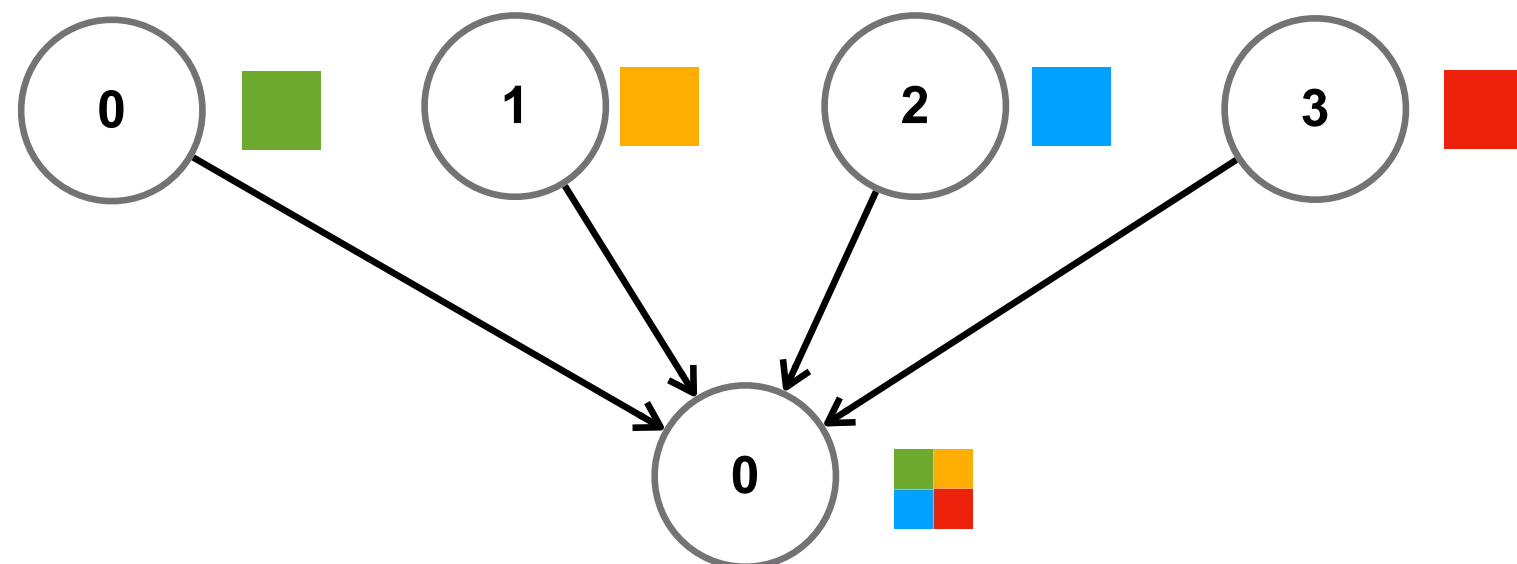
- Разделяем модель по тензорам



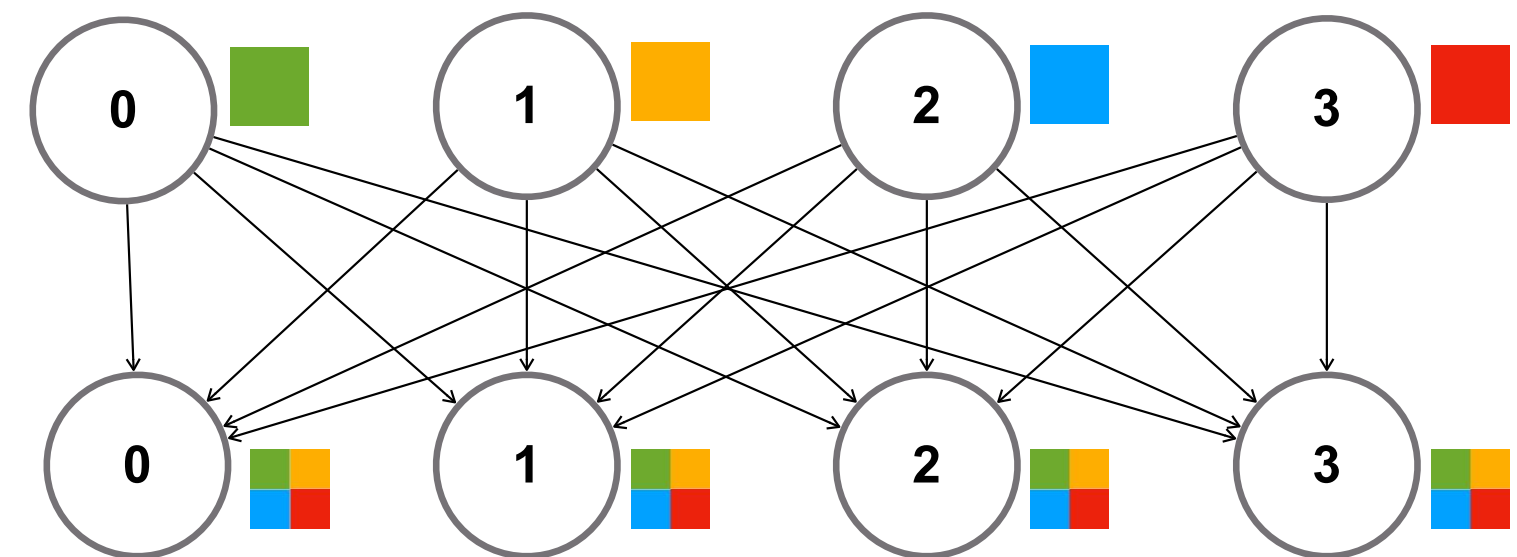
Резюме сегодняшней лекции

- Виды параллелизма
- Коллективные операции
(communication primitives)

Reduce

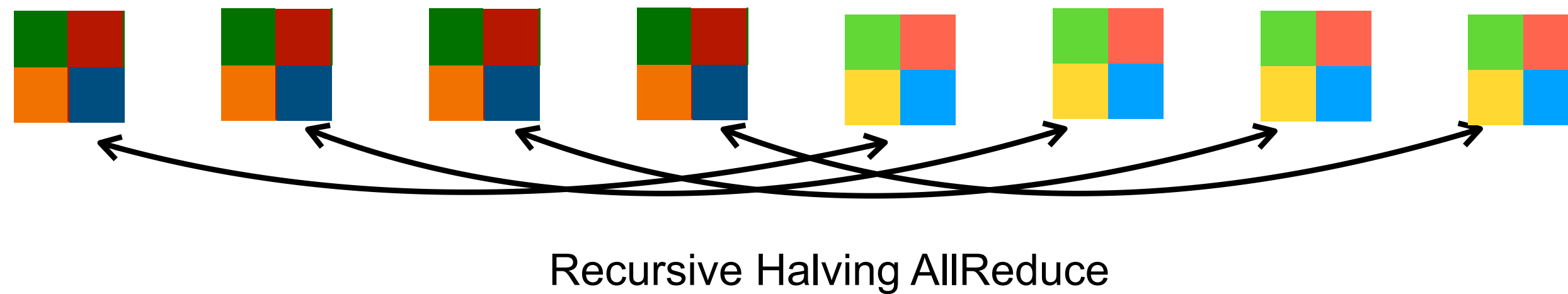


All-Reduce



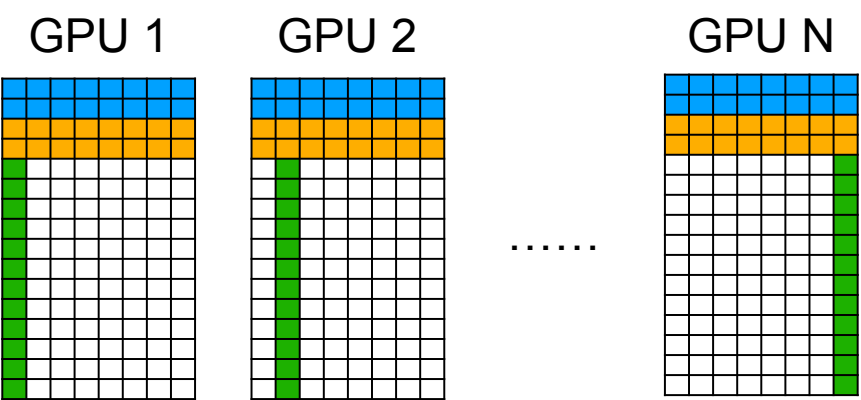
Резюме сегодняшней лекции

- Виды параллелизма
- Коллективные операции (communication primitives)
 - Ring AllReduce, Recursive Halving AllReduce



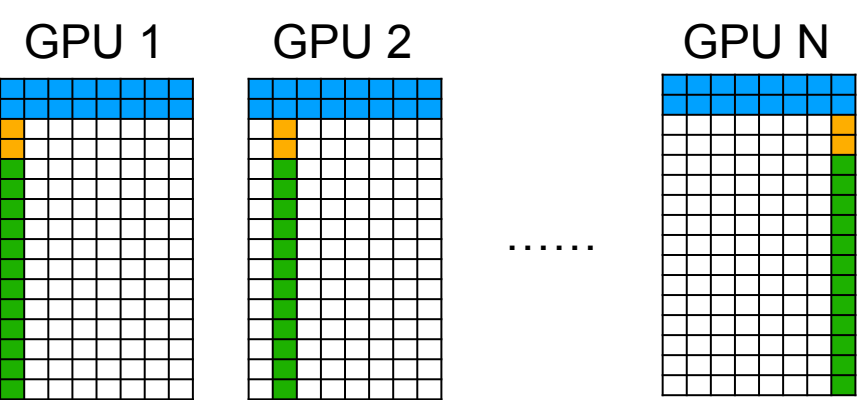
Резюме сегодняшней лекции

- Виды параллелизма
- Коллективные операции (communication primitives)
 - Ring AllReduce, Recursive Halving AllReduce
- ZeRO-1,2,3



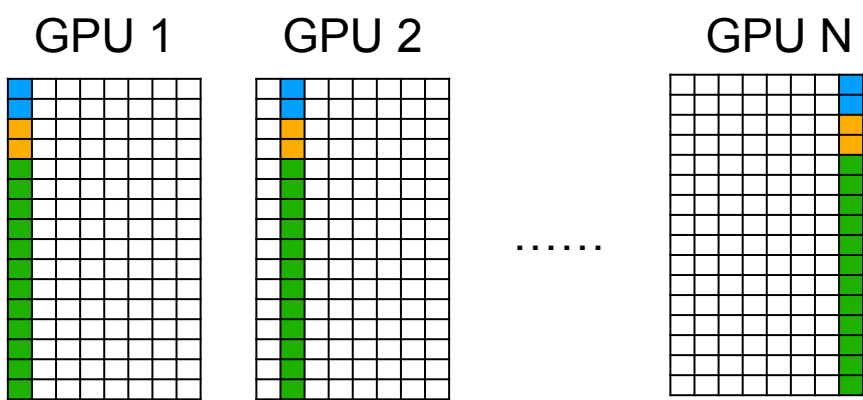
ZeRO-1

$$\left(\underbrace{2\text{bytes}}_{\text{weights}} + \underbrace{2\text{bytes}}_{\text{gradients}} + \underbrace{12/N \text{ bytes}}_{\text{optim states}} \right) \psi$$



ZeRO-2

$$\left(\underbrace{2\text{bytes}}_{\text{weights}} + \underbrace{2/N \text{ bytes}}_{\text{gradients}} + \underbrace{12/N \text{ bytes}}_{\text{optim states}} \right) \psi$$



ZeRO-3

$$\left(\underbrace{2/N \text{ bytes}}_{\text{weights}} + \underbrace{2/N \text{ bytes}}_{\text{gradients}} + \underbrace{12/N \text{ bytes}}_{\text{optim states}} \right) \psi$$

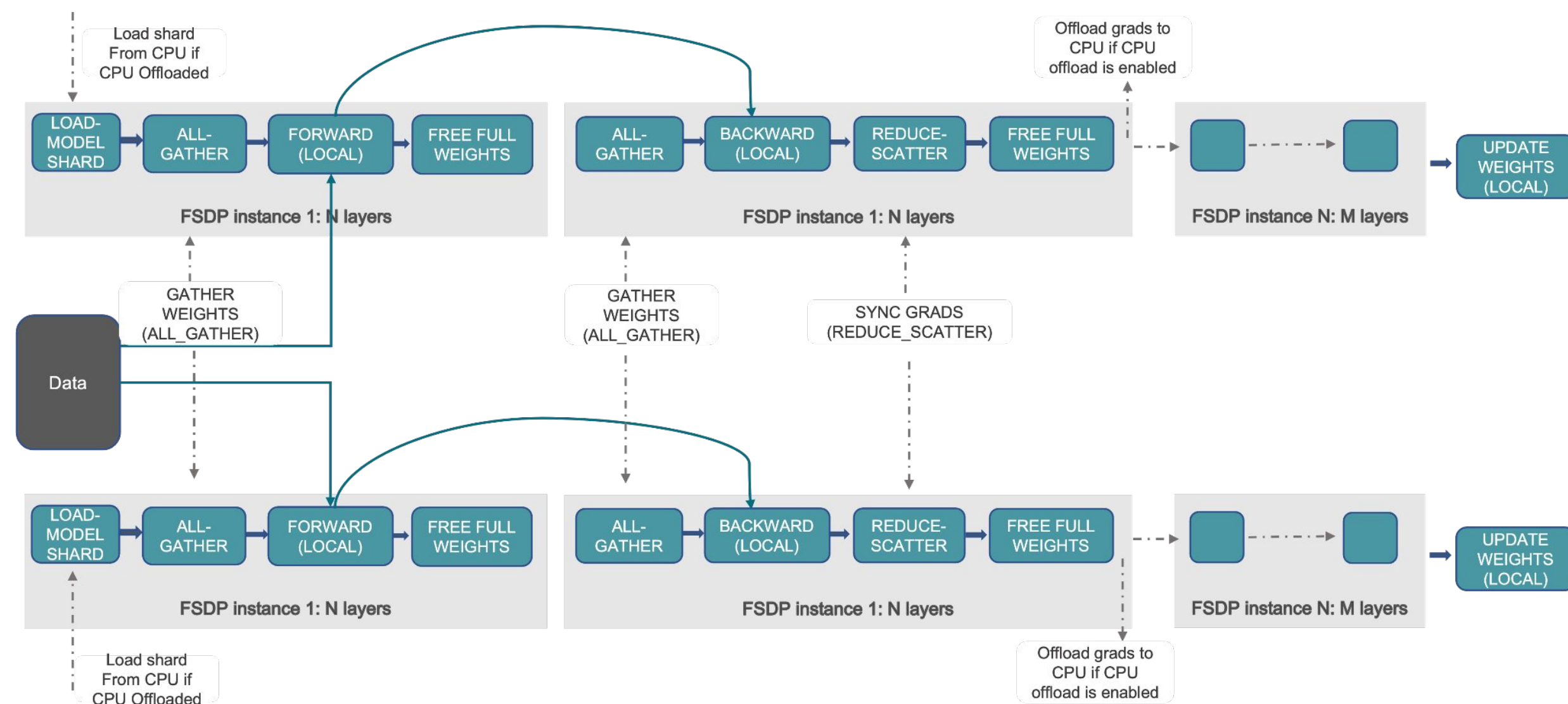
ozon{tech} $80\text{GB}/4.2\text{Bytes} = 19B$

$$80\text{GB}/2.2\text{Bytes} = 36B$$

$$80\text{GB}/0.25\text{Bytes} = 320B_{97}$$

Резюме сегодняшней лекции

- Виды параллелизма
- Коллективные операции (communication primitives)
 - Ring AllReduce, Recursive Halving AllReduce
- ZeRO-1,2,3
- FSDP



Ссылки

- TSM: Temporal Shift Module for Efficient Video Understanding [Lin et al. 2019]
- Scaling Distributed Machine Learning with the Parameter Server [Li et al. 2014]
- Optimization of collective communication operations in MPICH [Rajeev et al. 2005]
- Colossal-AI: A Unified Deep Learning System For Large-Scale Parallel Training [Li et al. 2021]
- ZeRO: Memory Optimizations Toward Training Trillion Parameter Models [Rajbhandari, et al, 2019]